

Télécom Nancy

Notes de cours du module Graphe et Recherche Opérationnelle (alias GRO) - partie 1

Bruno Pinçon, Jean-François Scheid

*Suite à la réforme en cours des enseignements à l'école, il a été décidé que le module GRO serait séparé en deux parties, l'une obligatoire et l'autre optionnelle. Ce polycopié et les fiches de TD et TP sont issus d'une réflexion sur la réorganisation de ce module. La partie obligatoire n'est pas la première partie de l'ancien module mais une sélection des thèmes abordés dans l'ancien module de sorte que l'élève ait quand même une bonne idée de ce que l'on peut faire avec les techniques de la programmation linéaire s'il ne suit que la partie obligatoire (aka GRO-1) et surtout puisse être capable d'utiliser ces techniques en pratique. Dans ce but GRO-1 comporte 3 séances de TP sur les 7 séances de 2h prévues pour les TD/TP dans laquelle on apprendra, entre autres, à utiliser le logiciel **Gurobi**¹ à travers sa (puissante) interface python. Pour que l'essentiel puisse tenir, une partie des détails concernant la programmation linéaire (chapitre 2) et la programmation linéaire en nombres entiers (chapitre 3) sera vue dans le module GRO-2. Enfin la partie graphe (chapitre 4) a été très allégée (idem elle sera complétée en GRO-2).*

Comment utiliser ce polycopié ? Il est fait pour être lu ! Sauf pour le premier cours, il vous sera demandé de lire et comprendre une partie du polycopié avant chaque cours, ce dernier devant être une séance de questions/réponses sur la partie que vous aurez à étudier. Une façon de procéder est d'utiliser un logiciel permettant de surligner/annoter/rajouter des pages d'un document pdf² et de l'utiliser lors de la lecture de ce document pour noter les points difficiles et poser ainsi vos questions lors du cours. Ce document comporte également des mini-exercices (dont vous trouverez le corrigé à la fin) permettant d'apprécier votre acquisition d'une partie du cours (il faudrait sans doute rajouter quelques exercices). Il doit aussi contenir quelques typos et erreurs malgré des relectures attentives, n'hésitez pas à m'en faire part ainsi que toute suggestion permettant de l'améliorer.

Ces notes (et les fiches de TD et TP) utilisent une grosse partie du matériel déjà écrit par mon collègue Jean-François Scheid, matériel que l'on a essayé tous les deux d'améliorer compléter, modifier par petits pas (on a jamais assez de temps...) au fur et à mesure des années. Cette nouvelle itération m'a quand même donné du fil à retordre et m'aura occupé une bonne partie de mes vacances d'été 2025.

1. Hélas non libre mais diablement efficace !

2. Le logiciel libre **xournal** ou **xournal++** est assez pratique pour cela.

Notations (essentiellement matricielles)

- Comme usuellement $\mathbb{R}^n = \mathbb{R} \times \cdots \times \mathbb{R}$ désigne l'espace (vectoriel) formé par les n -uplets de nombres réels. **Important** : un vecteur $x \in \mathbb{R}^n$ s'écrit par définition :

$$x = (x_1, x_2, \dots, x_n)$$

notation qui est souvent utilisée dans ce polycopié. Mais dès que l'on utilise un calcul matriciel, un tel vecteur est associé à une matrice colonne :

$$x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

Le produit scalaire canonique de $\mathbb{R}^n$ est noté :

$$(x|y) = \sum_{i=1}^n x_i y_i = x^\top y = y^\top x$$

les deux dernières notations ($x^\top y$ et $y^\top x$) utilisant la convention de considérer un vecteur de $\mathbb{R}^n$ comme une matrice colonne.

Remarque : l'espace $\mathbb{R}^n$ a selon le contexte un rôle d'espace de vecteurs et/ou aussi un rôle d'espace (affine) de "points".

- Si x et y sont deux vecteurs de même taille, l'écriture $x \leq y$ signifie que $x_i \leq y_i, \forall i$. De même si $a \in \mathbb{R}$, $x \leq a$ signifie que $x_i \leq a, \forall i$. Idem pour les autres opérateurs de comparaison $<, \geq$ et $>$.
- Les vecteurs de la base canonique de $\mathbb{R}^n$ sont notés $e^1, \dots, e^n$. Les composantes de e^i s'écrivent de façon compacte en utilisant le symbole de *Kronecker* $(e^i)_j = \delta_{i,j}$.
- $\mathbb{R}^{m \times n}$ (souvent noté $\mathcal{M}_{m,n}(\mathbb{R})$) désigne l'espace (vectoriel réel) des matrices à m lignes et n colonnes à coefficients réels.
- En général on utilise des lettres capitales pour désigner une matrice. Par exemple $A \in \mathbb{R}^{m \times n}$. De plus :
 - A_i ($i \in \llbracket 1, m \rrbracket$) désigne la i ème ligne de A
 - A^j ($j \in \llbracket 1, n \rrbracket$) désigne la j ème colonne de A
- Produit matriciel par blocs. Soit $A \in \mathbb{R}^{m \times n}$ et $B \in \mathbb{R}^{n \times p}$, le produit matriciel AB est alors bien défini et donne une matrice de $\mathbb{R}^{m \times p}$ avec :

$$(AB)_{i,j} = \sum_{k=1}^n a_{i,k} b_{k,j}, \quad \forall (i,j) \in \llbracket 1, m \rrbracket \times \llbracket 1, p \rrbracket$$

Remarque : avec les notations précédentes, on peut aussi écrire $(AB)_{i,j} = A_i B^j$. Cette formule du produit matriciel est aussi valable si on considère une décomposition par blocs des matrices A et B s'ils sont **compatibles**. Supposons qu'on décompose A et B de la façon suivante :

$$A = \left(\begin{array}{c|c|c} A_{1,1} & A_{1,2} & A_{1,3} \\ \hline A_{2,1} & A_{2,2} & A_{2,3} \end{array} \right), \quad B = \left(\begin{array}{c|c|c} B_{1,1} & B_{1,2} & B_{1,3} \\ \hline B_{2,1} & B_{2,2} & B_{2,3} \\ \hline B_{3,1} & B_{3,2} & B_{3,3} \end{array} \right),$$

avec $A_{i,j}$ de taille $m_i \times n_j$, $B_{i,j}$ de taille $n_i \times p_j$, $\sum_i m_i = m$, $\sum_k n_k = n$ et $\sum_j p_j = p$ alors le produit $C = AB$ peut s'écrire :

$$C = \left(\begin{array}{c|c|c} C_{1,1} & C_{1,2} & C_{1,3} \\ \hline C_{2,1} & C_{2,2} & C_{2,3} \end{array} \right) \text{ avec } C_{i,j} = A_{i,1} B_{1,j} + A_{i,2} B_{2,j} + A_{i,3} B_{3,j} = \sum_{k=1}^3 A_{i,k} B_{k,j}.$$

Un exemple souvent utilisé : on peut décomposer A en colonnes (i.e. par bloc de taille $m \times 1$) et B en lignes (par blocs de tailles $1 \times p$). On a alors :

$$AB = (A^1 | A^2 | \dots | A^n) \begin{pmatrix} \frac{B_1}{B_2} \\ \vdots \\ B_n \end{pmatrix} = \sum_{k=1}^n A^k B_k$$

Table des matières

1	Introduction	6
1.1	Qu'est ce que la recherche opérationnelle? Un peu d'histoire.	6
1.2	Un problème de production	9
1.2.1	Description et modélisation du problème	9
1.2.2	Résolution graphique	10
1.2.3	Observations	11
1.3	Un problème de logistique	11
1.4	Problèmes d'affectation	12
1.4.1	Un problème a priori simple	12
1.4.2	Un problème plus difficile : l'affectation des cours d'ouverture du Collégium INP de l'UL	13
1.5	Un problème posé sur un graphe	14
1.6	Formalisation	16
1.7	Quelques ouvrages de référence	17
2	Programmation linéaire	18
2.1	Différentes formes d'un PL	18
2.2	Rappels et/ou compléments de certaines notions mathématiques	20
2.2.1	Convexité	20
2.2.2	Équation et inéquation linéaires	21
2.2.3	Systèmes d'équations linéaires	22
2.2.4	Polyèdres (systèmes d'inéquations linéaires)	23
2.3	Le théorème fondamental de la PL	23
2.4	Caractéristiques d'un PL en forme standard	26
2.4.1	Solutions de base	26
2.4.2	Écriture du P.L. en fonction des variables hors-base	30
2.5	L'algorithme du simplexe (primal)	30
2.5.1	Une étape du simplexe	31
2.5.2	L'algorithme complet (avant les détails fâcheux) et un exemple	34
2.5.3	Convergence de l'algorithme du simplexe	36
3	Introduction à la résolution des PLNE et PLMI	40
3.1	Une technique dont on va s'inspirer	40
3.2	Optimiser sur l'enveloppe convexe	45
3.3	La première méthode générale (coupes de Gomory)	46
3.4	Une méthode de séparation et d'évaluation (branch and bound) basée sur les coupes entières	47
3.5	Les méthodes modernes	48

4	Graphes	50
4.1	Définitions	50
4.2	Représentation d'un graphe, matrices associées à un graphe	51
4.2.1	Listes d'adjacence	51
4.2.2	Matrice d'adjacence sommet-sommet	51
4.2.3	Matrice d'incidence sommet-arc	52
4.3	Codage informatique d'un graphe	53
4.4	Chemins et chaînes, cycles et circuits	55
4.5	Parcours de graphes	56
4.5.1	Le parcours en profondeur	56
4.5.2	Parcours en largeur	58
A	Correction des mini-exercices	60
A.1	mini-exercice 1	60
A.2	mini-exercice 2	60
A.3	mini-exercice 3	60
A.4	mini-exercice 4	61
A.5	mini-exercice 5	61
A.6	mini-exercice 6	62
A.7	mini-exercice 7	63
A.8	mini-exercice 8	63

Chapitre 1

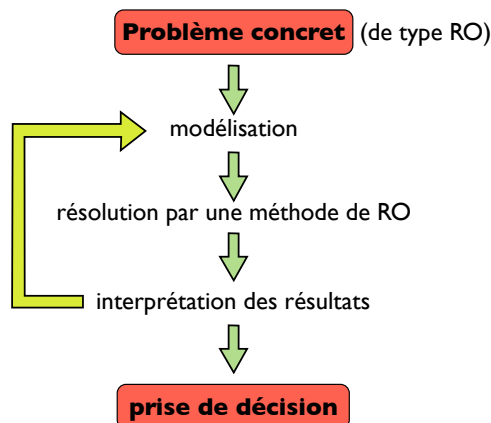
Introduction

1.1 Qu'est ce que la recherche opérationnelle ? Un peu d'histoire.

Le terme *recherche opérationnelle* (Operations Research en anglais) date de la seconde guerre mondiale où les anglo-saxons ont cherché à optimiser l'implantation de radars de défense anti-aérienne et à déterminer la taille optimale des convois d'approvisionnement. Une définition possible de la RO est la suivante :

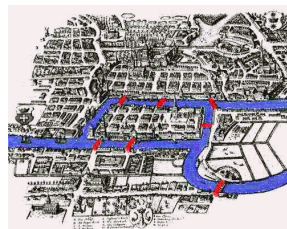
*Ensemble de méthodes d'analyse scientifique (maths et info) des phénomènes d'organisation qui traite de l'**optimisation** de l'architecture et du fonctionnement des systèmes (industriels, économiques, numériques ...).*

La RO est un outil d'aide à la décision :

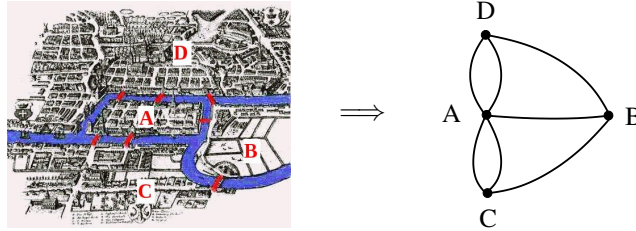


Un certain nombre de problèmes traités en RO sont posés naturellement sur des graphes (problème du voyageur de commerce, problèmes de flot maximal dans un graphe, etc.). Le concept de graphe (dont la naissance précède d'un peu plus de deux siècles celle de la RO) est du à *Léonard Euler* qui résout le problème *des ponts de Königsberg* en 1735.

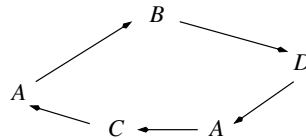
Dans cette ville, construite autour de deux îles, on se demande s'il existe un chemin permettant de revenir à son point de départ en empruntant une seule fois chaque pont de la ville.



La réponse est **non**. La preuve utilise un **graphe** : les arêtes du graphe représentent les ponts et les sommets les différents endroits de la ville (rives gauche et droite, les deux îles).



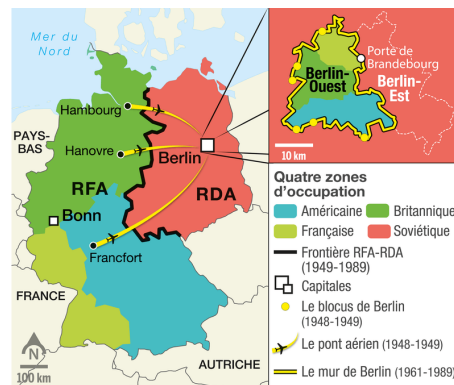
En effet si la réponse était oui alors il y aurait nombre pair d'arêtes reliées à chaque noeud d'où une contradiction. En effet (*idée de la preuve*) supposons qu'il existe un tel chemin, par exemple¹ :



Alors il est clair que le nombre d'arêtes (de ponts) relié un noeud quelconque doit être pair. Ici sur cet exemple, on aurait donc 4 arêtes reliant A et deux pour les autres noeuds.

L'exemple emblématique de la naissance de la RO

En 1947 *George Dantzig* invente la méthode du simplexe qui permet de résoudre des problèmes d'optimisation linéaire (ce terme sera défini ultérieurement). Suite à l'armistice de 1945, l'Allemagne est découpée en 4 zones d'occupation et en juin 1948 commence le blocus de Berlin-Ouest, les alliés de l'ouest (État-Unis, Grande Bretagne, France) décident de ravitailler Berlin-Ouest via des transports aériens intensifs :



À un moment il faut rationaliser et optimiser les convois d'avions étatsuniens et britanniques² pour approvisionner les 2 millions de personnes de Berlin-Ouest. Il y aura 1 avion toutes les 45s au plus fort du trafic avec 278000 vols en 15 mois. Il a été établi qu'il fallait envoyer chaque jour :

646 tonnes de farine et blé, 125 tonnes d'autres céréales, 64 tonnes de graisse/huile, 109 tonnes de viande et poisson, 180 tonnes de pommes de terre déshydratées, 180 tonnes de sucre, 11 tonnes de café, 19 tonnes de lait en poudre, 5 tonnes de lait entier pour les enfants, 3 tonnes de levure fraîche pour la boulangerie, 144 tonnes de légumes déshydratés, 38 tonnes de sel et 10 tonnes de fromage, 3475 tonnes de charbon et d'essence. Soit en simplifiant un peu environ 1500 tonnes de nourriture et 3500 tonnes pour le charbon et l'essence.

On dispose de 3 types d'avions :

1. Ce chemin n'est pas valable car le pont entre C et B n'est pas traversé de même que l'un des deux ponts entre A et D.
2. L'aviation française étant "occupée" en Indochine, la participation française aura été de construire en 90 jours un aéroport dans le secteur français de Berlin-Ouest.

- le type 1 qui peut embarquer une cargaison de 100 tonnes et pour lequel le vol vers Berlin (et le retour) coûte 1000 u.m. (unité monétaire) ; on peut disposer de 10 tels avions par jour ;
- le type 2 : cargaison de 200 tonnes, coût d'un vol 2000 u.m., disponibilité de 22 avions par jour ;
- le type 3 : cargaison de 150 tonnes, coût d'un vol 1200 u.m., disponibilité de 10 avions par jour.

Comme une cargaison ne mélange pas nourriture et charbon/essence, cela fait 6 variables à optimiser, on note X_{if} le nombre d'avions de type i avec une cargaison de nourriture et X_{ic} le nombre d'avions de type i avec une cargaison de charbon/essence.

Ce problème d'optimisation se pose ainsi : il faut minimiser la fonction (le coût total)

$$1000X_{1f} + 1000X_{1c} + 2000X_{2f} + 2000X_{2c} + 1200X_{3f} + 1200X_{3c} \quad (1.1)$$

sous les contraintes suivantes :

- il faut fournir au moins 1500 tonnes de nourriture, soit :

$$100X_{1f} + 200X_{2f} + 150X_{3f} \geq 1500 \quad (1.2)$$

- il faut fournir au moins 3500 tonnes de charbon et d'essence, soit :

$$100X_{1c} + 200X_{2c} + 150X_{3c} \geq 3500 \quad (1.3)$$

- on dispose d'un maximum de 10 avions de type 1, soit :

$$X_{1f} + X_{1c} \leq 10 \quad (1.4)$$

- on dispose d'un maximum de 22 avions de type 2, soit :

$$X_{2f} + X_{2c} \leq 22 \quad (1.5)$$

- on dispose d'un maximum de 10 avions de type 3, soit :

$$X_{3f} + X_{3c} \leq 10 \quad (1.6)$$

- pour éviter d'éventuelles solutions parasites non physiques, on impose :

$$X_{1f} \geq 0, X_{1c} \geq 0, X_{2f} \geq 0, X_{2c} \geq 0, X_{3f} \geq 0, X_{3c} \geq 0. \quad (1.7)$$

et en fait on doit obtenir des entiers.

Ce problème a été résolu à la main en utilisant la méthode du simplexe.

Remarque : l'approvisionnement de Berlin a subi divers changements allant vers une fourniture plus complète ; le problème précis exposé ci-avant correspond sans doute à une période donnée durant les 2 ans du blocage et provient du sujet de l'un des projets proposés par l'université de Virginie pour un module intitulé "Great Works in Computer Science" (voir <https://www.cs.virginia.edu/~evans/greatworks/>). Ce projet a été écrit par la professeure Wei Wang mais sans donner la source/référence correspondante pour ces données.

La RO à l'est ?

Si *George Dantzig* ouvre la voie avec la méthode du simplexe (et d'autres contributions ultérieures), un grand mathématicien russe, *Leonid Kantorovitch*, établit dès les années 1930, un certain nombre de concepts de la programmation linéaire puis écrit plusieurs ouvrages sur la planification. Il applique aussi ses connaissances lors du siège de *Léningrad/Saint-Petersbourg* en 1941 en étant responsable de la sécurité de la *Route de la vie* seul accès à cette ville pour les soviétiques³. Ses travaux ne seront connus à l'ouest qu'après la période stalinienne. Il obtient le prix nobel d'économie en 1975.

3. Il détermine la distance optimale à observer entre les voitures sur la surface gelée du lac Ladoga, en fonction de l'épaisseur de glace et de la température de l'air.

1.2 Un problème de production

Cet exemple simple (uniquement 2 variables ce qui permet de le résoudre graphiquement) va nous permettre de voir les principales caractéristiques d'un *programme linéaire*.

1.2.1 Description et modélisation du problème

Une usine fabrique deux produits P_1 et P_2 en utilisant une certaine quantité d'équipement, de main d'œuvre et de matières premières⁴ selon le tableau ci-dessous :

	P_1	P_2	disponibilité
équipement	3	9	81
main d'œuvre	4	5	55
matières premières	2	1	20

qui signifie que pour fabriquer une unité de produit P_1 , il faut 3 unités d'équipement, 4 de main d'œuvre et 2 de matière première. La disponibilité en équipement, main d'œuvre et matière première n'est pas infinie et est donnée dans la dernière colonne. Enfin la vente d'une unité de produit P_1 (resp. P_2) rapporte 6 (resp. 4) unités monétaires. On cherche à établir un plan (programme) de fabrication de façon à maximiser les bénéfices. On suppose que la "granularité" des deux produits est suffisamment fine pour qu'on puisse considérer les quantités x_1 et x_2 à produire comme des réels. On suppose aussi qu'on est sûr de vendre toute la production. Ainsi ce problème d'optimisation s'écrit :

$$\max_{x \in \mathcal{D}} f(x_1, x_2) = 6x_1 + 4x_2$$

où $\mathcal{D}$ est l'ensemble des points de $\mathbb{R}^2$ vérifiant :

$$\begin{aligned} 3x_1 + 9x_2 &\leq 81 \\ 4x_1 + 5x_2 &\leq 55 \\ 2x_1 + x_2 &\leq 20 \\ x_1 &\geq 0, x_2 &\geq 0 \end{aligned}$$

Un tel problème (de même que le précédent) s'appelle un programme linéaire (noté souvent PL) car la fonction à optimiser (minimiser ou maximiser) est linéaire en ses variables et les contraintes sont des équations linéaires et/ou des inéquations linéaires⁵.

En posant :

$$c = \begin{bmatrix} 6 \\ 4 \end{bmatrix}, b = \begin{bmatrix} 81 \\ 55 \\ 20 \end{bmatrix}, A = \begin{bmatrix} 3 & 9 \\ 4 & 5 \\ 2 & 1 \end{bmatrix}$$

on peut écrire ce problème de manière plus compacte⁶ :

$$\max_{x \in \mathcal{D}} f(x) = c^\top x, \mathcal{D} = \{x \in \mathbb{R}^2 : Ax \leq b \text{ et } x \geq 0\}$$

mini exercice 1 retour à Berlin-Ouest

Faire ce même travail (d'écriture compacte matricielle) sur le problème de l'optimisation du ravitaillement de Berlin-Ouest (cf (1.1),(1.2),(1.3),(1.4),(1.5),(1.6), (1.7)).

4. Pour rendre le problème plus simple, on suppose qu'une seule matière première est utilisée.

5. Dans cet exemple les contraintes sont toutes des inéquations linéaires.

6. Voir la page des notations pour la signification de $Ax \leq b$ et $x \geq 0$.

1.2.2 Résolution graphique

Cette résolution graphique consiste à délimiter le domaine $\mathcal{D}$ puis à tracer des lignes de niveaux $f(x_1, x_2) = Cte$ en faisant croître Cte jusqu'à arriver à une limite du domaine $\mathcal{D}$.

Dessiner le domaine $\mathcal{D}$

- Clairement le domaine $\mathcal{D}$ se situe dans le quadrant positif du plan $\mathbb{R}^2$ ($x_1 \geq 0$ et $x_2 \geq 0$).
- La contrainte $3x_1 + 9x_2 \leq 81$ est équivalente à $x_2 \leq 9 - \frac{1}{3}x_1$. Il s'agit donc de tous les points du plan qui sont sous la droite $x_2 = 9 - \frac{1}{3}x_1$. Pour tracer cette droite on peut voir qu'elle passe par les points $(0; 9)$ et $(9; 6)$ par exemple.
- La contrainte $4x_1 + 5x_2 \leq 55$ est équivalente à $x_2 \leq 11 - \frac{4}{5}x_1$. Il s'agit donc de tous les points du plan qui sont sous la droite $x_2 = 11 - \frac{4}{5}x_1$. Cette droite passe par les points $(0; 11)$ et $(10; 3)$.
- Enfin la contrainte $2x_1 + x_2 \leq 20$ est équivalente à $x_2 \leq 20 - 2x_1$. Il s'agit de tous les points du plan qui sont sous la droite $x_2 = 20 - 2x_1$. Cette droite passe par les points $(5; 10)$ et $(10; 0)$.

Dessiner les lignes de niveaux $f(x_1, x_2) = Cte$

Ces lignes de niveaux constituent une famille de droites parallèles orthogonales au vecteur $\nabla f = (6; 4)$ qui est le gradient de f . En effet⁷, soit deux points $x = (x_1, x_2)$ et $x' = (x'_1, x'_2)$ de la droite de niveau Cte :

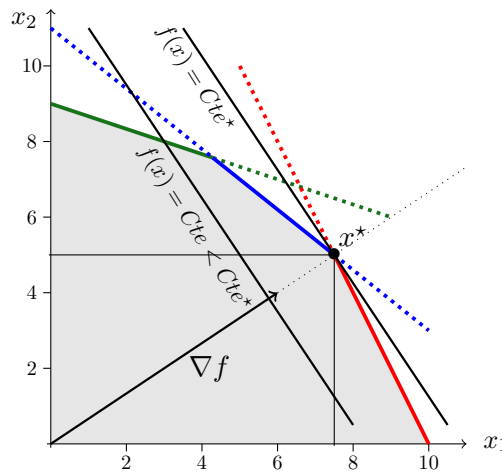
$$\begin{aligned} 6x_1 + 4x_2 &= Cte \\ 6x'_1 + 4x'_2 &= Cte \end{aligned}$$

En soustrayant il vient :

$$6(x'_1 - x_1) + 4(x'_2 - x_2) = (\nabla f | \vec{xx'}) = 0$$

Pour obtenir la droite limite, il suffit de placer le vecteur ∇f sur le dessin (ou tout vecteur $\alpha \nabla f$ avec $\alpha > 0$ pour respecter l'échelle du dessin) et d'y faire glisser une équerre (on obtient alors une ligne de niveau de f) dans le sens du gradient (donc vers une augmentation de f) jusqu'à la limite du domaine. Pour un problème de minimisation, il faudrait tracer $-\nabla f$.

Dessin



La résolution graphique nous donne $x^* = (7.5, 5)$ avec $f(x^*) = Cte^* = 6 \times 7.5 + 4 \times 5 = 65$.

7. Il y a un résultat plus général (vu en Math Num 2) : pour une fonction $f \in C^1(\mathbb{R}^n, \mathbb{R})$ l'hyperplan tangent en x_0 à l'ensemble de niveau $f(x) = f(x_0)$ est orthogonal à $\nabla f(x_0)$.

1.2.3 Observations

- Le domaine est un polyèdre convexe (ici un polygone puisque $n = 2$). Ce polygone est borné mais ce n'est pas toujours le cas.
- L'optimum est atteint en un sommet. Cependant en modifiant les coefficients c , de sorte que les lignes de niveau de f soient parallèles à l'une des arêtes, toute cette arête serait solution ; il n'y a ainsi pas toujours unicité de l'optimum mais il sera atteint au moins en un sommet.
- Un algorithme pourrait être de calculer les sommets puis d'évaluer la fonction f sur les sommets ; cependant comme nous le verrons le nombre de sommets explose avec le nombre de variables et le nombre de contraintes.
- Un autre algorithme pourrait être de partir d'un sommet et d'aller d'un sommet à un sommet voisin (c'est à dire deux sommets partageant une arête) de sorte que la fonction augmente (pour un max) ou diminue (pour un min), etc. jusqu'à arriver à l'optimum (ou un optimum) évitant ainsi de parcourir trop de sommets. C'est **la méthode du simplexe**.
- Enfin d'autres méthodes essaient d'aller plus directement à l'optimum en passant par l'intérieur du domaine, ce sont **les méthodes de points intérieurs** dont la première a été publiée par *Narendra Karmarkar* en 1984.

1.3 Un problème de logistique

Une entreprise dispose de plusieurs entrepôts contenant chacun un certain nombre de containers. Plusieurs magasins commandent des containers. On connaît le coût de transport de chaque dépôt aux magasins.



Quelle est l'organisation des livraisons des containers pour minimiser le coût total de transport ?

Un exemple

On considère 3 magasins (M_1, M_2, M_3) et deux entrepôts (E_1, E_2) avec les données suivantes⁸ :

		disponibilité entrepôts			
		M_1	M_2	M_3	↓
E_1		5	3	4	8
E_2		6	7	2	9
demande magasins	→	4	5	8	

On utilise les (6) variables x_{ij} , $1 \leq i \leq 2$ et $1 \leq j \leq 3$ où x_{ij} est le nombre de containers qui seront acheminés de l'entrepôt E_i au magasin M_j . La fonction à minimiser est le coût du transport :

$$\min_{\mathbf{x}} (F(\mathbf{x}) = 5x_{11} + 3x_{12} + 4x_{13} + 6x_{21} + 7x_{22} + 2x_{23})$$

8. La case (E_i, M_j) est le coût de transport d'un container de l'entrepôt E_i vers le magasin M_j .

avec les contraintes suivantes :

$$\begin{aligned}
x_{11} + x_{21} &= 4 && \text{(demande du magasin } M_1) \\
x_{12} + x_{22} &= 5 && \text{(demande du magasin } M_2) \\
x_{13} + x_{23} &= 8 && \text{(demande du magasin } M_3) \\
x_{11} + x_{12} + x_{13} &\leq 8 && \text{(disponibilité entrepôt } E_1) \\
x_{21} + x_{22} + x_{23} &\leq 9 && \text{(disponibilité entrepôt } E_2) \\
x_{ij} &\text{ entiers, } \forall i, j
\end{aligned}$$

Ce problème est de nouveau un PL (programme linéaire) avec à la fois des contraintes d'égalité et d'inégalité mais ici les variables sont clairement des entiers, on parle alors de PLNE (*programmation linéaire en nombres entiers*). Certains problèmes peuvent contenir à la fois des variables continues (réelles) et entières, on parle alors de PL mixtes (PLMI). Dès que certaines variables doivent être entières, cela peut compliquer fortement la résolution.

1.4 Problèmes d'affectation

1.4.1 Un problème a priori simple

n tâches doivent être affectées à n machines (chaque tâche doit être effectuée par une seule machine et chaque machine n'effectue qu'une seule tâche). Le coût d'exécution de chaque tâche par chacune des machines est connu, la tâche i exécutée sur la machine j à un coût égal à c_{ij} . *Comment trouver l'affectation qui minimise le coût total?* Ce genre de problème ressemble plus à un problème de combinatoire, voici une première approche.

Résolution en utilisant la force brute

Il y a $n!$ affectations possibles, chaque affectation peut être représentée par une permutation σ , par exemple $\sigma = (1, 4, 2, 3)$ signifierait que la tâche 1 est effectuée par la machine 1, la tâche 2 par la machine 4, la tâche 3 par la machine 2 et la tâche 4 par la machine 3 (d'une manière générale σ_i signifierait que la tâche i est effectuée par la machine σ_i). L'algorithme se présenterait alors sous la forme suivante :

```

 $F^* \leftarrow +\infty$ 
pour  $\sigma \in \Sigma_n$  :           on itère sur toutes les permutations
     $F \leftarrow \sum_{i=1}^n c_{i\sigma_i}$    calcul du coût associé à la permutation courante
    si  $F < F^*$  :
         $F^* \leftarrow F$  ;  $\sigma^* \leftarrow \sigma$ 

```

En dehors bien sûr de l'explosion du nombre de possibilités ($n!$), il reste aussi à générer les $n!$ permutations⁹.

Formuler le problème comme un PL (en variables binaires)

On utilise les n^2 variables binaires x_{ij} avec $x_{ij} = 1$ si la tâche i est effectuée par la machine j et $x_{ij} = 0$ sinon. Il faut alors minimiser la fonction :

$$F(x) := \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

9. Ce qui peut être réalisé sans utiliser beaucoup de mémoire avec l'algorithme de *Steinhaus-Johnson-Trotter*.

sous les $(2n)$ contraintes d'égalité suivantes :

$$\begin{aligned} \forall i \in \llbracket 1, n \rrbracket : \sum_{j=1}^n x_{ij} &= 1 \text{ (chaque tâche } i \text{ est effectuée par une seule machine)} \\ \forall j \in \llbracket 1, n \rrbracket : \sum_{i=1}^n x_{ij} &= 1 \text{ (chaque machine } j \text{ effectue une seule tâche)} \\ x_{ij} &\in \{0, 1\}, \forall (i, j) \in \llbracket 1, n \rrbracket^2 \end{aligned}$$

On verra (en GRO-2) qu'on a en fait pas besoin d'imposer que les variables soient binaires pour ce problème. On peut simplement les considérer comme des variables réelles positives ou nulles et utiliser la méthode du simplexe pour trouver la solution¹⁰. Néanmoins il existe une méthode particulière (appelée *méthode hongroise*) qui permet de résoudre très rapidement ce problème.

1.4.2 Un problème plus difficile : l'affectation des cours d'ouverture du Collégium INP de l'UL

Lorsque ces cours d'ouverture ont commencé (en 2014-2015), notre école a été chargée de réaliser un logiciel pour affecter les élèves aux cours d'ouverture selon certaines règles. Telecom Nancy Service a développé une interface web permettant à chaque élève de faire ses choix parmi les cours proposés et de les stocker dans une base de données et Jean-François Scheid a écrit la modélisation sous forme de PL ainsi qu'un code python permettant de récupérer les données d'affectation et de résoudre ce problème avec un solveur adapté. La version de 2014-2015 proposait 78 cours à 1578 élèves, soit plus de 123 000 variables :

$$x_{ij} = \begin{cases} 1 & \text{si l'élève } i \text{ suit le cours } j \\ 0 & \text{sinon} \end{cases}$$

Voici quelques éléments d'une modélisation simplifiée pour vous donner l'idée générale :

- Chaque élève doit suivre 3 cours et donne une liste de préférence d'au moins 6 cours (il "coche" six cours sans préférence entre ces 6 cours). La fonction à maximiser correspondait à la satisfaction collective (globale) :

$$\max_{x_{ij}} \sum_{i,j} c_{ij} x_{ij} \quad \text{où } c_{ij} = \begin{cases} 1 & \text{si l'élève } i \text{ a choisi le cours } j \text{ dans sa liste de préférence} \\ 0 & \text{sinon} \end{cases}$$

- Parmi les contraintes :

1. chaque élève doit suivre exactement 3 cours :

$$\sum_j x_{ij} = 3, \quad \text{pour tout } i$$

2. les cours ont un seuil maximum d'élèves (le cours j a une capacité maximale de N_j élèves) et aussi un seuil d'ouverture (il faut au moins n_j élèves pour que le cours ouvre), d'où les inégalités suivantes :

$$\sum_i x_{ij} \leq N_j, \quad \text{pour tout } j \quad \text{et} \quad \sum_i x_{ij} \geq n_j, \quad \text{pour tout } j$$

3. on aimerait faire en sorte d'attribuer à chaque élève 3 cours qu'il a explicitement choisi :

$$\sum_j c_{ij} x_{ij} \geq 3, \quad \text{pour tout } i$$

mais ce seuil (ci-dessus de 3) peut-être abaissé à 2 (ce qui est souvent le cas) s'il n'y a pas de solution.

10. Ce qui est bien plus performant que la méthode force brute.

Cette année là, le solveur CPLEX¹¹ a été utilisé et le problème d'affectation fut résolu en moins de 2 mn sur une station de travail Intel Xeon 2.26 GHz (4 cores/8 threads) comportant 24GB RAM. Le taux de satisfaction a été strictement supérieur à 97%.

1.5 Un problème posé sur un graphe

Soit un graphe non orienté $G = (S, \Gamma)$ où S est l'ensemble des sommets/noeuds et Γ l'ensemble des arêtes (une arête est alors une paire $\{i, k\}$ de noeuds de S). Si $\{i, k\} \in \Gamma$ on dit que le noeud k est un voisin du noeud i et inversement.

Le problème est d'attribuer une couleur à chaque sommet/noeud $s \in S$ du graphe avec les contraintes suivantes :

- *deux sommets s et s' reliés par une arête (donc $\{s, s'\} \in \Gamma$) doivent être coloriés avec deux couleurs différentes $col(s) \neq col(s')$.*
- *on veut utiliser un minimum de couleurs.*

Ce problème (qui a l'air purement théorique de prime abord) a plusieurs applications, voir par exemple la page wikipedia https://fr.wikipedia.org/wiki/Coloration_de_graphe.

Quelques définitions

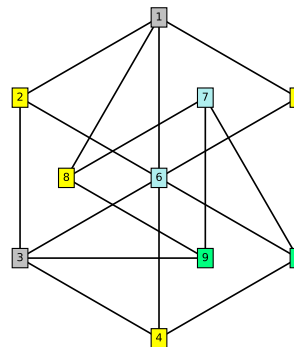
- Le nombre minimal de couleurs est appelé *nombre chromatique du graphe* et noté $\chi(G)$.
- On appelle degré d'un sommet x le nombre de ses voisins :

$$deg(x) := card(\{y \in S : \{x, y\} \in \Gamma\}).$$

- On note :

$$\Delta G := \max_{x \in S} deg(x)$$

le degré maximal du graphe G .



Coloration optimale du graphe de *Golomb*

mini exercice 2 Graphe de Golomb

Déterminer les ensembles S et Γ , ainsi que le degré maximal ΔG et le nombre chromatique de ce graphe.

Solution obtenue par une heuristique

Ce problème est très difficile à résoudre (problème *NP* complet sauf cas particuliers) et pour les applications pratiques on peut souvent se contenter d'une solution heuristique. Qu'est-ce qu'une *heuristique* ?

*Une heuristique consiste à utiliser des principes assez simples¹² pour obtenir une solution qui ne sera pas forcément optimale. Il est sous-entendu que le temps calcul d'une heuristique doit être assez faible vis à vis de celui d'une méthode exacte existante pour le problème qu'on cherche à résoudre. Nous verrons d'ailleurs (déjà sur cet exemple) qu'on peut aider une méthode exacte grâce à une heuristique. Comme une heuristique utilise peu de temps CPU, on a le concept de **méta-heuristique** qui consiste à en combiner plusieurs.*

11. Solveur développé par IBM pour l'optimisation linéaire (et aussi quadratique) avec variables continues ou entières ou les deux.

12. Par exemple des principes appelés "gloutons" et qui correspondent à une sorte d'optimisation locale.

Voici une heuristique assez simple, pour ce problème de coloration. Les couleurs sont assimilées à des entiers positifs ou nuls, -1 signifiant qu'un sommet n'a pas encore été colorié.

pour $s \in S$: *parcours des sommets de S*

- (i) on regarde l'ensemble des couleurs déjà attribuées aux voisins de s ;
- (ii) on en déduit le plus petit entier naturel qui n'appartient pas à cet ensemble ;
- (iii) on attribue cette couleur à s .

Modélisation comme un P.L.

Cette modélisation n'est pas si simple. Soit m un majorant du nombre chromatique (obtenu avec l'heuristique précédente ou plusieurs heuristiques) et n le nombre de sommets de G , on considère les variables binaires suivantes :

1. $x_{i,j}$ valant 1 si on attribue la couleur j au sommet i (0 sinon) ($i \in \llbracket 0, n-1 \rrbracket$, $j \in \llbracket 0, m-1 \rrbracket$) ;
2. y_j valant 1 si la couleur j est utilisée (0 sinon), $0 \leq j \leq m-1$.

A priori la fonction à minimiser est le nombre de couleurs soit :

$$\sum_{j=0}^{m-1} y_j$$

Pour les contraintes :

- Il faut exprimer qu'un sommet est colorié par une couleur et une seule, soit :

$$\forall i \in \llbracket 0, n-1 \rrbracket : \sum_{j=0}^{m-1} x_{ij} = 1$$

ce qui donne n contraintes d'égalité.

- Il faut exprimer le fait que deux sommets voisins i et k (donc partageant une arête) ne doivent pas être coloriés avec la même couleur j . Ceci peut s'exprimer avec l'inégalité suivante :

$$\forall \{i, k\} \in \Gamma : x_{i,j} + x_{k,j} \leq y_j, \quad j = 0, 1, \dots, m-1$$

si a le nombre d'arêtes cela nous donne $a \times m$ inégalités.

Dans cette modélisation n'importe quelle permutation des couleurs d'une solution optimale est encore une solution optimale. Pour diminuer le nombre de solutions, on peut vouloir forcer l'utilisation de la couleur j avant la couleur $j+1$, soit :

$$y_0 \geq y_1 \geq y_2 \geq \dots \geq y_{m-1}$$

ce qui rajoute $m-1$ inégalités. Une astuce pour éviter cela est de privilégier les couleurs dans la fonction objectif en utilisant (par exemple) :

$$\sum_{j=0}^{m-1} (j+1)y_j$$

En effet supposons que le nombre chromatique soit égal à $m-1$ alors la solution (concernant l'utilisation des couleurs) $y^* = (1, 1, \dots, 1, 0)$ donne bien la plus petite valeur de cette fonction par rapport à celle qu'on obtient avec toute permutation des composantes de y^* .

Remarque : on voit ici l'importance d'utiliser une heuristique pour diminuer le nombre de variables et de contraintes si on veut résoudre exactement ce problème par une approche PLNE.

1.6 Formalisation

Presque tous les problèmes mentionnés précédemment et ceux abordés dans les TD peuvent se formaliser de la façon suivante¹³ :

$$\max \text{ ou } \min \{f(x), x \in X\} \quad (1.8)$$

- $f : X \rightarrow \mathbb{R}$ est la fonction **objectif** qui peut être linéaire, quadratique, nonlinéaire...
- X est l'ensemble des solutions possibles dites **réalisables** (les contraintes). L'ensemble X est soit infini non dénombrable (pas de contraintes d'intégrité sur les variables) soit fini (les variables doivent être entières, voire binaires) mais en général de très grande taille. Il y a aussi des problèmes mixtes où seulement une partie des variables doivent être entière ou binaire.

*Insistons sur ce point de vocabulaire particulier, dans ce contexte une **solution réalisable** est juste un point x satisfaisant les contraintes (i.e. $x \in X$) et pas un éventuel maximiseur ou minimiseur de f sur X .*

- On dit que f est majorée sur X si il existe $M \in \mathbb{R}$ tel que $f(x) \leq M, \forall x \in X$ (M est appelée *majorant* de f sur X) ; le plus petit des majorants¹⁴ est appelé *borne supérieure* ou *supremum* de f sur X et est noté :

$$m := \sup_{x \in X} f(x) \text{ ou } \sup\{f(x), x \in X\}$$

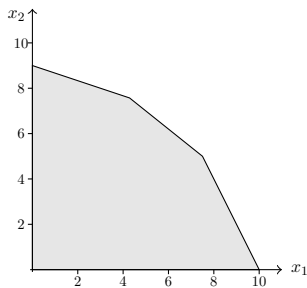
Quand f n'est pas majorée sur X , on note $\sup_{x \in X} f(x) = +\infty$. Si f est majorée sur X et qu'il existe $x^* \in X$ tel que $f(x^*) = m$, on dit que m est le maximum de f sur X et que x^* est un point de maximum de f ou *maximiseur* de f mais par abus de langage on dit souvent que x^* est un maximum de f . Même notions dans l'autre sens (minorant, borne inférieure/infinum, minimum, minimiseur).

Dans ce cours on va surtout s'intéresser aux programmes linéaires (PL) qui sont caractérisés par :

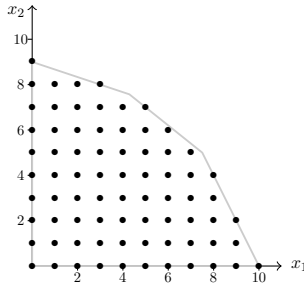
- la linéarité de la fonction objectif par rapport aux variables ;
- la linéarité des contraintes (égalités ou inégalités) aux variables.

Lorsque toutes les variables sont entières (ou binaires) on parle de programme linéaire en nombre entier (PLNE) et lorsqu'une partie des variables seulement doivent être entières (et les autres réelles) on parle de programme linéaire mixte (PLMI).

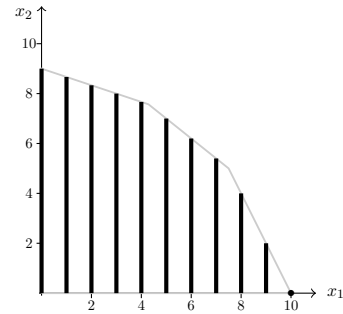
Lorsque les variables sont continues, le domaine formé par les contraintes est un polyèdre convexe (cette notion sera précisée dans le chapitre suivant). En reprenant l'exemple du problème de production voici à quoi peuvent ressembler, en comparaison avec le domaine continu $\mathcal{D}$, le domaine d'un PLNE et d'un PLMI :



$\mathcal{D}$



$\mathcal{D} \cap \{(x_1, x_2) \in \mathbb{N} \times \mathbb{N}\}$



$\mathcal{D} \cap \{x_1 \in \mathbb{N}\}$

13. En toute rigueur (voir plus loin), il faudrait écrire sup à la place de max et inf à la place de min, on suit ici la tradition des cours d'optimisation.

14. Quand l'espace but de f est $\mathbb{R}$ et que f est majorée, ce suprémum existe toujours (ce qui n'est pas forcément le cas si, par exemple, l'espace but est $\mathbb{Q}$).

1.7 Quelques ouvrages de référence

En français :

- [1] *Précis de recherche opérationnelle – Méthodes et exercices d’application*, Faure, Lemaire, Picouleau, 2008. Ouvrage de référence, un ”classique”.
- [2] *Exercices et problèmes résolus de recherche opérationnelle : Tome 1 (Graphes : leurs usages, leurs algorithmes), Tome 3 (Programmation linéaire et extensions, problèmes classiques)*. Roseaux : Billionnet, Carlier, Chrétienne, Lemaire, Faure ; 1985/1998. Contient des exercices classiques en lien avec [1].
- [3] *Optimisation discrète – De la modélisation à la résolution par des logiciels de programmation mathématique*, Billionnet, 2007. Ouvrage plus récent qui contient un exposé de méthodes plus modernes que [1]. Ouvrage intéressant.
- [4] *Recherche opérationnelle pour ingénieurs Tome I et II*, Hêche, Liebling, de Werra ; 2003. Malgré son titre (!), contenu assez mathématique ; contient des démonstrations intéressantes.
- [5] *Introduction à l’algorithmique*, Corman, Leiserson, Rivest, 2002. Ouvrage de référence en algorithmique et structures de données et, en particulier, décrit de nombreux algorithmes opérant sur des graphes. Style parfois un peu lourd cependant.

En anglais :

- [6] *Linear programming 1 : Introduction*, G. Dantzig, M. Thapa, 1997. et [7] *Linear programming 2 : Theory and Extensions*, G. Dantzig, M. Thapa, 2003. Ouvrage très complet.
- [8] *Linear programming*, V. Chvátal, Freeman, New York 1983. Un grand classique qui se lit facilement. Cet ouvrage a popularisé l’utilisation des dictionnaires à la place des tableaux de simplexe (merci monsieur Chvátal!). Petit bémol V. Chvátal n’est pas un fan de l’interprétation géométrique contrairement aux deux auteurs de ce polycopié ;-).
- [9] *Integer Programming*, M. Conforti, G. Cornuéjols, G. Zambelli, Springer, 2014. L’ouvrage de référence actuel pour la programmation linéaire en nombres entiers. Ouvrage très intéressant mais pas toujours très pédagogique.

Chapitre 2

Programmation linéaire

2.1 Différentes formes d'un PL

Nous avons vu qu'un PL correspond à minimiser ou maximiser une fonction *linéaire* de n variables, $f(x) = c^\top x = (c|x)$ sur un domaine $\mathcal{D}$ (dont nous verrons qu'il s'agit d'un polyèdre convexe) défini par des égalités et/ou des inégalités *linéaires par rapport aux variables*. Selon le type des contraintes on parle d'un PL :

- en **forme canonique pure** lorsque toutes les variables sont positives et que les contraintes additionnelles sont toutes des inégalités, c'est à dire lorsque

$$\mathcal{D} = \{x \in \mathbb{R}^n : Ax \leq b \text{ et } x \geq 0\}$$

avec $A \in \mathbb{R}^{m \times n}$ et $b \in \mathbb{R}^m$ pour m inégalités.

Il est clair que les contraintes $x \geq 0$ pourraient être incluses dans les inégalités additionnelles $Ax \leq b$ mais la positivité des variables jouant un rôle important, on préfère scinder les inégalités en deux parties distinctes.

- en **forme standard** lorsque, de même, toutes les variables sont positives, mais les contraintes additionnelles sont toutes des égalités, c'est à dire lorsque

$$\mathcal{D} = \{x \in \mathbb{R}^n : Ax = b \text{ et } x \geq 0\}$$

avec $A \in \mathbb{R}^{m \times n}$ et $b \in \mathbb{R}^m$ lorsqu'on a m contraintes d'égalité. La forme standard est importante car elle permet d'utiliser la méthode du simplexe.

- en **forme mixte** lorsqu'on a, à la fois des contraintes d'égalité et d'inégalité, et/ou que toutes les variables ne sont pas forcément astreintes à être positives. Dans ce cas :

$$\mathcal{D} = \{x \in \mathbb{R}^n : Ax = b \text{ et } A'x \leq b' \text{ et } x_i \geq 0 \text{ pour } i \in I\}$$

avec $A \in \mathbb{R}^{m_e \times n}$, $b \in \mathbb{R}^{m_e}$, $A' \in \mathbb{R}^{m_i \times n}$, $b' \in \mathbb{R}^{m_i}$ lorsqu'on a m_e contraintes d'égalité et m_i contraintes d'inégalité et $I \subset \llbracket 1, n \rrbracket$ est l'ensemble des indices correspondants aux variables positives (les autres variables sont dites "variables non signées").

Remarques :

- Lorsqu'une inégalité est donnée sous la forme $a_1x_1 + \dots + a_nx_n \geq b$, on se ramène à une inéquation de la forme $a'_1x_1 + \dots + a'_nx_n \leq b'$ en multipliant la première par -1 .
- Lorsqu'une variable est bornée $\alpha \leq x_i \leq \beta$, en posant $x'_i := x_i - \alpha$, on a $0 \leq x'_i \leq \beta - \alpha$. On se ramène donc à une variable positive, la partie $x'_i \leq \beta - \alpha$ est à ajouter aux inéquations¹.
- Pour une variable majorée $x_i \leq \alpha$, on pose $x'_i := \alpha - x_i$.

À l'aide de ces artifices, on peut toujours se ramener à la forme mixte précédente.

1. Il est néanmoins possible d'adapter la méthode du simplexe à ce cas.

Passage en forme standard

Il y a deux ingrédients pour passer vers la forme standard :

1. une inégalité $a_1x_1 + \dots + a_nx_n \leq b$ est transformée en égalité en ajoutant une variable positive e appelée *variable d'écart*, en effet :

$$\sum_{j=1}^n a_j x_j \leq b \iff \exists e \geq 0 : \sum_{j=1}^n a_j x_j + e = b.$$

On doit ajouter autant de variables d'écart qu'il y a d'inégalités. Si on reprend l'exemple donné au début, il faut rajouter 3 variables d'écart pour obtenir la forme standard :

$$\begin{array}{rrrrr} 3x_1 & +9x_2 & +e_1 & & = & 81 \\ 4x_1 & +5x_2 & & +e_2 & = & 55 \\ 2x_1 & +x_2 & & & +e_3 & = 20 \\ x_1 \geq 0, & x_2 \geq 0, & e_1 \geq 0, & e_2 \geq 0, & e_3 \geq 0 & \end{array}$$

On obtient donc un problème posé dans $\mathbb{R}^5$ au lieu de $\mathbb{R}^2$ où les variables d'écart n'interviennent pas (directement) dans la fonction. En posant :

$$X = \begin{bmatrix} x_1 \\ x_2 \\ e_1 \\ e_2 \\ e_3 \end{bmatrix}, \quad C = \begin{bmatrix} 6 \\ 4 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathcal{A} = \begin{bmatrix} 3 & 9 & 1 & 0 & 0 \\ 4 & 5 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 & 1 \end{bmatrix} = [A \mid I_3]$$

Le problème s'écrit :

$$\max_{X \in \mathcal{D}'} \tilde{f}(X) = C^\top X, \text{ où } \mathcal{D}' = \{X \in \mathbb{R}^5, X \geq 0 \text{ et } \mathcal{A}X = b\}$$

Il peut paraître bizarre de passer d'un problème posé dans $\mathbb{R}^2$ à un problème posé dans $\mathbb{R}^5$, cependant on peut montrer que le domaine $\mathcal{D}'$ est un polyèdre dans $\mathbb{R}^5$ dont la "dimension volumique" est 2 (il s'agit d'un sous-espace affine de dimension 2 "limité" par les inégalités $X \geq 0$) et que la projection orthogonale de $\mathcal{D}'$ sur les 2 premières coordonnées nous redonne $\mathcal{D}$.

2. Pour chaque variable non signée x_i on se ramène à utiliser deux variables positives $u_i \geq 0$ et $v_i \geq 0$ telles que $x_i = u_i - v_i$. Un choix possible est de définir $u_i := (x_i)^+$ et $v_i := (x_i)^-$ mais peu importe² puisqu'après avoir résolu le problème, on obtient $x_i^* = u_i^* - v_i^*$. Nous verrons des exemples en TD mais voici un exercice dont vous trouverez le corrigé en annexe.

mini exercice 3 Utilisation des 2 ingrédients pour passer en forme standard

Mettre sous forme standard le PL :

$$\begin{array}{l} \max_{x \in \mathbb{R}^2} [F(x_1, x_2, x_3) = 2x_1 + 5x_2 + 9x_3]. \\ \text{sous les contraintes :} \\ \left\{ \begin{array}{l} 3x_1 + 7x_2 + 3x_3 \leq 12 \\ 4x_1 + 5x_2 + 6x_3 = 15 \end{array} \right. \\ \left\{ \begin{array}{l} x_1 \geq 0, x_2 \geq 1, x_3 \text{ non signée} \end{array} \right. \end{array}$$

[À voir :] Ajouter quelques exemples pour comprendre les variable d'écart, par exemple en 1d avec le domaine $x \in [0, 1]$ ce qui peut permettre de représenter une fonction et en 2d avec le triangle unité ?

2. La fonction $x \mapsto (x)^+$ définie par $(x)^+ = x$ si $x \geq 0$ et $(x)^+ = 0$ si $x < 0$ est appelée partie positive et la fonction $x \mapsto (x)^-$ définie par $(x)^- = -x$ si $x \leq 0$ et $(x)^- = 0$ si $x > 0$, partie négative.

Passage de la forme standard à la forme canonique

Pour cela il suffit d'écrire les m contraintes d'égalité $Ax = b$ comme $2m$ contraintes d'inégalité :

$$Ax = b \iff \begin{cases} Ax \leq b \\ Ax \geq b \end{cases} \iff \begin{cases} Ax \leq b \\ -Ax \leq -b \end{cases} \iff \begin{pmatrix} A \\ -A \end{pmatrix} x \leq \begin{pmatrix} b \\ -b \end{pmatrix}$$

Maximiser ou Minimiser ?

Certains problèmes, comme celui de l'exemple précédent, conduisent naturellement à maximiser une fonction et d'autres à minimiser. Cependant peu importe puisque :

$$\sup\{f(x) : x \in E\} = -\inf\{-f(x) : x \in E\}.$$

2.2 Rappels et/ou compléments de certaines notions mathématiques

2.2.1 Convexité

Définition 1 ensemble convexe

Soit $E \subset \mathbb{R}^n$, on dit que E est convexe si et seulement si, étant donné deux points $x, y \in E$, le segment $[x, y]$ est lui-même inclus dans E , autrement dit :

$$\forall x, y \in E, \forall \lambda \in [0, 1], (1 - \lambda)x + \lambda y \in E$$

Plus généralement un sous-ensemble E d'un ensemble affine A est convexe si tout segment dont les extrémités sont des points de E est inclus lui-même dans E .

D'après cette définition, il est clair que l'intersection d'une famille arbitraire d'ensembles convexes $(E_i)_{i \in I}$ est elle-même un ensemble convexe (qui peut être vide).

Définition 2 enveloppe convexe

Soit $E \subset \mathbb{R}^n$, l'intersection de tous les ensembles convexes contenant E est donc le plus petit (au sens de l'inclusion) ensemble convexe contenant E . Cet ensemble, noté $\text{conv}(E)$, est appelé l'enveloppe convexe de E .

Définition 3 combinaison convexe

Un point $x \in \mathbb{R}^n$ est appelé combinaison convexe de points de $E \subset \mathbb{R}^n$ si il existe un nombre fini $x^1, \dots, x^p$ de points de E et des scalaires $\lambda_1, \dots, \lambda_p$ tels que :

$$x = \lambda_1 x^1 + \dots + \lambda_p x^p \text{ avec } \sum_{i=1}^p \lambda_i = 1 \text{ et } \lambda_1, \dots, \lambda_p \geq 0$$

Proposition 1 Une autre caractérisation de l'enveloppe convexe

Soit $E \subset \mathbb{R}^n$, on a le résultat suivant :

$$\text{conv}(E) = \{x \in \mathbb{R}^n : x \text{ est une combinaison convexe de points de } E\}$$

Preuve : appelons $cc(E)$ l'ensemble des combinaisons convexes de points de E . On a $cc(E) \subset \text{conv}(E)$ car tout sous-ensemble convexe contenant E contient les combinaisons convexes de points de E . D'autre part $cc(E)$ est lui-même un ensemble convexe. En effet soient $x, y \in cc(E)$, par définition de $cc(E)$:

$$\begin{aligned} x &= \sum_{i=1}^p \lambda_i x^i, \text{ avec } \sum_{i=1}^p \lambda_i = 1, \lambda_i \geq 0 \text{ et } x^i \in E \forall i \in \llbracket 1, p \rrbracket \\ y &= \sum_{j=1}^q \mu_j y^j, \text{ avec } \sum_{j=1}^q \mu_j = 1, \mu_j \geq 0 \text{ et } y^j \in E \forall j \in \llbracket 1, q \rrbracket \end{aligned}$$

Soit $\theta \in [0, 1]$, on a :

$$(1 - \theta)x + \theta y = \sum_{i=1}^p (1 - \theta)\lambda_i x^i + \sum_{j=1}^q \theta \mu_j y^j$$

clairement $(1 - \theta)\lambda_i \geq 0, \forall i$ et $\theta \mu_j \geq 0, \forall j$, de plus :

$$\sum_{i=1}^p (1 - \theta)\lambda_i + \sum_{j=1}^q \theta \mu_j = (1 - \theta) \sum_{i=1}^p \lambda_i + \theta \sum_{j=1}^q \mu_j = (1 - \theta) + \theta = 1$$

ainsi $(1 - \theta)x + \theta y$ est bien une combinaison convexe de $(p + q)$ points de E , donc $(1 - \theta)x + \theta y \in cc(E)$. D'où $conv(E) \subset cc(E)$ puisque $conv(E)$ est le plus petit ensemble convexe contenant E . $\square$

2.2.2 Équation et inéquation linéaires

On se place dans $\mathbb{R}^n$. Une équation du type :

$$a_1 x_1 + a_2 x_2 + \dots + a_n x_n = b \quad (2.1)$$

où les données sont le scalaire b et les scalaires $a_i, i = 1, \dots, n$ dont on suppose qu'au moins l'un d'entre eux est non nul ($a_{i^*} \neq 0$) correspond à un *hyperplan affine*³ $\mathcal{H}$ dans $\mathbb{R}^n$. On note $a := (a_1, \dots, a_n) \in \mathbb{R}^n$ et $x^* \in \mathbb{R}^n$ tel que toutes ses composantes sont nulles sauf $x_{i^*}^* = \frac{b}{a_{i^*}}$. Il est clair que $x^* \in \mathcal{H}$, puisqu'il vérifie l'équation (2.1) :

$$\sum_{i=1}^n a_i x_i^* = a_{i^*} x_{i^*}^* = a_{i^*} \frac{b}{a_{i^*}} = b$$

et on a alors l'interprétation suivante : $\mathcal{H}$ correspond également à l'ensemble des points $x \in \mathbb{R}^n$ tels que :

$$(a|x - x^*) = 0 \quad (\iff (a|x) - \underbrace{(a|x^*)}_{=b} = 0 \iff a_1 x_1 + a_2 x_2 + \dots + a_n x_n = b)$$

c'est à dire tels que le vecteur $x - x^*$ est orthogonal au vecteur a qu'on appelle "normale" de $\mathcal{H}$.

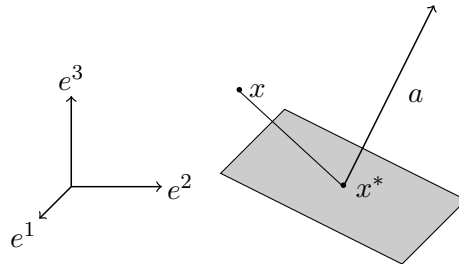
De même l'inéquation :

$$a_1 x_1 + a_2 x_2 + \dots + a_n x_n \geq b \quad (2.2)$$

correspond à l'ensemble des points $x \in \mathbb{R}^n$ tels que :

$$(a|x - x^*) \geq 0$$

Un tel ensemble est appelé "demi-espace" de $\mathbb{R}^n$, il correspond à tous les points "au dessus" de l'hyperplan $\mathcal{H}$ selon le sens donné par la normale a de $\mathcal{H}$. Notons que le demi-espace $a_1 x_1 + a_2 x_2 + \dots + a_n x_n \leq b$ correspond lui à tous les points "en dessous" de $\mathcal{H}$. Exemple en 3D, avec l'hyperplan $0x_1 + x_2 + 2x_3 = 3$:



Le point x dessiné vérifie $(a|x - x^*) > 0$ (c'est à dire $x_2 + 2x_3 > 3$) il est bien strictement "au-dessus" du plan vis à vis de l'orientation donnée par le vecteur $a = (0, 1, 2)$ (le point du plan x^* choisi a pour coordonnée $x^* = (0, 3, 0)$).

mini exercice 4 convexité d'un hyperplan et d'un demi-espace

Montrer en utilisant la définition que ces deux objets sont bien convexes.

3. Une droite si $n = 2$, un plan si $n = 3$, un sous-espace affine de dimension $n - 1$ dans tous les cas.

2.2.3 Systèmes d'équations linéaires

Si on doit vérifier plusieurs équations linéaires (ici m) :

$$\begin{cases} \mathcal{H}_1 : & a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n & = & b_1 \\ & \vdots & & \vdots \\ \mathcal{H}_i : & a_{i,1}x_1 + a_{i,2}x_2 + \cdots + a_{i,n}x_n & = & b_i \\ & \vdots & & \vdots \\ \mathcal{H}_m : & a_{m,1}x_1 + a_{m,2}x_2 + \cdots + a_{m,n}x_n & = & b_m \end{cases}$$

il s'agit donc, géométriquement, de l'intersection de m hyperplans affines dans $\mathbb{R}^n$. Un tel système se met sous la forme $Ax = b$, $A \in \mathbb{R}^{m \times n}$ avec $(A)_{i,j} = a_{i,j}$ et $b = (b_1, b_2, \dots, b_m) \in \mathbb{R}^m$. Il y a deux cas possible :

cas 1 Les normales $A_1^\top, \dots, A_m^\top$ des m hyperplans $\mathcal{H}_i$ sont **linéairement indépendantes**⁴, alors :

- si $n \geq 2$, $\mathcal{H}_1 \cap \mathcal{H}_2$ est un sous-espace affine de dimension $n - 2$:
- pour $n = 2$, il s'agit de l'intersection de deux droites non parallèles, on obtient un sous-espace affine ne contenant qu'un seul point, l'unique solution du système $Ax = b$;
- pour $n = 3$, il s'agit de l'intersection de deux plans non parallèles, on obtient un sous-espace affine de dimension 1, une droite ;
- pour $n = 4$, il s'agit d'un sous-espace affine de dimension 2, etc.
- si $n \geq 3$, $\mathcal{H}_1 \cap \mathcal{H}_2 \cap \mathcal{H}_3$ est un sous-espace affine de dimension $n - 3$.
- si $m \leq n$ l'ensemble des solutions de $Ax = b$ (c'est à dire $\cap_{i=1}^m \mathcal{H}_i$) est un espace affine de dimension $n - m$.

cas 2 Les normales forment **une famille liée**. Dans ce cas⁵, on a soit (au moins) une équation redondante (elle se déduit des autres équations), soit impossibilité (l'ensemble des solutions est vide). En effet si il existe i tel que :

$$A_i = \sum_{\substack{j=1 \\ j \neq i}}^m \alpha_j A_j$$

alors on a à la fois $A_i x = b_i$ et :

$$A_i x = \sum_{\substack{j=1 \\ j \neq i}}^m \alpha_j A_j x = \sum_{\substack{j=1 \\ j \neq i}}^m \alpha_j b_j =: b'_i.$$

Donc soit $b'_i = b_i$ auquel cas l'équation i est redondante, soit $b'_i \neq b_i$ et $Ax = b$ est impossible. Avec un prétraitement adéquat on peut détecter l'impossibilité ou supprimer les équations redondantes. Si $b \in \text{Im} A$ il n'y a que le problème de la redondance, et, en éliminant les équations redondantes, on se retrouve avec $m' \leq n$ équations d'hyperplans dont les normales forment une famille libre, on est revenu au cas 1.

Remarque : le cas $m > n$ (plus d'équations que d'inconnues) avec généralement $b \notin \text{Im} A$ (mais sans exclure le cas $b \in \text{Im} A$) est important dans les problèmes d'identification de paramètres par la méthode des moindres carrés ou d'autres. Dans ce cas on cherche en fait un vecteur x tel que $E(x) := \|Ax - b\|$ soit minimal. Lorsque la norme euclidienne est utilisée il s'agit de la méthode des moindres carrés mais on verra en TD que l'utilisation de la norme infinie ou de la norme 1 conduisent à résoudre un P.L..

4. Les $A_i^\top$ étant des vecteurs de $\mathbb{R}^n$ cela impose $m \leq n$: dans un espace vectoriel de dimension n , toute famille de plus de n vecteurs est nécessairement liée.

5. Dans ce cas on peut avoir $m > n$ aussi bien que $m \leq n$.

2.2.4 Polyèdres (systèmes d'inéquations linéaires)

Si on doit vérifier plusieurs inéquations linéaires (ici m) :

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n & \leq & b_1 \\ & \vdots & \\ a_{i,1}x_1 + a_{i,2}x_2 + \cdots + a_{i,n}x_n & \leq & b_i \\ & \vdots & \\ a_{m,1}x_1 + a_{m,2}x_2 + \cdots + a_{m,n}x_n & \leq & b_m \end{cases}$$

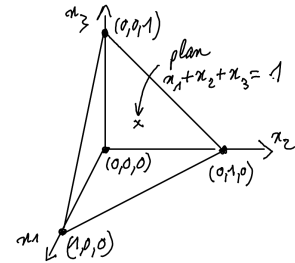
il s'agit donc, géométriquement, de l'intersection de m demi-espaces dans $\mathbb{R}^n$. Ces derniers étant des ensembles convexes, on a encore affaire à un ensemble convexe. Matriciellement on le note $Ax \leq b$.

Définition 4 *polyèdre convexe dans $\mathbb{R}^n$*

Un ensemble de la forme⁶ $\{x \in \mathbb{R}^n : Ax \leq b\}$ où $A \in \mathbb{R}^{m \times n}$ et $b \in \mathbb{R}^m$ est appelé un *polyèdre convexe* de $\mathbb{R}^n$. Lorsqu'en plus il est borné on parle de *polytope*.

Exemple dans $\mathbb{R}^3$ (le simplexe unité) :

$$\begin{cases} x_1 \geq 0 \\ x_2 \geq 0 \\ x_3 \geq 0 \\ x_1 + x_2 + x_3 \leq 1 \end{cases} \iff \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \leq \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

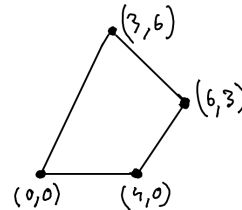


L'idée est qu'un tel objet a une dimension volumique de n mais en fait, en reprenant l'exemple ci-dessus et en lui ajoutant, l'inéquation $x_1 + x_2 + x_3 \geq 1$ ($\iff -x_1 - x_2 - x_3 \leq -1$), il s'agit alors du triangle dont les sommets sont $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$ soit un objet de dimension 2. Exemple de polyèdre convexe non borné :

$$\begin{cases} -x_1 & \leq & 0 \\ & -x_2 & \leq & 0 \\ & & -x_3 & \leq & 0 \\ -x_1 & -x_2 & -x_3 & \leq & -1 \end{cases}$$

mini exercice 5

Écrire sous la forme d'intersection de demi-plans (c'est à dire sous la forme $\{x \in \mathbb{R}^2 : Ax \leq b\}$) le polygone convexe dessiné à droite.



Autre propriété : du fait des contraintes d'inégalités larges et pas strictes utilisées dans la définition, un polyèdre convexe est un ensemble *fermé* dans $\mathbb{R}^n$.

2.3 Le théorème fondamental de la PL

Un PL se définit donc comme l'optimisation d'une fonction linéaire sur un polyèdre. Par exemple si on prend le domaine d'un PL en forme standard avec m contraintes d'égalité :

$$\mathcal{D} = \{x \in \mathbb{R}^n : Ax = b \text{ et } x \geq 0\}, \text{ avec } A \in \mathbb{R}^{m \times n} \text{ et } b \in \mathbb{R}^m$$

6. *Rmq :* un tel ensemble peut être vide, ce qui peut arriver en pratique quand on ajoute trop de contraintes...

il ne s'écrit pas directement comme un polyèdre. On a vu (cf passage de la forme standard à la forme canonique) qu'on peut écrire les contraintes sous la forme :

$$\underbrace{\begin{pmatrix} A \\ -A \\ -I \end{pmatrix}}_{\mathcal{A}} x \leq \underbrace{\begin{pmatrix} b \\ -b \\ 0 \end{pmatrix}}_{\mathcal{B}}$$

la dernière partie correspondant à l'écriture de $x \geq 0$ sous la forme $-x \leq 0$ ⁷. Finalement $\mathcal{D}$ s'écrit donc :

$$\mathcal{D} = \{x \in \mathbb{R}^n : \mathcal{A}x \leq \mathcal{B}\}, \text{ avec } \mathcal{A} \in \mathbb{R}^{(2m+n) \times n} \text{ et } \mathcal{B} \in \mathbb{R}^{2m+n}$$

Théorème 1 *théorème fondamental de la PL*

Soit une fonction linéaire f dans $\mathbb{R}^n$ (donc de la forme $f(x) = c^\top x$), et $\mathcal{D} \subset \mathbb{R}^n$ un polyèdre. Pour le problème :

$$\sup\{f(x) : x \in \mathcal{D}\}$$

3 alternatives et seulement 3 peuvent se présenter :

- (i) $\mathcal{D} = \emptyset$ est vide (on dit que le problème est impossible) ;
- (ii) $\mathcal{D} \neq \emptyset$ mais f n'est pas majorée⁸ sur $\mathcal{D}$ soit $\sup\{f(x) : x \in \mathcal{D}\} = +\infty$;
- (iii) $\mathcal{D} \neq \emptyset$ et f est majorée sur $\mathcal{D}$, alors il existe une solution optimale (un maximiseur) $x^* \in \mathcal{D}$ telle que $f(x^*) = \sup\{f(x) : x \in \mathcal{D}\}$.

Remarques :

- dans le cas (iii) la solution optimale n'est pas forcément unique ;
- même théorème pour le problème $\inf\{f(x) : x \in \mathcal{D}\}$, soit le domaine est vide, soit il n'est pas vide mais $\inf\{f(x) : x \in \mathcal{D}\} = -\infty$ et sinon (f minorée sur $\mathcal{D}$) on a l'existence d'une solution optimale (un minimiseur) ;
- la preuve de ce théorème s'obtient en général en le démontrant sur la forme standard (et s'étend ensuite aisément aux autres formes) ; lorsqu'on a affaire à un polytope, c'est à dire lorsque $\mathcal{D}$ est borné, il est évident ; en effet f étant linéaire elle est continue et $\mathcal{D}$ étant fermé et donc borné (ce qu'on appelle un *compact*), f atteint ses bornes sup et inf sur $\mathcal{D}$; dans le cas contraire la démonstration n'est pas si facile et est omise dans ces notes.

Quelques exemples

Si vous avez compris la résolution graphique vue au chapitre 1, il est facile de donner des exemples qui illustrent ce théorème. Évacuons d'abord le cas où $\mathcal{D}$ est vide. Par exemple les deux inéquations suivantes correspondent évidemment à un domaine vide :

$$\begin{array}{rcl} x + y & \leq & 1 \\ x + y & \geq & 2 \end{array} \iff \begin{pmatrix} 1 & 1 \\ -1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \leq \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Mais ce n'est pas toujours si évident, par exemple les inéquations suivantes :

$$\begin{cases} x_1 + 2x_2 & \leq & 5 \\ 2x_1 + x_2 & \leq & 6 \\ x_1 + x_2 & \geq & 4 \\ x_1 \geq 0, x_2 & \geq & 0 \end{cases}$$

7. Dans $\mathcal{A}$, I est la matrice identité $n \times n$ et dans $\mathcal{B}$, 0 un vecteur de 0 à n composantes.

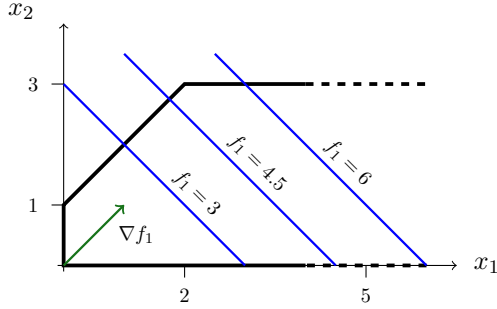
8. Il existe une suite $(x^{(k)}) \in \mathcal{D}$ telle que $\lim_{k \rightarrow \infty} f(x^{(k)}) = +\infty$.

conduisent à un domaine vide⁹.

On considère le domaine suivant qui est un polygône convexe (non borné) :

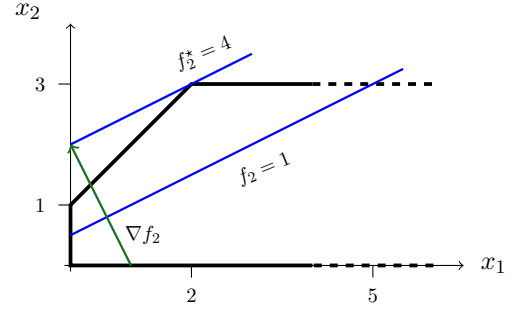
$$\mathcal{D} : \begin{cases} -x_1 + x_2 \leq 1 \\ x_2 \leq 3 \\ x_1 \geq 0, x_2 \geq 0 \end{cases}$$

Voici 4 fonctions linéaires à maximiser sur $\mathcal{D}$ et qui vont conduire à des situations différentes :



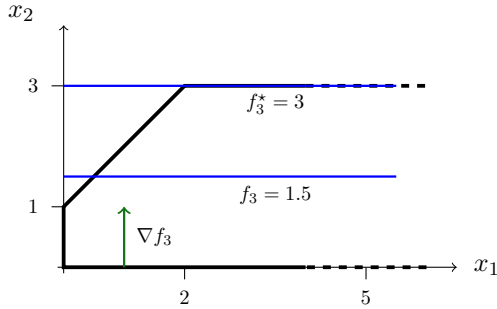
$$f_1(x_1, x_2) = x_1 + x_2$$

f_1 non majorée sur $\mathcal{D}$ ($\sup\{f_1(x) : x \in \mathcal{D}\} = +\infty$)



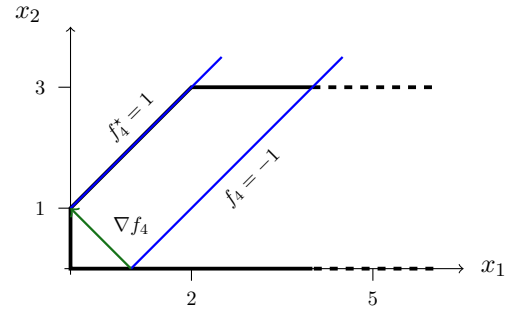
$$f_2(x_1, x_2) = -x_1 + 2x_2$$

f_2 majorée sur $\mathcal{D}$, maximum unique en $x^* = (2, 3)$



$$f_3(x_1, x_2) = x_2$$

f_3 majorée sur $\mathcal{D}$, la demi-droite $\{x_1 \geq 2, x_2 = 3\}$ est optimale.



$$f_4(x_1, x_2) = -x_1 + x_2$$

f_4 majorée sur $\mathcal{D}$, tout le segment $[(0, 1), (2, 3)]$ est optimal

Remarque : dans les cas 3 et 4, on voit qu'au moins un sommet est solution et aussi on comprend que ces deux cas sont un peu exceptionnels :

- si on perturbe légèrement (les coefficients de) f_4 , en fonction de la perturbation c'est uniquement le sommet $(2, 3)$ ou le sommet $(0, 1)$ qui sera l'unique point réalisant le maximum de f_4 sur $\mathcal{D}$;
- la même opération sur f_3 donnera soit le sommet $(2, 3)$ comme unique optimum, soit f_3 deviendra non majorée sur $\mathcal{D}$ (comme dans le cas de f_1).

9. En GRO-2 on verra que le *simplexe phase 1* permet de détecter ce problème et aussi de détecter l'éventuelle redondance des équations (le simplexe phase 1 travaille à partir de la forme standard, il faut transformer les inéquations en équations à l'aide de variables d'écart) et finalement de trouver un point du polyèdre.

2.4 Caractéristiques d'un PL en forme standard

On considère donc le problème :

$$\left(\min_{x \in \mathcal{D}} \text{ ou } \max_{x \in \mathcal{D}} \right) f(x) = c^\top x \quad \text{où } \mathcal{D} := \{x \in \mathbb{R}^n : Ax = b, x \geq 0\} \quad (2.3)$$

avec $A \in \mathbb{R}^{m \times n}$ et $b \in \mathbb{R}^m$. Concernant le système $Ax = b$, nous avons vu (cf 2.2.3) que, soit il n'a pas de solution (on a affaire à un problème impossible ($\mathcal{D} = \emptyset$)), soit on peut se restreindre au cas où $m \leq n$ avec $\text{rang}(A) = m$. Évacuons le cas $m = n$. Dans ce cas on a un système carré inversible avec donc une unique solution $x^* = A^{-1}b$ ¹⁰. Soit $x^* \geq 0$ et c'est la solution du problème (que ce soit un min ou un max puisque l'ensemble des solutions réalisables ne comporte que x^*) et dans le cas contraire, il s'agit d'un problème impossible ($\mathcal{D} = \emptyset$).

Dans la suite de cette section, on considère donc que $m < n$ et $\text{rang}(A) = m$. On en déduit que $\text{Im}A = \mathbb{R}^m$ et $\dim(\text{Ker}A) = n - m > 0$, il y a donc une infinité de solutions à $Ax = b$ mais (pour un b donné) cela n'implique pas qu'il y ait des solutions positives, ainsi malgré tout $\mathcal{D}$ peut-être vide (exemple avec $n = 2$, $m = 1$, $A = [1, 1]$ et $b = -1$).

2.4.1 Solutions de base

Définition 5 *Base en P.L.*

Soit B un sous-ensemble de $\llbracket 1, n \rrbracket$ à m éléments ($\text{card}(B) = \#B = m$). On dit que B est une base si les colonnes $\{A^j, j \in B\}$ sont linéairement indépendantes. Les variables $\{x_j, j \in B\}$ sont appelées variables de la base B . On note $H := \llbracket 1, n \rrbracket \setminus B$, les variables $\{x_j, j \in H\}$ sont appelées variables hors-base ou variables libres.

Remarques :

- Dans la suite lorsqu'on voudra définir un vecteur contenant les variables de base (et un autre contenant les variables hors-base) on considèrera B et H comme respectivement un m -uplet et un $(n - m)$ -uplet, c'est à dire avec un ordre de leurs éléments respectifs, $B = (\ell_1, \dots, \ell_m)$, $H = (h_1, \dots, h_{n-m})$, ce qui permettra donc de définir les matrices et vecteurs suivants :

$$A^B = [A^{\ell_1} \mid A^{\ell_2} \mid \dots \mid A^{\ell_m}] \in \mathbb{R}^{m \times m}, \quad A^H = [A^{h_1} \mid A^{h_2} \mid \dots \mid A^{h_{n-m}}] \in \mathbb{R}^{m \times (n-m)}$$

$$x_B = \begin{bmatrix} x_{\ell_1} \\ \vdots \\ x_{\ell_m} \end{bmatrix} \in \mathbb{R}^m, \quad c_B = \begin{bmatrix} c_{\ell_1} \\ \vdots \\ c_{\ell_m} \end{bmatrix} \in \mathbb{R}^m, \quad x_H = \begin{bmatrix} x_{h_1} \\ \vdots \\ x_{h_{n-m}} \end{bmatrix} \in \mathbb{R}^{n-m}, \quad c_H = \begin{bmatrix} c_{h_1} \\ \vdots \\ c_{h_{n-m}} \end{bmatrix} \in \mathbb{R}^{n-m},$$

Exemple : supposons que $n = 5$ et $m = 3$ et que $\{1, 3, 4\}$ est une base (les colonnes A^1, A^3, A^4 sont linéairement indépendantes ou encore la matrice $[A^1 | A^3 | A^4]$ est inversible); on peut choisir $B = (3, 4, 1)$ et $H = (2, 5)$, dans ce cas x_3 est considérée comme la première variable en base (i.e. selon cet ordre), x_4 comme la deuxième, etc. Cela nous donne donc :

$$A^B = [A^3 | A^4 | A^1] \in \mathbb{R}^{3 \times 3}, \quad A^H = [A^2 | A^5] \in \mathbb{R}^{3 \times 2}$$

$$x_B = \begin{bmatrix} x_3 \\ x_4 \\ x_1 \end{bmatrix} \in \mathbb{R}^3, \quad c_B = \begin{bmatrix} c_3 \\ c_4 \\ c_1 \end{bmatrix} \in \mathbb{R}^3, \quad x_H = \begin{bmatrix} x_2 \\ x_5 \end{bmatrix} \in \mathbb{R}^2, \quad c_H = \begin{bmatrix} c_2 \\ c_5 \end{bmatrix} \in \mathbb{R}^2,$$

¹⁰. Que l'on peut obtenir avec une méthode de Gauss-LU, cf MN1.

- Souvent (dans les exercices) plutôt que de donner les indices des variables qui sont en base, on peut directement désigner les variables de base elles-mêmes, par exemple au lieu d'écrire $B = \{1, 3\}$, on pourra écrire $B = \{x_1, x_3\}$. Cela est pratique quand on rajoute des variables d'écart et que l'on a envie de garder les noms $e_1, e_2, \dots$ pour se souvenir ensuite que ces variables proviennent d'inégalités.

Écriture de $Ax = b$ en utilisant une base

Avec les hypothèses sur A , il existe au moins une base B (au sens précédent), ce qui permet d'écrire $Ax = b$ d'une autre façon. Tout d'abord :

$$Ax = b \iff \sum_{j=1}^n A^j x_j = b \iff \sum_{j \in B} A^j x_j + \sum_{j \in H} A^j x_j = b$$

La dernière expression obtenue peut s'écrire matriciellement :

$$[A^{\ell_1} | \dots | A^{\ell_m}] \begin{bmatrix} x_{\ell_1} \\ \vdots \\ x_{\ell_m} \end{bmatrix} + [A^{h_1} | \dots | A^{h_{n-m}}] \begin{bmatrix} x_{h_1} \\ \vdots \\ x_{h_{n-m}} \end{bmatrix} = b \iff A^B x_B + A^H x_H = b$$

et comme A^B est inversible (B étant une base), on obtient :

$$Ax = b \iff x_B = (A^B)^{-1}b - (A^B)^{-1}A^H x_H$$

Cette deuxième expression est plus forte que la première dans le sens où l'on a obtenu un *paramétrage de l'ensemble des solutions* de $Ax = b$ à l'aide des $n - m$ variables hors-base ou libres. En choisissant n'importe quelles valeurs pour les composantes de $x_H \in \mathbb{R}^{n-m}$, on calcule les autres composantes (la valeur des variables de base) en utilisant $x_B = (A^B)^{-1}b - (A^B)^{-1}A^H x_H$ et, en recombinaison composantes en base et hors-base, on obtient un vecteur x vérifiant $Ax = b$. Pour se rapprocher de calculs que vous avez pu faire précédemment (paramétrage d'une droite ou d'un plan), on pourrait écrire :

$$\{x \in \mathbb{R}^n : Ax = b\} = \{x \in \mathbb{R}^n : \begin{cases} x_B &= (A^B)^{-1}b - (A^B)^{-1}A^H t \\ x_H &= t \end{cases}, t \in \mathbb{R}^{n-m}\}$$

Définition 6 *Solution de base, solution de base réalisable.*

Soit B une base, la solution de base $\underline{x}$ associée à la base B est la solution particulière du système $Ax = b$ qui s'obtient avec le choix le plus simple possible pour ses composantes hors-base : $\underline{x}_H := 0$. Il en résulte que ses composantes en base sont $\underline{x}_B = (A^B)^{-1}b$. Souvent on écrira¹¹ :

$$\underline{x} = \begin{pmatrix} \underline{x}_B &= (A^B)^{-1}b \\ \underline{x}_H &= 0 \end{pmatrix}$$

La solution de base $\underline{x}$ associée à la base B (qui vérifie déjà par définition $A\underline{x} = b$) est **réalisable** (c'est à dire telle que $\underline{x} \in \mathcal{D}$) si de plus $\underline{x} \geq 0$, soit $(A^J)^{-1}b \geq 0$.

11. Attention il ne faut pas comprendre que les m premières composantes de $\underline{x}$ sont les composantes de $\underline{x}_B$ et que les $n - m$ dernières composantes de $\underline{x}$ sont celles de $\underline{x}_H$. Il faut appliquer une permutation, précisément on aura $\underline{x} = P_\sigma \begin{pmatrix} \underline{x}_B \\ \underline{x}_H \end{pmatrix}$ voir la définition de σ ci-après.

Remettre les composantes dans l'ordre

Il est pratique (surtout si on veut écrire un code) de définir la permutation σ suivante :

$$\sigma(k) = \begin{cases} \ell_k & \text{pour } k \in \llbracket 1, m \rrbracket \\ h_{k-m} & \text{pour } k \in \llbracket m+1, n \rrbracket \end{cases}$$

alors :

$$x_{\sigma(k)} = \begin{cases} x_{\ell_k} & \text{pour } k \in \llbracket 1, m \rrbracket : \text{les variables de base} \\ x_{h_{k-m}} & \text{pour } k \in \llbracket m+1, n \rrbracket : \text{les variables hors-base} \end{cases}$$

Soit P_σ la matrice de permutation associée à σ (définie par $Pe^j = e^{\sigma(j)}$) alors :

$$P_\sigma^{-1}x = \begin{pmatrix} x_B \\ x_H \end{pmatrix} \text{ et } x = P_\sigma \begin{pmatrix} x_B \\ x_H \end{pmatrix}$$

Un exemple

Avec :

$$A = \begin{pmatrix} 1 & 2 & -1 & 0 & 1 \\ 3 & 6 & 1 & 1 & 2 \end{pmatrix}, b = \begin{bmatrix} 1 \\ 4 \end{bmatrix}$$

il est clair que $\{1, 2\}$ n'est pas une base puisque $A^2 = 2A^1$ les deux premières colonnes sont liées. Par contre $B = (1, 4)$ est bien une base ! On a $H = (2, 3, 5)$ et :

$$A^B = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}, A^H = \begin{pmatrix} 2 & -1 & 1 \\ 6 & 1 & 2 \end{pmatrix}$$

Le calcul de l'inverse donne :

$$(A^B)^{-1} = \begin{pmatrix} 1 & 0 \\ -3 & 1 \end{pmatrix}$$

On a donc :

$$(A^B)^{-1}b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, (A^B)^{-1}A^H = \begin{pmatrix} 2 & -1 & 1 \\ 0 & 4 & -1 \end{pmatrix}$$

Soit :

$$Ax = b \iff x_J = (A^B)^{-1}b - (A^B)^{-1}A^H x_H \iff \begin{bmatrix} x_1 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} - \begin{pmatrix} 2 & -1 & 1 \\ 0 & 4 & -1 \end{pmatrix} \begin{bmatrix} x_2 \\ x_3 \\ x_5 \end{bmatrix}$$

La solution de base $\underline{x}$ associée à la base B est donc :

$$\underline{x}_H = 0 \text{ soit } \begin{bmatrix} x_2 \\ x_3 \\ x_5 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \text{ et } \underline{x}_B = (A^B)^{-1}b \text{ soit } \begin{bmatrix} x_1 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ d'où : } \underline{x} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

Avec les choix de B et H , on a : $\sigma(1) = 1, \sigma(2) = 4, \sigma(3) = 2, \sigma(4) = 3, \sigma(5) = 5$, ce qui nous donne :

$$P_\sigma = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad P_{\sigma^{-1}} = (P_\sigma)^{-1} = (P_\sigma)^\top = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

On vérifie bien que $x = P_\sigma \begin{bmatrix} x_B \\ x_H \end{bmatrix}$:

$$P_\sigma \begin{bmatrix} x_B \\ x_H \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_4 \\ x_2 \\ x_3 \\ x_5 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}$$

mini exercice 6 *exercice fortement recommandé pour bien comprendre les solutions de base d'un PL*

Soit le PL : $\max_{x \in \mathcal{D}_R} c^\top x$ avec $\mathcal{D}_R = \{x = (x_1, x_2) \in \mathbb{R}^2 : x_1 - x_2 \leq 5, x_2 \leq 5, x \geq 0\}$ et $c = (1, 3)$.

1. Dessiner l'ensemble $\mathcal{D}_R$.
2. Mettre ce PL sous la forme standard : $\max_{X \in \mathbb{R}^4} C^\top X$ avec $AX = b, X \geq 0$.
3. Déterminer toutes les solutions de base ; dessiner la projection des solutions de base réalisables dans le plan (x_1, x_2) . **Aide** : ne pas calculer les inverses des matrices A^B mais résoudre directement les systèmes linéaires associés (c'est à dire partir de $Ax = b$ en mettant les variables hors-base à 0).

Interprétation géométrique des solutions de base réalisables

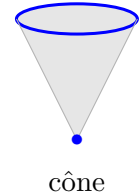
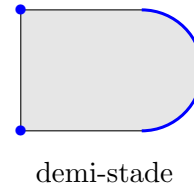
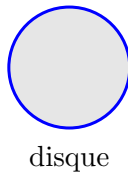
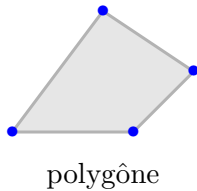
Les solutions de base réalisables (déterminées de façon complètement algébrique) ont une signification géométrique forte : ce sont les sommets du polyèdre $\mathcal{D}$.

Définition 7 *Point extrémal d'un ensemble convexe*

Soit $\mathcal{C} \subset \mathbb{R}^n$ un ensemble convexe. On dit que $x \in \mathcal{C}$ est *extrémal* s'il n'existe pas $y, z \in \mathcal{C}, y \neq z$ et $\theta \in]0, 1[$ tels que $x = (1 - \theta)y + \theta z$.

Remarques :

- la définition nous dit simplement que x est extrémal s'il n'est pas une combinaison convexe stricte de deux points distincts de $\mathcal{C}$;
- pour un polyèdre convexe, un point extrémal est appelé un sommet ;
- quelques illustrations (les points extrémaux sont en bleu) :



Théorème 2

Soit $\mathcal{D} = \{x \in \mathbb{R}^n : Ax = b \text{ et } x \geq 0\}$ avec les hypothèses naturelles ($\text{rang}(A) = m, m < n$) qu'on suppose non vide. Les points extrémaux de $\mathcal{D}$ sont ses solutions de base réalisables. *Preuve* : cf TD.

On a aussi le résultat suivant :

Proposition 2 *Équivalence des sommets d'un PL sous forme canonique pure et sous forme standard*

Soient $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$ et les ensembles :

$$\mathcal{B} = \{x \in \mathbb{R}^n : Ax \leq b \text{ et } x \geq 0\} \text{ et } \mathcal{D} = \{(x, e) \in \mathbb{R}^n \times \mathbb{R}^m : Ax + e = b, (x, e) \geq 0\}.$$

Si x^* est un point extrémal de $\mathcal{B}$ alors $(x^*, e^* := b - Ax^*)$ est un point extrémal de $\mathcal{D}$ et inversement si (x^*, e^*) est un point extrémal de $\mathcal{D}$ alors x^* est un point extrémal de $\mathcal{B}$.

2.4.2 Écriture du P.L. en fonction des variables hors-base

Soit B une base, nous avons vu précédemment que $Ax = b \iff x_B = (A^B)^{-1}(b - A^H x_H)$. Comme :

$$\begin{aligned} c^\top x &= \sum_{j=1}^n c_j x_j = \sum_{j \in B} c_j x_j + \sum_{j \in H} c_j x_j \\ &= [c_{\ell_1} | \dots | c_{\ell_m}] \begin{bmatrix} x_{\ell_1} \\ \vdots \\ x_{\ell_m} \end{bmatrix} + [c_{h_1} | \dots | c_{h_{n-m}}] \begin{bmatrix} x_{h_1} \\ \vdots \\ x_{h_{n-m}} \end{bmatrix} \\ &= c_B^\top x_B + c_H^\top x_H \end{aligned}$$

En utilisant l'expression de x_B en fonction de x_H , on peut écrire la fonction objectif uniquement en fonction des variables hors-base :

$$\begin{aligned} c^\top x &= c_B^\top (A^B)^{-1}(b - A^H x_H) + c_H^\top x_H \\ &= c_B^\top (A^B)^{-1}b + \underbrace{(c_H^\top - c_B^\top (A^B)^{-1} A^H)}_{=\tilde{c}^\top} x_H \end{aligned}$$

en posant $\tilde{c}^\top = c_H^\top - c_B^\top (A^B)^{-1} A^H$, appelé **vecteur des coûts réduits**, notre P.L. en forme standard s'écrit de manière équivalente :

$$\begin{aligned} (\min \text{ ou } \max) \quad & c_B^\top (A^B)^{-1}b + \tilde{c}^\top x_H \\ & x_B = (A^B)^{-1}b - (A^B)^{-1} A^H x_H \\ & x \geq 0 \end{aligned}$$

Cette écriture, appelée “forme canonique” par rapport à la base B , est centrale dans la méthode du simplexe. Supposons que la solution de base $\underline{x}$ associée à la base B ($\underline{x} = (\underline{x}_B = (A^B)^{-1}b, \underline{x}_H = 0)$) soit réalisable (i.e. $(A^B)^{-1}b \geq 0$). Elle constitue donc un sommet de $\mathcal{D}$ et :

$$f(\underline{x}) = c_B^\top \underline{x}_B + c_H^\top \underline{x}_H = c_B^\top (A^B)^{-1}b$$

En posant $\tilde{A} := (A^B)^{-1} A^H$, le problème s'écrit :

$$\begin{aligned} (\min \text{ ou } \max) \quad & f(\underline{x}) + \tilde{c}^\top x_H \\ & x_B = \underline{x}_B - \tilde{A} x_H \\ & x \geq 0 \end{aligned} \tag{2.4}$$

2.5 L'algorithme du simplexe (primal)

Cette méthode^a, inventée par Georges Dantzig en 1947, est basée sur l'écriture canonique d'un PL en forme standard par rapport à une base B (dont la solution de base $\underline{x}$ est réalisable) que l'on a vu juste précédemment (2.4). Il faut donc connaître une telle solution, c'est à dire un sommet du polyèdre ce qui sera le cas des problèmes que nous traiterons en TD pour appliquer cet algorithme. Dans le cas contraire, on peut utiliser le simplexe phase 1^b, pour trouver un sommet initial. Ce simplexe phase 1 permet aussi de détecter l'impossibilité ($\mathcal{D} = \emptyset$) ainsi que les redondances éventuelles dans les équations, il sera vu en GRO-2. Ce nom *simplexe phase 1* fait qu'on appelle parfois l'algorithme du *simplexe primal* le *simplexe phase 2*.

^a. Cette première version s'appelle le *simplexe primal* ; en 1954 G. Dantzig invente une variante appelée *simplexe dual* très utilisée aussi et qui sera entrevue en GRO-2.

^b. Qui est un simplexe primal appliqué sur un PL auxiliaire pour lequel un sommet initial est connu.



G. Dantzig, 1914-2005

2.5.1 Une étape du simplexe

Dans la suite on considère que l'on maximise, on veut donc résoudre le problème :

$$\max_{x \in \mathcal{D}} f(x) = c^\top x, \text{ où } \mathcal{D} = \{x \in \mathbb{R}^n : Ax = b \text{ et } x \geq 0\} \text{ avec } A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m \quad (2.5)$$

pour lequel on suppose donc connue une base B dont la solution de base associée $\underline{x}$ est réalisable. On suppose aussi que l'on a effectué les calculs¹² de façon à mettre (2.5) sous la forme (2.4) :

$$\begin{aligned} \max \quad & f(\underline{x}) + \tilde{c}^\top x_H \\ & x_B = \underline{x}_B - \tilde{A}x_H \\ & x \geq 0 \end{aligned} \quad (2.6)$$

Dans les applications pratiques, on utilisera la forme suivante appelée dictionnaire (associé à la base B) :

$$\begin{array}{rcl} x_B & = & \underline{x}_B - \tilde{A}x_H \\ \hline F & = & f(\underline{x}) + \tilde{c}^\top x_H \end{array}$$

Il faut bien comprendre les points suivants :

- la relation $x_B = \underline{x}_B - \tilde{A}x_H$ est strictement équivalente à $Ax = b$ avec dans la partie droite les variables hors-base et dans la partie gauche les variables de base ;
- de même la relation $F = f(\underline{x}) + \tilde{c}^\top x_H$ donne exactement la même valeur que $f(x) = c^\top x$ sous la condition $Ax = b$ (mais formellement la partie droite est une fonction de x_H) ;
- si on impose $x_H = 0$ dans le dictionnaire, on se retrouve positionné sur le sommet $\underline{x}$ avec pour valeur $f(\underline{x})$.

Le principe général de l'algorithme est le suivant :

Trouver une autre base B' correspondant à un sommet voisin $\underline{x}'$ tel que (a priori) $f(\underline{x}') > f(\underline{x})$. Un tel sommet voisin (géométriquement partageant une arête avec $\underline{x}$) correspond à une base B' "voisine" de B au sens où elles diffèrent seulement de 2 variables (une variable hors-base (pour B) va se trouver dans la base B' et une variable de base de B va se retrouver hors-base pour B').

Ce principe va conduire à décomposer une étape du simplexe en 3 parties.

Partie 1 : examen des coûts réduits $\tilde{c}$

D'après le principe général, il faut donc faire entrer en base une des variables hors-base actuelle qui permettrait d'augmenter la valeur de la fonction objectif, or, sous la condition $Ax = b$:

$$\begin{aligned} f(x) (= c^\top x) &= f(\underline{x}) + \tilde{c}^\top x_H \\ &= f(\underline{x}) + \tilde{c}_1(x_H)_1 + \tilde{c}_2(x_H)_2 + \cdots + \tilde{c}_{n-m}(x_H)_{n-m} \\ &= f(\underline{x}) + \tilde{c}_1 x_{h_1} + \tilde{c}_2 x_{h_2} + \cdots + \tilde{c}_{n-m} x_{h_{n-m}} \end{aligned}$$

Actuellement (c'est à dire pour le sommet $\underline{x}$) toutes les variables hors-base sont nulles et il est clair que si il existe k^* tel que $\tilde{c}_{k^*} > 0$ alors augmenter $(x_H)_{k^*} = x_{h_{k^*}}$ de 0 à une valeur strictement positive t (en laissant les autres variables hors-base à 0) permettra une augmentation de la fonction égale à $\tilde{c}_{k^*}t$. Que se passe-t-il si $\tilde{c} \leq 0$ (c'est à dire si $\tilde{c}_k \leq 0, \forall k \in \llbracket 1, n-m \rrbracket$) ? Et bien $\underline{x}$ est optimal :

Proposition 3 condition suffisante d'optimalité

- (i) Si $\tilde{c} \leq 0$ alors $\underline{x}$ est un sommet de $\mathcal{D}$ réalisant le maximum de f sur $\mathcal{D}$.
- (ii) Si de plus $\tilde{c} < 0$ alors $\underline{x}$ est l'unique maximiseur de f sur $\mathcal{D}$.

12. C'est à dire, étant donné B , le calcul de $\underline{x}$ (en fait juste $\underline{x}_B = (A^B)^{-1}b$) mais aussi le calcul de $f(\underline{x})$, de $\tilde{c}$ et de $\tilde{A}$.

Preuve : (i) il faut montrer que $\forall x \in \mathcal{D}, f(x) \leq f(\underline{x})$. Soit donc $x \in \mathcal{D}$ (i.e. une solution réalisable), on a $f(x) = c^\top x$ mais sous la condition $Ax = b$, on a vu que $f(x) = f(\underline{x}) + \tilde{c}^\top x_H$, d'autre part $x \geq 0$ (et donc $x_H \geq 0$) d'où :

$$f(x) = f(\underline{x}) + \underbrace{\sum_{k=1}^{n-m} \tilde{c}_k}_{\leq 0} \underbrace{x_{h_k}}_{\geq 0} \leq f(\underline{x})$$

Pour le (ii) cf TD.

Remarques :

- Lorsqu'il y a plusieurs coûts réduits strictement positifs, quelle variable hors-base fait-on entrer ? C'est une question intéressante, initialement *Dantzig* propose la règle suivante (qu'on utilisera systématiquement) on fait entrer la variable hors-base dont le coût réduit est maximal, si k^* est tel que :

$$\tilde{c}_{k^*} = \max\{\tilde{c}_k, k \in \llbracket 1, n-m \rrbracket\} > 0$$

alors $(x_H)_{k^*} = x_{h_{k^*}}$ entre en base (si le max est atteint pour plusieurs valeurs de k on en choisit une plus ou moins arbitrairement). On reviendra sur cette question ultérieurement.

- Si on veut minimiser, la condition suffisante d'optimalité est $\tilde{c} \geq 0$ et dans le cas contraire on choisit de faire entrer en base la variable qui a le coût réduit le plus négatif.

Partie 2 : qui sort (ou pas) de la base ?

On envisage donc de faire entrer $x_{h_{k^*}}$ en base. On pose $e = h_{k^*}$ le numéro de la variable entrante. Que se passe-t-il si on augmente x_e de 0 à une certaine valeur tout en laissant les autres variables hors-base à 0 ? Il faut rester dans $\mathcal{D}$ c'est à dire maintenir $Ax = b$ et $x \geq 0$. Comme $Ax = b$ est équivalent à $x_B = \underline{x}_B - \tilde{A}x_H = \underline{x}_B - \sum_{k=1}^{n-m} \tilde{A}^k x_{h_k}$ mais qu'ici on garde les autres variables hors-base à 0 ($x_{h_k} = 0$ pour $k \neq k^*$), cela nous donne :

$$x_B = \underline{x} - \tilde{A}^{k^*} x_e \iff \begin{cases} x_{\ell_1} = \underline{x}_{\ell_1} - \tilde{A}_{1,k^*} x_e \\ \vdots = \vdots \\ x_{\ell_i} = \underline{x}_{\ell_i} - \tilde{A}_{i,k^*} x_e \\ \vdots = \vdots \\ x_{\ell_m} = \underline{x}_{\ell_m} - \tilde{A}_{m,k^*} x_e \end{cases}$$

On s'occupe maintenant de garder la positivité : lorsque $x_e \nearrow$ on aura $x_{\ell_i} \searrow$ si et seulement si $\tilde{A}_{i,k^*} > 0$ et dans ce cas x_{ℓ_i} atteint 0 pour :

$$x_e = \frac{\underline{x}_{\ell_i}}{\tilde{A}_{i,k^*}}$$

Ici deux possibilités :

- Si $\tilde{A}_{i,k^*} \leq 0, \forall i \in \llbracket 1, m \rrbracket$ que se passe-t-il ? **Réponse :** on découvre que f est **non majorée** sur $\mathcal{D}$. En effet dans ce cas on peut augmenter x_e autant que l'on veut tout en restant dans $\mathcal{D}$ (puisque on vérifie $Ax = b$ et $x \geq 0$) avec donc :

$$f(x) = f(\underline{x}) + \underbrace{\tilde{c}_{k^*}}_{>0} x_e \rightarrow +\infty \text{ quand } x_e \rightarrow +\infty$$

- Dans le cas contraire au moins une variable qui était en base revient vers 0 (on note $I := \{i \in \llbracket 1, m \rrbracket : \tilde{A}_{i,k^*} > 0\} \neq \emptyset$), on arrête alors la progression de x_e dès qu'une variable de base atteint 0, soit pour :

$$x_e = t := \min\left\{\frac{\underline{x}_{\ell_i}}{\tilde{A}_{i,k^*}} : i \in I\right\}$$

Ce minimum va être atteint pour $i := i^*$ et c'est la variable $x_{\ell_{i^*}}$ qui sort alors de la base¹³. On a trouvé la nouvelle base, en posant $s = \ell_{i^*}$:

$$B' = (B \setminus \{s\}) \cup \{e\}, \quad H' = (H \setminus \{e\}) \cup \{s\}$$

la solution de base associée $\underline{x}'$ (qui est réalisable par construction) vérifie :

$$\begin{aligned} \underline{x}'_{\ell_i} &= \underline{x}_{\ell_i} - \tilde{A}_{i,k^*} t \quad \text{pour } i \in \llbracket 1, m \rrbracket \text{ (avec } \underline{x}_s = 0) \\ \underline{x}'_e &= t \\ \underline{x}'_{h_k} &= 0 \quad \text{pour } k \in \llbracket 1, n - m \rrbracket \setminus \{k^*\} \\ f(\underline{x}') &= f(\underline{x}) + \underbrace{\tilde{c}_{k^*}}_{>0} \underbrace{t}_{\geq 0} \geq f(\underline{x}) \end{aligned}$$

Interprétation géométrique : lorsque la variable entrante x_e passe de 0 à t on parcourt l'arête $[\underline{x}, \underline{x}']$ du polyèdre $\mathcal{D}$.

Partie 3 : mise à jour du dictionnaire

Il faut maintenant écrire le problème sous la forme (2.4) mais pour la base B' et, en pratique, obtenir le dictionnaire associé

$$\begin{aligned} \max \quad & f(\underline{x}') + (\tilde{c}')^\top x_{H'} \\ \text{s.t.} \quad & x_{B'} = \underline{x}_{B'} - \tilde{A}' x_{H'} \\ & x \geq 0 \end{aligned} \quad \text{dictionnaire : } \frac{x_{B'} = \underline{x}_{B'} - \tilde{A}' x_{H'}}{F = f(\underline{x}') + (\tilde{c}')^\top x_{H'}}$$

Nous donnons ici le principe de la mise à jour :

1. dans l'équation de la variable sortante (ligne i^* du dictionnaire précédent) on échange les rôles de la variable sortante $x_s = x_{\ell_{i^*}}$ et de la variable entrante $x_e = x_{h_{k^*}}$ (on exprime x_e en fonction des (nouvelles) variables hors-base) ;
2. dans toutes les autres équations¹⁴, on remplace x_e par l'expression obtenue ci-dessus.

De cette façon il est clair que nous obtenons bien le dictionnaire associé à la base B' puisqu'on exprime les nouvelles variables de base $x_{B'}$ (c'est à dire les anciennes sauf x_s qui est remplacée par x_e) en fonction des nouvelles variables hors-base $x_{H'}$ (c'est à dire les anciennes sauf x_e qui est remplacée par x_s) pour la partie haute et de même la fonction objectif est exprimée fonction des nouvelles variables hors-base.

Remarques :

- On peut établir des formules précises de cette mise à jour et donc écrire un code¹⁵.
- Pour les calculs “à la main” il est un peu plus rapide de les organiser sous la forme d'un tableau (les fameux tableaux de simplexe) mais les dictionnaires sont beaucoup plus lisibles et, de plus, si vous comprenez les principes de l'algorithme, résoudre un (petit) PL à la main avec les dictionnaires est beaucoup plus évident (du fait de la lisibilité) qu'en utilisant un tableau de simplexe.

13. Plusieurs variables de base peuvent atteindre 0 simultanément auquel cas on choisit plus ou moins arbitrairement celle qui sort de la base.

14. C'est à dire toutes les autres équations de la partie haute mais aussi l'équation de la fonction (partie basse du dictionnaire)

15. Il faut aussi mettre à jour la permutation σ (voir 2.4.1) ce qui se fait facilement, on avait $\sigma(i^*) = s$ et $\sigma(m + k^*) = e$, on a alors $\sigma'(i^*) = e$ et $\sigma'(m + k^*) = s$ et $\sigma'(j) := \sigma(j)$ pour $j \neq i^*$ et $j \neq k^*$. Noter que même si on peut toujours utiliser cette approche, les codes modernes utilisent une autre formulation appelée *simplexe révisé* qu'on pourra aborder dans un exercice.

2.5.2 L'algorithme complet (avant les détails fâcheux) et un exemple

L'algorithme consiste à itérer les étapes de simplexe jusqu'à trouver le (ou un) point optimal ou bien la non majoration de la fonction objectif :

répéter :

(i) examen des coûts réduits $\tilde{c}$:

si $\tilde{c} \leq 0$:

fin de l'algorithme le sommet actuel est optimal.

sinon :

choix de la variable entrante x_e (par exemple celle correspondant au coût réduit $\tilde{c}_{k^*}$ maximal¹⁶)

(ii) détection de la non majoration ou choix de la variable sortante :

si $I := \{i : \tilde{A}_{i,k^*} > 0\} = \emptyset$:

fin de l'algorithme f est non majorée sur $\mathcal{D}$.

sinon :

la variable x_s avec $s = \ell_{i^*}$ où i^* réalise le min de $\{\underline{x}_{\ell_i} / \tilde{A}_{i,k^*} : i \in I\}$
sort de la base

(iii) mises à jour diverses

Un exemple

On reprend le problème de production que l'on a résolu de manière graphique (cf 1.2) et qu'on écrit en forme standard :

$$\begin{aligned} \max f(X) &= 6x_1 + 4x_2 \\ \begin{cases} 3x_1 + 9x_2 + e_1 & = 81 \\ 4x_1 + 5x_2 + e_2 & = 55 \\ 2x_1 + x_2 + e_3 & = 20 \\ X = (x_1, x_2, e_1, e_2, e_3) & \geq 0 \end{cases} \end{aligned}$$

Il est clair que $B^{(0)} = \{e_1, e_2, e_3\}$ (ou $B^{(0)} = \{3, 4, 5\}$) est une base ($A^{B^{(0)}} = I$) dont la solution de base associée est réalisable : en mettant les variables hors-base x_1 et x_2 à 0, il vient :

$$X^{(0)} = (x_1^{(0)} = 0, x_2^{(0)} = 0, e_1^{(0)} = 81, e_2^{(0)} = 55, e_3^{(0)} = 20) \geq 0$$

Le dictionnaire initial est immédiat (pas de calculs à faire) :

$e_1 = 81 - 3x_1 - 9x_2$
$e_2 = 55 - 4x_1 - 5x_2$
$e_3 = 20 - 2x_1 - x_2$
$F = 0 + 6x_1 + 4x_2$

Étape 1

(i) les coûts réduits $((6, 4))$ sont tous strictement positifs, on fait entrer en base la variable associée au plus grand soit x_1 .

(ii) quand on augmente x_1 (en laissant $x_2 = 0$) les variables en base évoluent selon :

$$\begin{aligned} e_1 &= 81 - 3x_1 & e_1 \searrow & \text{et } e_1 = 0 \text{ pour } x_1 = 81/3 = 27 \\ e_2 &= 55 - 4x_1 & e_2 \searrow & \text{et } e_2 = 0 \text{ pour } x_1 = 55/4 = 13.75 \\ e_3 &= 20 - 2x_1 & e_3 \searrow & \text{et } e_3 = 0 \text{ pour } x_1 = 10 \end{aligned}$$

C'est donc e_3 qui sort de la base. La nouvelle base est donc $B^{(1)} = \{x_1, e_1, e_2\}$ (et $H^{(1)} = \{x_2, e_3\}$)

¹⁶. Rappel : on fait entrer en base la variable dont l'indice dans le vecteur x_H est k^* mais son indice dans le vecteur x est $e = h_{k^*}$.

(iii) mise à jour du dictionnaire : dans la troisième équation, on échange le rôle de e_3 et x_1 :

$$x_1 = 10 - \frac{1}{2}e_3 - \frac{1}{2}x_2$$

dans les autres équations, on remplace x_1 par cette expression :

$$\begin{aligned} e_1 &= 81 - 3(10 - \frac{1}{2}e_3 - \frac{1}{2}x_2) - 9x_2 \\ e_2 &= 55 - 4(10 - \frac{1}{2}e_3 - \frac{1}{2}x_2) - 5x_2 \\ F &= 0 + 6(10 - \frac{1}{2}e_3 - \frac{1}{2}x_2) + 4x_2 \end{aligned}$$

on obtient :

$x_1 = 10 - \frac{1}{2}x_2 - \frac{1}{2}e_3$
$e_1 = 51 - \frac{15}{2}x_2 + \frac{3}{2}e_3$
$e_2 = 15 - 3x_2 + 2e_3$
$F = 60 + x_2 - 3e_3$

La solution de base associée est :

$$X^{(1)} = (x_1^{(1)} = 10, x_2^{(1)} = 0, e_1^{(1)} = 51, e_2^{(1)} = 15, e_3^{(1)} = 0) \geq 0$$

avec $f(X^{(1)}) = 60$.

Étape 2

- (i) les coûts réduits sont $((1, -3))$ seule la variable x_2 peut entrer en base pour faire augmenter la fonction ;
- (ii) quand on augmente x_2 (en laissant $e_3 = 0$) les variables en base évoluent selon :

$$\begin{aligned} x_1 &= 10 - \frac{1}{2}x_2 & x_1 \searrow \text{ et } x_1 = 0 \text{ pour } x_2 = 20 \\ e_1 &= 51 - \frac{15}{2}x_2 & e_1 \searrow \text{ et } e_1 = 0 \text{ pour } x_2 = 102/15 = 6.8 \\ e_2 &= 15 - 3x_2 & e_2 \searrow \text{ et } e_2 = 0 \text{ pour } x_2 = 5 \end{aligned}$$

C'est donc e_2 qui sort de la base. La nouvelle base est donc $B^{(2)} = \{x_1, x_2, e_1\}$ (et $H^{(1)} = \{e_2, e_3\}$)

(iii) mise à jour du dictionnaire : dans la troisième équation, on échange le rôle de e_2 et x_2 :

$$x_2 = 5 - \frac{1}{3}e_2 - \frac{2}{3}e_3$$

dans les autres équations, on remplace x_2 par cette expression, on obtient le dictionnaire :

$x_1 = \frac{15}{2} + \frac{1}{6}e_2 - \frac{5}{6}e_3$
$x_2 = 5 - \frac{1}{3}e_2 + \frac{2}{3}e_3$
$e_1 = \frac{27}{2} + \frac{5}{2}e_2 - \frac{7}{2}e_3$
$F = 65 - \frac{1}{3}e_2 - \frac{7}{3}e_3$

La solution de base associée est :

$$X^{(2)} = (x_1^{(2)} = \frac{15}{2}, x_2^{(2)} = 5, e_1^{(2)} = \frac{27}{2}, e_2^{(2)} = 0, e_3^{(2)} = 0) \geq 0$$

avec $f(X^{(2)}) = 65$.

Étape 3

- (i) Les coûts réduits sont strictement négatifs la solution de base actuelle ($X^{(2)}$) est l'unique maximiseur de f .

mini exercice 7 suivre géométriquement la progression de l'algorithme

Suivre la progression de l'algorithme dans le plan (x_1, x_2) : reprendre le dessin de la résolution graphique et placez-y la projection des solutions de base pour vous rendre compte qu'on passe bien d'un sommet à un sommet voisin.

2.5.3 Convergence de l'algorithme du simplexe

Il y a 3 points (notés A, B et C) concernant ce sujet.

A/ Cyclage possible de l'algorithme

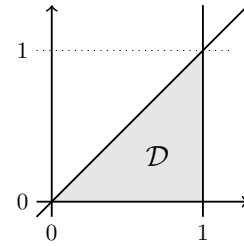
Lors d'une étape du simplexe, la fonction objectif peut stagner¹⁷ lorsque la solution de base actuelle $\underline{x}$ est *dégénérée*. On peut passer d'une base réalisable B (de solution $\underline{x}$) à une autre base $B' \neq B$ (de solution $\underline{x}'$) mais avec $\underline{x}' = \underline{x}$ (et donc $f(\underline{x}') = f(\underline{x})$).

Définition 8 *Solution de base réalisable dégénérée.*

Une solution de base réalisable $\underline{x}$ (liée à la base B) est **dégénérée** si au moins une de ses composantes en base, i.e. une composante de $\underline{x}_B = (A^B)^{-1}b$ est nulle.

Comment se produit ce phénomène de dégénérescence ? *Essentiellement* du fait qu'un même sommet $\underline{x} \in \mathcal{D}$ peut être la solution de base réalisable de plusieurs bases par une redondance des contraintes en $\underline{x}$. Voici un exemple simple. On considère le problème suivant :

$$\begin{aligned} \max_{x \in \mathcal{D}} f(x) &= x_1 + x_2 \\ \mathcal{D} &= \{(x_1, x_2) \in \mathbb{R}^2 : x \geq 0, x_2 \leq x_1, x_1 \leq 1\} \end{aligned}$$



On voit que le sommet $(0,0)$ est à la fois l'intersection des droites $x_1 = 0$ et $x_2 = 0$ mais aussi l'intersection des droites $x_1 = 0$ et $x_2 = x_1$ ou encore l'intersection des droites $x_2 = 0$ et $x_1 = x_2$. Il y a donc trois bases possibles pour ce sommet comme nous allons le voir en passant en forme standard qui s'obtient en rajoutant deux variables d'écart :

$$\max_{(x,e) \in \mathcal{D}'} f(x,e) = x_1 + x_2, \quad \mathcal{D}' = \{(x,e) \in \mathbb{R}^2 \times \mathbb{R}^2 : (x,e) \geq 0, -x_1 + x_2 + e_1 = 0, x_1 + e_2 = 1\}$$

La matrice des contraintes d'égalité s'écrit : $A = \begin{pmatrix} -1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}$

On vérifie facilement (le faire !) que les bases $\{1, 4\}$ (i.e. $\{x_1, e_2\}$ en terme de nom de variables), $\{2, 4\}$ et $\{3, 4\}$ donne la même solution de base $(0,0,0,1)$ qui correspond bien au sommet $(0,0)$ du domaine initial. Le terme “essentiellement” utilisé plus haut s'explique par le résultat suivant :

Proposition 4 *Sur les solutions dégénérées*

- (i) Si deux bases différentes donnent la même solution de base, celle-ci est dégénérée.
- (ii) Si la solution de base $\underline{x}$ associée à une base B est dégénérée (c'est à dire qu'il existe $i^* \in B$ tel que $\underline{x}_{i^*} = 0$) alors soit elle admet une autre représentation de base (il existe $B' \neq B$ tel que la solution de base associée à B' est égale à $\underline{x}$), soit la variable x_{i^*} est toujours nulle (toutes les solutions de $Ax = b$ vérifient $x_{i^*} = 0$) et peut donc être éliminée.

Revenons maintenant en arrière sur la partie 2 d'une étape du simplexe (“qui sort de la base?”). Si $\underline{x}_{\ell_{i^*}} = 0$ avec $i^* \in I$ (rappel $I = \{i \in \llbracket 1, m \rrbracket : \tilde{A}_{i,k^*} > 0\}$) alors la “progression” de la variable entrante x_e est immédiatement bloquée puisque dans ce cas :

$$x_e = t := \min\left\{\frac{\underline{x}_{\ell_i}}{\tilde{A}_{i,k^*}}, i \in I\right\} = 0$$

17. i.e. ne pas augmenter quand on maximise et peut ne pas diminuer quand on minimise.

Il y a bien un changement de base (x_e entre en base et $x_s = x_{\ell_i^}$ sort de la base) mais géométriquement on ne bouge pas : on a juste trouvé une représentation de base différente du même sommet $\underline{x}$.*

Revenons à notre exemple. Si on choisit la base gratuite initiale (cf exercice 2 feuille 2) formée par les deux variables d'écart, on a le dictionnaire initial suivant :

$$\begin{array}{rclclcl} e_1 & = & 0 & + & x_1 & - & x_2 \\ e_2 & = & 1 & - & x_1 & & \\ \hline F & = & 0 & + & x_1 & + & x_2 \end{array}$$

Si on choisit de faire entrer la variable x_1 tout se passe bien, on découvre alors le sommet $(1, 0)$ mais si on choisit de faire entrer x_2 on change juste de base (e_1 sort de la base) sans géométriquement bouger. On obtient le dictionnaire :

$$\begin{array}{rclclcl} x_2 & = & 0 & + & x_1 & - & e_1 \\ e_2 & = & 1 & - & x_1 & & \\ \hline F & = & 0 & + & 2x_1 & - & e_1 \end{array}$$

Ici une nouvelle itération de simplexe nous fait “décoller” de $(0, 0)$ pour obtenir le sommet $(1, 0)$ mais **est-il possible de rester *ad vitam aeternam*** sur le même sommet $\underline{x}$ en alternant tout ou partie des bases dont il est la solution (de base)? **Réponse** oui. Ce phénomène est appelé *cyclage* (de l'algorithme du simplexe), un exemple historique est celui de *E.M.L. Beale* de 1955 :

$$\begin{array}{rclclcl} \max f(x) & = & \frac{3}{4}x_4 - 20x_5 + \frac{1}{2}x_6 - 6x_7 \\ x_1 & & + \frac{1}{4}x_4 & - 8x_5 & - x_6 & + 9x_7 & = 0 \\ x_2 & & + \frac{1}{2}x_4 & - 12x_5 & - \frac{1}{2}x_6 & + 3x_7 & = 0 \\ x_3 & & & & + x_6 & & = 1 \\ x & \geq & 0 \end{array}$$

Dans cet exemple (où les variables x_1, x_2 et x_3 sont utilisées comme base initiale), l'algorithme du simplexe “cycle” sur la solution de base associée si on utilise la règle usuelle pour la variable entrante (coût réduit le plus grand) et la règle du plus petit indice s en cas de plusieurs variables sortantes possible.

La découverte de cette possibilité dans le déroulement du simplexe a suscité beaucoup de questions et de papiers de recherche. Comme ce phénomène est assez rare, il a souvent été occulté dans les codes ; de plus comme les calculs en arithmétique flottante génèrent de petites erreurs, il est possible que ces dernières aient un effet anti-cyclage. Il y a plusieurs stratégies qui permettent d'éviter ce phénomène, la plus simple étant les règles de *R. G. Bland* (1977) :

- La variable entrante est celle qui a le plus petit indice r et un coût réduit > 0 (< 0 si on minimise).
Avec les notations précédentes : $r = h_{k^*}$ avec $\tilde{c}_{k^*} > 0$ et $\forall k \neq k^*$ tel que $\tilde{c}_k > 0$, on a $h_k > r$.
- En cas d'ambiguïté pour la variable sortante, celle avec le plus petit indice (global) est choisie.

Ces règles peuvent se substituer momentanément à d'autres règles (plus efficaces pour diminuer le nombre d'itérations (voir plus loin)) dès qu'on observe la stagnation de la fonction.

B/ Finitude du simplexe

Définition 9 *PL non dégénéré*

Un PL (P) est dit non dégénéré si toutes ses solutions de base réalisables sont non dégénérées.

On a le théorème facile suivant :

Théorème 3 *Finitude du simplexe pour un PL non dégénéré*

Pour un PL (P) non dégénéré, l'algorithme du simplexe se termine en un nombre fini d'itérations.

Preuve (cas où on maximise) : une itération donnée :

- ou bien $\tilde{c} \leq 0$, la solution de base réalisable actuelle $\underline{x}$ est optimale ;
- ou bien on détecte que la fonction objectif est non majorée sur $\mathcal{D}$ ($I = \emptyset$) ;
- ou bien on découvre une nouvelle solution de base réalisable $\underline{x}'$ telle que $f(\underline{x}') > f(\underline{x})$ du fait de l'hypothèse de non dégénérescence. Et donc nécessairement $\underline{x}' \neq \underline{x}$.

Ainsi on ne rencontre jamais la même base : tant que l'on a pas découvert l'optimalité ou la non minoration, les bases déjà parcourues sont nécessairement toutes distinctes. Comme le nombre de bases est fini (borné par C_n^m), on arrive soit à un sommet optimal, soit à un sommet où l'on découvre la non minoration de la fonction.

Remarque : si on utilise des règles anti-cyclage, ce théorème s'étend à tous les PL.

C/ Complexité de l'algorithme du simplexe

C'est bien que l'algorithme se termine mais au bout de combien d'itérations atteint-on un optimum¹⁸ ? Tout d'abord un petit point **sur les différentes règles pour choisir la variable entrante**. Dans la suite on considère qu'on maximise. Si vous avez compris le principe de l'algorithme toute variable hors-base associée à un coût réduit strictement positif va faire augmenter la fonction sur une itération (sauf éventuellement en cas de dégénérescence). Initialement *Dantzig* propose deux règles :

- (i) choisir le coût réduit le plus grand (plus une autre règle pour désigner une variable si le coût réduit max est atteint plusieurs fois)
- (ii) tester toutes les possibilités de variables entrantes possibles (i.e. celles liées à un coût réduit strictement positif) faire les calculs associés et garder celle qui permet d'augmenter le plus la fonction.

Il est clair que la deuxième règle coûte cher : si il y a 20 coûts réduits strictement positifs, on fait 20 fois plus de calculs pour cette étape ! Cependant si intuitivement il est naturel de choisir le coût réduit le plus grand (pour une même progression t de la variable entrante, c'est là où la fonction va augmenter le plus), cela n'est absolument pas garanti. Par exemple avec un coût réduit de 10 si la variable entrante vaut 1 (de sorte que la première variable en base atteigne 0), on a augmenté de 10 alors qu'avec un coût de 9 (plus petit donc) si la variable entrante vaut 2, on a augmenté de 18. D'autres règles sont apparues ensuite et se sont avérées plus efficaces en moyenne. Ces règles sont basées sur l'utilisation du gradient : on essaie de choisir le sommet voisin $\underline{x}'$ tel que l'angle entre $\overrightarrow{\underline{x} \underline{x}'}$ et $\overrightarrow{\nabla f}$ soit le plus petit possible. Il y a un peu plus de calculs par itération mais on gagne au final en faisant moins d'itérations¹⁹. D'autres règles continuent de voir le jour²⁰.

Mais combien d'itérations ?

Sur les problèmes rencontrés en pratique, on observait le comportement suivant (avec la règle (i) du coût réduit le plus grand pour la variable entrante) :

- de m à $3m$ itérations (i.e. bases à visiter), ceci lorsque n varie linéairement en fonction de m (par exemple $n = 5m$) ;
- lorsqu'on fixe m et qu'on augmente n , le nombre d'itérations tend à augmenter selon $\ln(n)$ donc assez faiblement.

Ainsi, même si personne n'avait encore réussi à montrer que l'algorithme du simplexe était de complexité maximale polynomiale²¹, il était tout à fait performant et satisfaisant. Or en 1972, *V. Klee* et

18. Ou la détection de la non bornitude de la fonction sur le domaine.

19. La première stratégie basée sur cette idée (appelé *Devev rule*) est due à *Paula Harris* ("Pivot selection methods of the Devev LP code." Mathematical programming 5.1 (1973) : 1-28.). On dit que cet article a été publié plusieurs années après l'implémentation de ces règles dans un code afin de préserver l'avantage ainsi obtenu ; souvent les meilleurs codes d'optimisation ne sont pas des logiciels libres...

20. Celles-ci semblent basées sur le fait de ne pas calculer a priori tous les coûts réduits mais seulement quelques uns (c'est ici qu'une économie est faite) avec des stratégies de sélection/rejet des variables hors-base à tester...

21. On savait qu'il y avait une potentialité de complexité exponentielle puisque le nombre de bases maximal est C_n^m mais à la vue des résultats précédents il semblait être polynomial et il était difficilement envisageable (donc avant 1972...) que

G. Minty, ont montré qu'on peut construire un PL avec n variables et n contraintes d'inégalité (donc $2n$ variables en incluant les variables d'écart) où le nombre de base à visiter est égal à $2^n - 1$ avec la règle usuelle (de même avec les règles de *Bland*), il s'agit du PL suivant :

$$\begin{aligned} \min \quad & f(x) = - \sum_{j=1}^n 10^{n-j} x_j \\ & 2 \sum_{j=1}^{i-1} 10^{i-j} x_j + x_i \leq 100^{i-1}, \quad i = 1, \dots, n \\ & x \geq 0 \end{aligned}$$

Bien sûr il s'agit d'un PL construit très astucieusement pour visiter toutes les bases et on ne rencontre pas ce genre de PL en pratique mais ce résultat a donné un essor nouveau pour la recherche d'autres algorithmes ce qui a donné naissance en 1979 à l'algorithme de l'ellipsoïde de *L. G. Khachiyan* de complexité polynomiale²² et en 1983 à la méthode de *N. K. Karmarkar*, elle aussi de complexité polynomiale et qui fut à l'origine des méthodes de points intérieurs. Les méthodes de points intérieurs peuvent être plus performantes que celle du simplexe selon les cas (par exemple lorsque la matrice A est très creuse) mais les codes de simplexe (révisé) modernes font aussi de gros progrès. De plus la méthode du simplexe s'adapte facilement pour résoudre des PLNE et PLMI (cf chapitre suivant). La recherche a aussi progressé sur les cas où l'algorithme du simplexe devient exponentiel. Récemment, en 2022, *Sophie Huiberts* a montré que les PL du type de celui de *V. Klee* et *G. Minty* sont extrêmement rares au sens où une faible perturbation des coefficients d'un tel PL fait que la méthode redevient rapide.

l'algorithme puisse visiter suffisamment de bases pour être de complexité exponentielle.

22. Mais qui s'est avéré très lent en pratique et finalement n'est pas utilisé pour résoudre des PL, néanmoins ces travaux ont été fructueux et inspirants pour la communauté des théoriciens de la complexité.

Chapitre 3

Introduction à la résolution des PLNE et PLMI

Dans ce chapitre nous allons entrevoir comment on peut résoudre des PLNE et PLMI en utilisant les techniques de résolution des PL à variables continues. Il y a certains problèmes pour lesquels la solution est “naturellement” entière même en considérant les variables comme étant réelles. C’est par exemple le cas du premier problème d’affectation (cf 1.4.1). On connaît des conditions suffisantes sur la matrice et le second membre des contraintes qui permettent de reconnaître cette propriété¹. Dans ce cas on peut directement utiliser une méthode générale de résolution comme le simplexe (ou une autre) ou encore une méthode spécialisée. Cependant pour beaucoup de PLNE et PLMI relacher les contraintes d’intégrité (i.e. considérer les variables entières comme réelles, ce qui est appelé *relaxation* du PLNE ou du PLMI) ne donnera pas (sauf accident) une solution qui les respectera. C’est par exemple le cas du problème du sac à dos que l’on va exposer juste après. On pourrait se dire aussi que l’écart entre la solution d’un PLNE ou PLMI et celle du PL relaxé n’est pas très grande et qu’il suffit d’arrondir judicieusement la solution de ce dernier pour obtenir une solution du problème initial ou au moins une solution convenable. Ceci n’est pas vrai en général² ce qui est assez intuitif quand le PLNE ou PLMI comporte des variables binaires ou restreinte à un intervalle entier petit. Malgré cela c’est bien l’utilisation de PL relaxés qui a permis de faire de nets progrès dans la résolution de problèmes de combinatoire qui peuvent se poser comme des PLNE.

3.1 Une technique dont on va s’inspirer

Dans cette partie, on va expliquer une méthode de résolution de problèmes de *sac à dos* dont l’un des aspects est ré-utilisé pour résoudre des PLNE et PLMI. On expose ici une variante naïve facile à comprendre mais d’autres variantes peuvent être performantes pour résoudre ce genre de problèmes.

Problèmes de sac à dos

Un randonneur emporte dans son sac à dos un poids limité à d . Chaque objet i qu’il peut emporter pèse un poids a_i et possède une utilité c_i pour la randonnée.

Quels objets le randonneur doit-il emporter pour maximiser l’utilité totale, sans dépasser le poids total permis ?

En utilisant les variables binaires x_i ($x_i = 1$ si le randonneur prend l’objet i et $x_i = 0$ sinon), le

1. En particulier la matrice des contraintes doit être *totalelement unimodulaire*, ce point sera vu en GRO-2.
2. Cf exercice 1 de la feuille associée à ce chapitre.

problème se pose alors comme un PLNE :

$$\begin{aligned} \max_x F(x) &= \sum_{i=1}^n c_i x_i \quad (\text{maximisation de l'utilité totale}) \\ \sum_{i=1}^n a_i x_i &\leq d \quad (\text{limitation du poids total}) \\ x_i &\in \{0, 1\}, \forall i \in \llbracket 1, n \rrbracket \text{ i.e. } x \in \{0, 1\}^n \end{aligned}$$

D'une manière plus générale on peut considérer plusieurs contraintes que l'on écrit sous forme compacte $Ax \leq d$ avec $A \in \mathbb{R}^{m \times n}$, $d \in \mathbb{R}^m$ pour m contraintes :

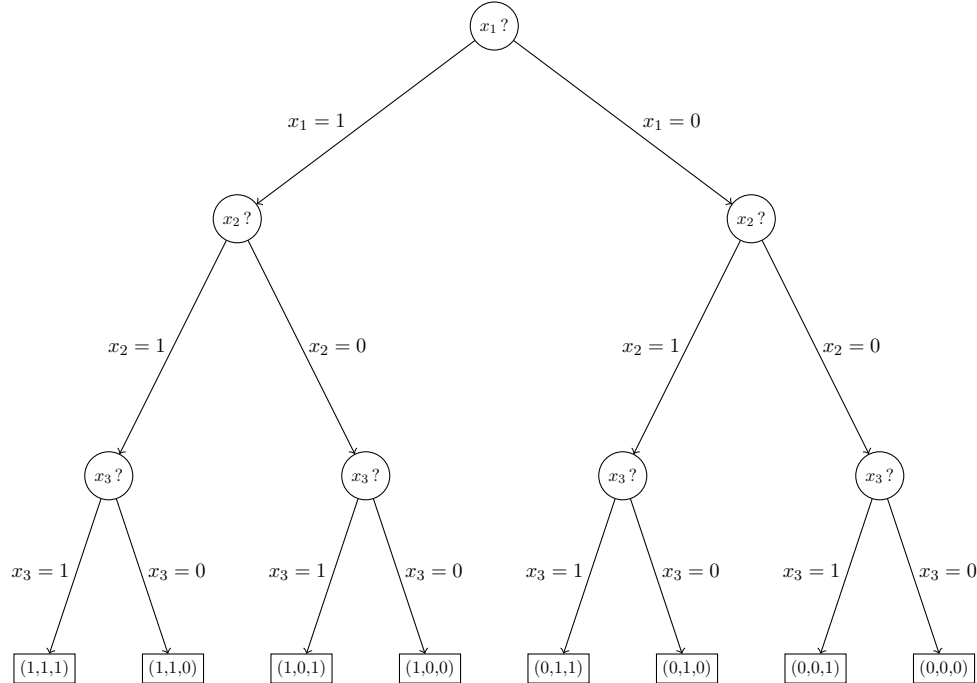
$$\begin{aligned} \max_x F(x) &= \sum_{i=1}^n c_i x_i \\ Ax &\leq d \\ x &\in \{0, 1\}^n \end{aligned}$$

Dans la suite on va considérer $c > 0$, $d > 0$, $a > 0$ (dans le cas vectoriel $A \geq 0$ mais aucune colonne ne doit être nulle³) mais la méthode peut se généraliser à d'autres cas.

Une méthode de séparation et évaluation

L'idée de cette méthode de *séparation et évaluation* (*branch and bound*) est de parcourir l'ensemble des solutions réalisables en écartant celles qui ne seront pas intéressantes, ses ingrédients sont les suivants :

- (i) On choisit un ordre d'examen des variables⁴, l'ensemble $B^n := \{0, 1\}^n$ peut être décomposé comme un arbre dont les feuilles sont tous ses éléments, cf figure suivante où $n = 3$:



3. Si $a_j = 0$ dans le cas scalaire et $A^j = 0$ dans le cas de plusieurs contraintes, l'objet j est nécessairement inclus dans le sac, on peut alors se ramener à un problème avec $n - 1$ objets.

4. On va considérer ici l'ordre naturel x_1 puis x_2 , etc.

Attention, tous les sommets de cet arbre (et donc les branches qui en découlent) ne seront pas forcément explorés. Se trouver sur un sommet S_k de l'arbre placé à une profondeur p du sommet, c'est avoir choisi les valeurs prises par $x_1, \dots, x_p$ (les variables de $x_{p+1}, \dots, x_n$ n'ayant pas encore été examinées) et vérifier d'autres considérations expliquées ci-après (en particulier S doit correspondre à un sous-ensemble de solutions réalisables (i.e. vérifiant les contraintes)).

(ii) À l'aide d'heuristiques (voir ci-après), on calcule **préalablement** une solution réalisable $(\bar{x}, \bar{F})$ qui ne sera sans doute pas optimale mais qu'on espère de bonne qualité... *Remarque* : le couple $(\bar{x}, \bar{F})$ est **mis à jour** dès qu'on trouve, à l'aide de l'arbre, une solution plus optimale.

(iii) À chaque nœud S_k de l'arbre⁵ sont attachés des indicateurs qui vont permettre d'orienter la recherche :

1. la **potentialité** du nœud : on évalue une valeur b_k qui est un **majorant** de la meilleure solution correspondant à S_k , soit celle obtenue si on met toutes les variables non encore examinées à 1 :

$$b_k = \sum_{i=1}^p c_i x_i^{(k)} + \sum_{i=p+1}^n c_i \times 1$$

2. le **respect des contraintes** : on calcule un indicateur e_k qui nous donne la provision restante du second membre des contraintes. Dans le cas d'une seule contrainte :

$$e_k := d - \sum_{j=1}^p a_j x_j^{(k)}$$

c'est le poids maximal dont on peut encore charger notre sac (après avoir fait une sélection parmi les objets $1, \dots, p$) et bien sûr e_k doit être positif ($e_k = 0$ est valable mais seule la feuille terminale avec $x_{p+1} = \dots = x_n = 0$ correspondra à une solution réalisable). Pour plusieurs contraintes :

$$(e_k)_i := d_i - \sum_{j=1}^p A_{i,j} x_j^{(k)}, i = 1, \dots, m \quad \Longleftrightarrow \quad e_k = d - \sum_{j=1}^p A^j x_j^{(k)}$$

auquel cas e_k est un vecteur de $\mathbb{R}^m$ (dont toutes les composantes doivent être ≥ 0).

Au début le sommet S_0 est créé avec $b_0 = \sum_{i=1}^n c_i$ et $e_0 = d$, ce sommet est mis dans *une file de priorité (max)* initialement vide. Lors de la création de chaque sommet/nœud S_k de l'arbre, celui-ci est inséré dans la file de priorité avec son indicateur b_k comme priorité.

(iv) **Création éventuelle d'un ou deux nœuds** : la recherche est pilotée par la file de priorité qui permet d'examiner les sommets selon leur potentialité⁶. Supposons que l'arbre contienne actuellement $k' + 1$ sommets $(S_0, S_1, \dots, S_{k'})$ et que le sommet S_k (de profondeur p) **sorte** de la file de priorité (car sa priorité est donc maximale parmi les sommets encore contenus dans la file), alors :

1. On teste⁷ si $b_k > \bar{F}$. Si le test est négatif aucun nouveau nœud n'est créé (à partir de S_k).
2. Si oui, on envisage alors la création de l'enfant gauche ; pour cela il faut simplement vérifier qu'en choisissant $x_{p+1} = 1$, on continue à respecter les contraintes, c'est dire si :

$$t := e_k - a_{p+1} \geq 0 \text{ (cas scalaire) , } t := e_k - A^{p+1} \geq 0 \text{ (plusieurs contraintes)}$$

5. Rappel : pour le sommet S_k on connaît donc la valeur des p premières variables $x_1^{(k)}, \dots, x_p^{(k)}$.

6. Il peut exister d'autres stratégies, par exemple la priorité pourrait être modulée en utilisant une combinaison basée sur la potentialité et le fait d'être proche d'une feuille de l'arbre.

7. Lors de la création de S_k , l'indicateur b_k était strictement supérieur au $\bar{F}$ courant mais celui-ci a pu être mis à jour et donc augmenté.

si oui l'enfant gauche $S_{k'+1}$ est créé avec :

$$\begin{aligned} b_{k'+1} &:= b_k \\ e_{k'+1} &:= t \end{aligned}$$

et mis dans la file **sauf** s'il correspond à un nœud terminal (i.e. $p+1 = n$) auquel cas on met à jour la solution "optimale" courante : $\bar{x} \leftarrow x^{(k'+1)}$, $\bar{F} \leftarrow b_{k'+1}$.

3. On envisage ensuite la création de l'enfant droit en choisissant $x_{p+1} = 0$; dans ce cas on ne va pas consommer de contraintes mais la potentialité diminue, on calcule alors :

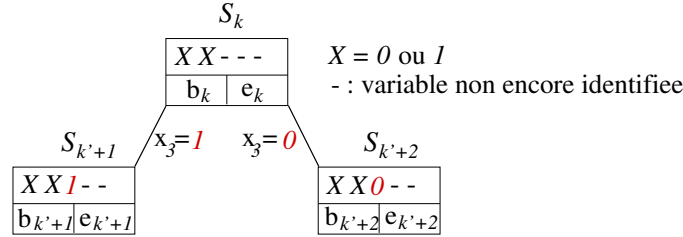
$$t := b_k - c_{p+1}$$

et si $t > \bar{F}$ alors on crée un nouveau sommet S_q ($q = k' + 2$ si l'enfant gauche est créé et $q = k' + 1$ sinon) avec :

$$\begin{aligned} b_q &:= t \\ e_q &:= e_k \end{aligned}$$

que l'on met dans la file de priorité **sauf** s'il correspond à un nœud terminal (i.e. $p+1 = n$), auquel cas on met à jour la solution "optimale" courante : $\bar{x} \leftarrow x^{(q)}$, $\bar{F} \leftarrow b_q$.

Illustration avec deux nœud supplémentaires créés (ici $p = 2$).



- (v) La recherche se termine lorsque la file est vide, dans ce cas $(\bar{x}, \bar{F})$ est le (un) couple optimal, i.e. $\bar{x}$ est un maximiseur et $\bar{F}$ un maximum de la fonction F sur le domaine $\mathcal{D} = \{x \in \{0, 1\}^n : Ax \leq d\}$.

Deux heuristiques

Nous allons voir deux heuristiques dont la deuxième ne s'applique pas directement aux problèmes à plusieurs contraintes⁸. Ces heuristiques peuvent être considérées comme des *algorithmes gloutons*.

Heuristique 1. Elle utilise les deux étapes suivantes :

- (i) on trie les objets selon leur utilité décroissante, après renumérotation on peut donc considérer que :

$$c_1 \geq c_2 \geq \dots \geq c_n$$

- (ii) on examine alors si on peut prendre l'objet 1, pour cela il faut que $a_1 \leq d$ et dans ce cas on pourra rajouter par la suite un poids maximal de $d - a_1$ dans le sac. Supposons que l'on ait déjà choisi parmi les objets $1, 2, \dots, k$, avec $k < n$, on connaît donc $x_1^{(1)}, \dots, x_k^{(1)}$ et l'on peut encore ajouter un poids de :

$$d' := d - \sum_{i=1}^k a_i x_i^{(1)}$$

dans le sac ; on met alors l'objet $k+1$ dans le sac si $a_{k+1} \leq d'$ auquel cas $d' \leftarrow d' - a_{k+1}$; on recommence jusqu'à avoir ainsi examiné tous les objets.

8. L'application de la deuxième heuristique pour ces problèmes fera l'objet d'un exercice de TD.

Heuristique 2. Seule l'étape (i) diffère. L'heuristique 1 ne tient absolument pas compte du poids des objets⁹, dans ce but il semble préférable de trier les objets selon leur ratio "utilité sur poids", après renumérotation, on peut donc considérer que :

$$\frac{c_1}{a_1} \geq \frac{c_2}{a_2} \geq \dots \geq \frac{c_n}{a_n}$$

On sélectionne ensuite les objets à prendre exactement selon l'étape (ii) de l'heuristique 1.

Remarque : l'heuristique 2 a l'air "plus subtile" que l'heuristique 1 mais pas en toujours en pratique ; de toute façon ces heuristiques ne coûtent pas grand chose (des opérations arithmétiques en $O(n)$, un tri $O(n \ln n)$, n tests), on peut donc les utiliser toutes les deux et prendre la meilleure ce qui nous fait une *méta-heuristique*.

Un exemple

On veut résoudre :
$$\begin{cases} \max_x F(x) = 16x_1 + 18x_2 + 15x_3 \\ x_1 + 4x_2 + 3x_3 \leq 7 \\ x_i \in \{0, 1\} \end{cases}$$

Estimations initiales : $b_0 = c_1 + c_2 + c_3 = 49$; $e_0 = 7$.

Détermination de $\bar{F}$:

i	1	2	3
c_i	16	18	15
c_i/a_i	16	4,5	5

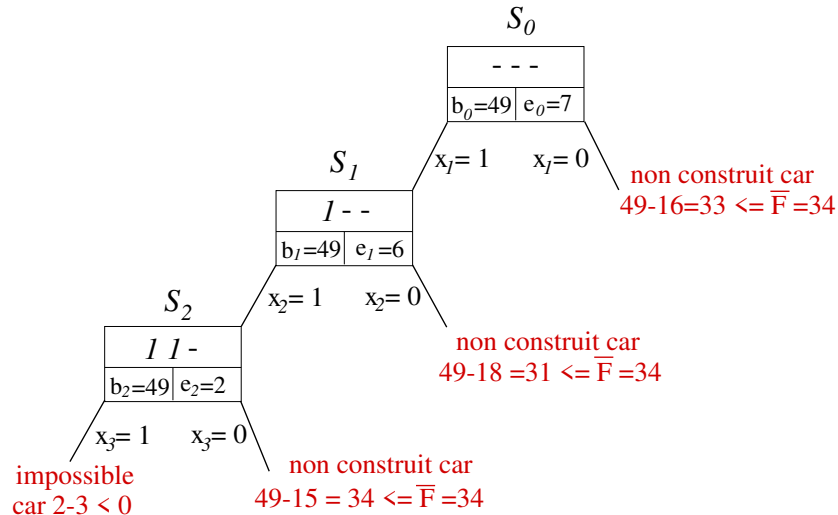
Pour l'heuristique 1, on examine les objets selon l'ordre 2, 1, 3, on obtient :

$$\bar{x}^{(1)} = (1, 1, 0), \quad \bar{F}^{(1)} = 16 + 18 = 34.$$

Pour l'heuristique 2 on examine les objets selon l'ordre 1, 3, 2, on obtient :

$$\bar{x}^{(2)} = (1, 0, 1), \quad \bar{F}^{(2)} = 16 + 15 = 31.$$

On choisit donc $\bar{x} = (1, 1, 0)$ et $\bar{F} = 34$.



9. L'objet le plus utile peut avoir une utilité seulement légèrement supérieure à son suivant mais un poids beaucoup plus important, il semble préférable de considérer cet autre objet avant le plus utile.

On obtient (on confirme que) $\max F = 34$ avec $x_1^* = 1$, $x_2^* = 1$, $x_3^* = 0$. Il s'agit d'un exemple minimaliste où la solution heuristique s'est révélée être la solution optimale.

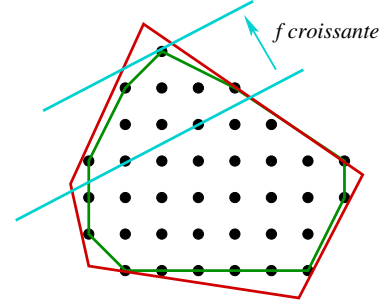
mini exercice 8

Donner les états successifs de la file de priorité dans cet exemple.

3.2 Optimiser sur l'enveloppe convexe

Un exemple en dimension 2.

On veut maximiser la fonction $f(x_1, x_2) = -x_1 + 2x_2$ sur l'ensemble E défini par les couples d'entiers (dessinés avec des points noirs) appartenant au polygone dont la frontière est dessinée en rouge. Il est assez intuitif de voir que cela est équivalent à maximiser f sur le domaine continu (polygone convexe dont la frontière est dessinée en vert) correspondant à l'enveloppe convexe de E ce qui pourrait ce faire (une fois l'enveloppe convexe calculée...) en utilisant la méthode du simplexe.



Résultat général

Le lemme suivant donne l'équivalence entre optimiser une fonction linéaire définie sur $E \subset \mathbb{R}^n$ et optimiser cette même fonction sur $\text{conv}(E)$.

Lemme 1

Soit $E \subset \mathbb{R}^n$ et $c \in \mathbb{R}^n$ alors :

$$\sup\{c^\top x : x \in E\} = \sup\{c^\top x : x \in \text{conv}(E)\}$$

De plus le suprémum de $c^\top x$ est atteint sur E si et seulement si il est atteint sur $\text{conv}(E)$.

Preuve :

- Première partie. Comme $E \subset \text{conv}(E)$ on a évidemment $\sup\{c^\top x : x \in E\} \leq \sup\{c^\top x : x \in \text{conv}(E)\}$. On montre maintenant l'inégalité inverse. On note $f^* := \sup\{c^\top x : x \in E\}$, on suppose que $f^* < +\infty$ (sinon le résultat est trivial). On définit le demi-espace $D := \{x \in \mathbb{R}^n : c^\top x \leq f^*\}$. D est convexe et par définition de f^* il contient E (en effet $\forall x \in E$ on a $c^\top x \leq f^*$ et donc $x \in D$). Par définition $\text{conv}(E)$ est contenu dans tout ensemble convexe contenant E donc $\text{conv}(E) \subset D$, soit $\sup\{c^\top x : x \in \text{conv}(E)\} \leq f^*$.
- Deuxième partie. Supposons que $\sup\{c^\top x : x \in E\} = c^\top \bar{x}$ pour un $\bar{x} \in E$. Alors il est clair que $\bar{x} \in \text{conv}(E)$ et d'après la première partie $\sup\{c^\top x : x \in \text{conv}(E)\} = c^\top \bar{x}$. Supposons maintenant qu'il existe $\bar{x} \in \text{conv}(E)$ tel que $\sup\{c^\top x : x \in \text{conv}(E)\} = c^\top \bar{x}$. D'après la caractérisation de $\text{conv}(E)$ comme l'ensemble des combinaisons convexes finies de points de E (voir proposition 1 (2.2.1)), il existe des points $x^1, \dots, x^p$ de E et des scalaires (strictement) positifs $\lambda_1, \dots, \lambda_p$ avec $\sum_{j=1}^p \lambda_j = 1$ tels que $\bar{x} = \sum_{j=1}^p \lambda_j x^j$. Notons j^* l'indice tel que $c^\top x^{j^*} = \max_{j \in [1, p]} c^\top x^j$, il vient :

$$c^\top \bar{x} = \sum_{j=1}^p \lambda_j c^\top x^j \leq \sum_{j=1}^p \lambda_j \left(\max_{k \in [1, p]} c^\top x^k \right) = c^\top x^{j^*} \underbrace{\sum_{j=1}^p \lambda_j}_{=1} = c^\top x^{j^*} \leq c^\top \bar{x}$$

Donc le suprémum $c^\top \bar{x}$ de la fonction $x \mapsto c^\top x$ définie sur $\text{conv}(E)$ est aussi atteint par le point $x^{j^*} \in E$.

mini exercice 9

Montrer qu'en fait tous les points $x^j, j = 1, \dots, p$ sont tels que $c^\top x^j = c^\top \bar{x}$.

Application de cette idée

Hélas, le calcul de l'enveloppe convexe du domaine d'un PLNE ou d'un PLMI est une opération coûteuse (complexité non polynomiale), ce principe n'est donc pas appliqué "tel quel". On pourrait se dire qu'il ne faudrait calculer qu'une partie de l'enveloppe convexe, grosso modo, là où la fonction est grande si on maximise (et inversement si on minimise), mais de même cela n'est pas si facile ! Néanmoins c'est un peu ce qui se passe avec la première méthode générale de résolution des PLNE élaborée en 1958 par *Ralph Gomory* que l'on explique très brièvement ci-après.

3.3 La première méthode générale (coupes de Gomory)

Soit f une fonction linéaire que l'on doit maximiser ou minimiser sur le domaine $\tilde{\mathcal{D}}$ intersection d'un polyèdre $\mathcal{D}$ de $\mathbb{R}^n$ avec $\mathbb{N}^n$. La méthode de *Gomory* procède de la façon suivante :

1. On résout d'abord (en utilisant la méthode du simplexe) le problème relaxé qu'on appelle $(\mathcal{P}_0)$ qui consiste à optimiser f sur $\mathcal{D}$ (toutes les n variables sont considérées comme réelles) ;
2. Soit $x^{(0)}$ la solution de $(\mathcal{P}_0)$, si $x^{(0)} \in \mathbb{N}^n$ c'est gagné ; sinon chaque variable non entière $x_i^{(0)}$ permet de définir une inégalité (ne faisant intervenir que les variables hors-base du dictionnaire optimal associé à $x^{(0)}$), appelée *coupe de Gomory* :

$$\mathcal{C}^{(i)} = \{x \in \mathbb{R}^n : \sum_{j \in H} \alpha_j^{(i)} x_j \geq \beta^{(i)}\}$$

et qui exclut la solution $x^{(0)}$, précisément :

$$x^{(0)} \notin \mathcal{D} \cup \mathcal{C}^{(i)} \text{ et } \text{conv}(\tilde{\mathcal{D}}) \subset \mathcal{D} \cup \mathcal{C}^{(i)} \subsetneq \mathcal{D}$$

3. On peut ajouter une ou plusieurs de ces coupes et créer le polyèdre $\mathcal{D}^{(1)}$ intersection de $\mathcal{D}$ avec ces coupes puis résoudre le problème $(\mathcal{P}_1)$ (optimiser f sur $\mathcal{D}^{(1)}$). Soit $x^{(1)}$ la solution de $(\mathcal{P}_1)$, si $x^{(1)} \in \mathbb{N}^n$ c'est gagné, sinon on procède de même en rajoutant de telles coupes de Gomory à partir des variables non entières, permettant de définir un domaine $\mathcal{D}^{(2)}$ et résoudre $(\mathcal{P}_2)$, etc.

Remarques :

- L'ajout successif de ces coupes font que les domaines $\mathcal{D}^{(1)}, \dots, \mathcal{D}^{(k)}, \dots$ qui vérifient :

$$\text{conv}(\tilde{\mathcal{D}}) \subset \dots \mathcal{D}^{(k)} \subsetneq \mathcal{D}^{(k-1)} \subsetneq \dots \subsetneq \mathcal{D}^{(1)} \subsetneq \mathcal{D}$$

vont ressembler localement à l'enveloppe convexe de $\tilde{\mathcal{D}}$ dans la zone d'intérêt.

- Les coupes de Gomory sont assez pratiques car elles sont déterminées à partir du dictionnaire optimal précédent et se rajoutent à ce dictionnaire. On obtient alors un nouveau dictionnaire qui est optimal (au sens des coûts réduits) mais dont la solution de base associée n'est plus réalisable. Le calcul est alors effectué par l'algorithme du *simplexe dual*. Ces détails seront vus en GRO2¹⁰
- *Gomory* démontre que sa méthode converge, cependant si les premiers essais numériques¹¹ semblent prometteurs, la méthode semble "patiner" et instable dès que l'on sort d'un petit problème. Les coupes de *Gomory* vont alors avoir mauvaise réputation et être quasi-oubliées (pour les calculs effectifs) durant près de 30 ans avant de renaître et d'être reconnue comme un outil puissant pour résoudre des PLNE et PLMI.

10. L'algorithme du simplexe dual mais aussi les deux types de coupes inventées par Gomory.

11. Pour cela Gomory utilise le premier langage dédié au calcul à savoir le fortran (de formula translation) dont la première version vient de sortir et pas un langage du type assembleur !

3.4 Une méthode de séparation et d'évaluation (branch and bound) basée sur les coupes entières

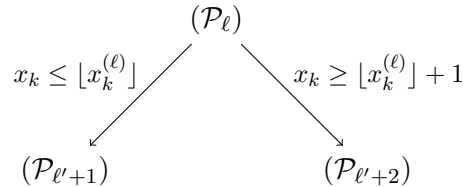
De même que précédemment, on considère une fonction linéaire f à maximiser ou minimiser sur le domaine $\tilde{\mathcal{D}}$ intersection d'un polyèdre $\mathcal{D}$ de $\mathbb{R}^n$ avec $\mathbb{N}^n$. On va supposer que l'on cherche à maximiser f . Cette méthode procède de la façon suivante :

1. Si possible obtenir une solution réalisable $(\bar{x}, \bar{F})$ en utilisant une ou des heuristiques. Sinon il faut attendre de trouver, grâce à la méthode, une première solution réalisable.
2. On va générer un arbre. Son sommet consiste en le problème relaxé appelé $(\mathcal{P}_0)$ c'est à dire la maximisation de f sur $\mathcal{D}^{(0)} := \mathcal{D}$. De même que pour la méthode basique vue pour résoudre un problème de sac à dos, les sommets de l'arbre, lors de leur création, sont mis dans une file de priorité¹².
3. Supposons que le problème $(\mathcal{P}_\ell)$ sorte de la file, on résout alors $(\mathcal{P}_\ell)$:
 - (a) On peut découvrir que $\mathcal{D}^{(\ell)} = \emptyset$.
 - (b) Sinon soit $(x^{(\ell)}, f(x^{(\ell)}))$ la solution optimale trouvée :
 - i. Si $(\bar{x}, \bar{F})$ est déjà défini **et** que $f(x^{(\ell)}) \leq \bar{F}$ inutile d'aller plus loin.
 - ii. Sinon si $x^{(\ell)}$ est entière alors l'optimum courant est mis à jour ou défini pour la première fois :

$$\bar{x} \leftarrow x^{(\ell)}, \quad \bar{F} \leftarrow f(x^{(\ell)})$$

- iii. Sinon on choisit¹³ une composante $x_k^{(\ell)}$ **non entière** et on rajoute deux sommets¹⁴ dans l'arbre correspondant aux problèmes :

$$\begin{aligned} (\mathcal{P}_{\ell'+1}) &: \max\{f(x), x \in \mathcal{D}^{(\ell'+1)}\}, \text{ où } \mathcal{D}^{(\ell'+1)} := \mathcal{D}^{(\ell)} \cap \{x : x_k \leq \lfloor x_k^{(\ell)} \rfloor\} \\ (\mathcal{P}_{\ell'+2}) &: \max\{f(x), x \in \mathcal{D}^{(\ell'+2)}\}, \text{ où } \mathcal{D}^{(\ell'+2)} := \mathcal{D}^{(\ell)} \cap \{x : x_k \geq \lfloor x_k^{(\ell)} \rfloor + 1\} \end{aligned}$$



Ces deux nouveaux nœuds sont mis dans la file de priorité.

Remarque : dans les cas (a), (b)-i et (b)-ii, $(\mathcal{P}_\ell)$ est un nœud terminal de l'arbre.

4. On itère ainsi jusqu'à ce que la file soit vide. Dans ce cas si $(\bar{x}, \bar{F})$ est défini, c'est la (une) solution optimale du problème (sinon cela veut dire que le problème n'a pas de solution réalisable, i.e. $\tilde{\mathcal{D}} = \emptyset$). L'arrêt peut aussi s'effectuer :
 - (a) lorsqu'un temps CPU maximum prédéfini est atteint, on peut bénéficier de la "solution" $(\bar{x}, \bar{F})$ si elle est définie mais on ne sait pas si elle est optimale (voir cependant le point c) ;
 - (b) même idée avec l'occupation mémoire ;
 - (c) il est possible de mesurer la qualité de $(\bar{x}, \bar{F})$ ¹⁵ : au cours des calculs on va disposer a priori d'un majorant F^Δ de plus en plus "ajusté" du maximum exact F^* de f sur $\tilde{\mathcal{D}}$ et lorsque $F^\Delta - \bar{F} \leq tol$ (ou un test en relatif comme $F^\Delta - \bar{F} \leq \bar{F} \times tol$) alors on peut arrêter le calcul en jugeant que la solution réalisable actuelle est suffisamment bonne (sans que l'on sache si elle est effectivement optimale).

12. On suppose avoir une stratégie qui permet de définir une priorité pour examiner les sommets de la file.

13. En cas de plusieurs variables non-entières, il existe différentes stratégies de choix pour en sélectionner une.

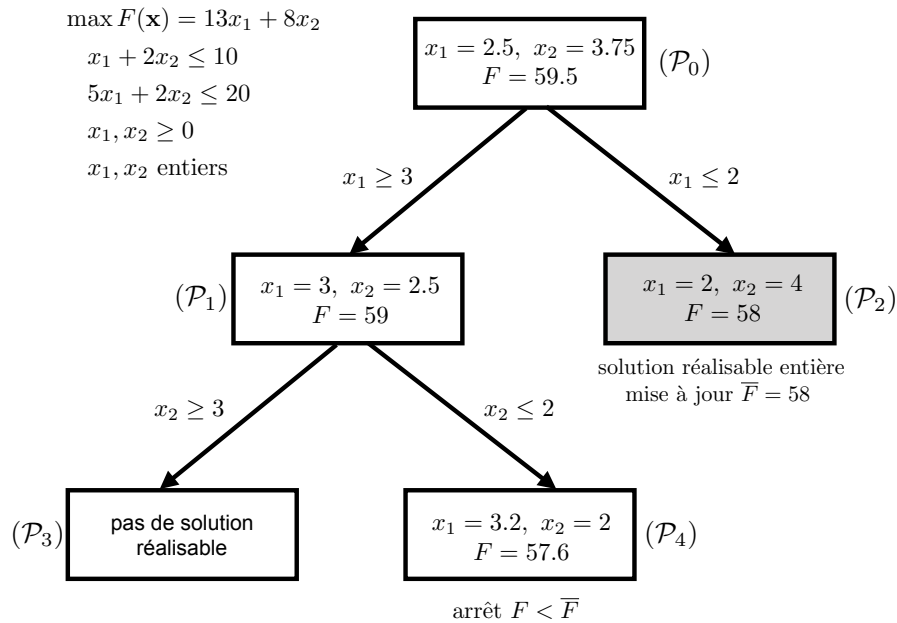
14. On suppose ici que l'arbre comprend à ce moment les $\ell' + 1$ sommets $(\mathcal{P}_0), \dots, (\mathcal{P}_{\ell'})$.

15. Il faut bien sûr avoir déjà réussi à calculer une solution réalisable $(\bar{x}, \bar{F})$.

Remarques :

- L'idée dans le (b)-iii est d'essayer de forcer la variable x_k à être entière mais cela n'impose pas qu'elle le soit effectivement dans la solution de chacun des 2 sous-problèmes. L'aspect essentiel est que le découpage du domaine de cette manière n'occulte aucune solution réalisable du problème initial.
- L'arbre obtenu peut être gigantesque mais il est assez intuitif que cette méthode converge.
- Pour définir une priorité d'exploration des sommets on peut utiliser :
 - La potentialité d'un sommet : par exemple les sommets $(\mathcal{P}_{\ell'+1})$ et $(\mathcal{P}_{\ell'+2})$ sont issus de la séparation (en 2) du domaine du problème $(\mathcal{P}_\ell)$ qu'on a résolu. On sait alors que $f(x^{(\ell)})$ est un majorant¹⁶ pour ces deux problèmes : si $\mathcal{D}^{(\ell'+1)} \neq 0$ alors $f(x^{(\ell'+1)}) \leq f(x^{(\ell)})$ et de même si $\mathcal{D}^{(\ell'+2)} \neq 0$ alors $f(x^{(\ell'+2)}) \leq f(x^{(\ell)})$.
 - Le nombre de composantes non entières : pour tenter d'obtenir rapidement une solution réalisable on peut privilégier les sommets ayant le moins de composantes non entières (même s'il n'est pas certain que les sous-problèmes n'aient pas plus de composantes non-entières...).
- La séparation en sous-problèmes permet d'exploiter les possibilités de calcul parallèle.
- De même que pour les coupes de Gomory, cette méthode va de pair avec l'utilisation de la méthode du simplexe car on construit très facilement les dictionnaires des sous-problèmes $(\mathcal{P}_{\ell'+1})$ et $(\mathcal{P}_{\ell'+2})$ à partir du dictionnaire optimal de $(\mathcal{P}_\ell)$. Il faut aussi résoudre avec la méthode du *simplexe dual*.

Un exemple



3.5 Les méthodes modernes

Pour le moment nous n'avons pas parlé des PLMI. En fait si on reprend la méthode précédente elle est très facile à adapter dans ce cas. Un PLMI est défini de la façon suivante :

$$\max_{x \in \tilde{\mathcal{D}}} \text{ ou } \min_{x \in \tilde{\mathcal{D}}} f(x) = c^\top x, \quad \tilde{\mathcal{D}} = \mathcal{D} \cap \{x : x_i \in \mathbb{N}, i \in I\}$$

16. Ce majorant sera en général strict sauf éventuellement lorsque la solution de $(\mathcal{P}_\ell)$ n'est pas unique, par exemple une arête dont l'un des sommets serait $x^{(\ell)}$ et cette arête comprendrait un point de coordonnées entières.

où $\mathcal{D}$ est un polyèdre de $\mathbb{R}^n$ et $I \subset \llbracket 1, n \rrbracket$ l'ensemble des indices spécifiant les variables qui doivent être entières. Les coupes entières se font alors uniquement sur celles-ci. De même, *Gomory*, peu après avoir décrit ses premières coupes appelée *fractionnaires*, met au point des coupes qui s'adaptent aux PLMI¹⁷, la méthode vue précédemment s'applique alors aussi avec des coupes issues des variables non entières qui doivent l'être.

Sans entrer dans les détails, les méthodes modernes combinent les deux approches. Essentiellement lorsque l'on décide de couper le domaine en deux à l'aide d'une coupe entière¹⁸, on peut à ce moment rajouter des coupes du type *Gomory* ou d'autres, il existe en effet tout un panel de "coupes" différentes certaines étant adaptées à des problèmes particuliers. Les coupes de *Gomory* semblent être toujours actuellement les meilleures coupes génériques. Un des aspects qui a permis leur retour est la meilleure stabilité et efficacité des solveurs de simplexe.

Les logiciels associés sont d'une extrême complexité, en jouent sur différents tableaux :

- utilisation du parallélisme ;
- algorithmes de simplexe (primal et dual) utilisant des techniques d'algèbre linéaire creuse très sophistiquées ;
- utilisation d'un **pré-solveur** qui analyse le problème et peut le simplifier (voire parfois le résoudre directement) et/ou reconnaître sa forme et en déduire des stratégies de coupes adaptées ou peut-être utiliser une méthode spécialisée ;
- intégration des derniers développement de la recherche.

17. Qu'on appelle GMI Cuts pour Gomory Mixed Integer Cuts ; ses coupes sont en fait meilleures que les coupes fractionnaires lorsqu'on les utilise sur des PLNE, elles sont donc systématiquement utilisées.

18. Remarque : d'autres coupes peuvent faire de même, c'est à dire couper le domaine en 3 avec une zone milieu dans laquelle on sait qu'aucune solution réalisable ne peut s'y trouver et 2 autres qui vont permettre de définir deux sous-problèmes.

Chapitre 4

Graphes

4.1 Définitions

Définition 10 *Graphe (fini)*

Un graphe (fini) est un objet défini par un couple $G = (E, \Gamma)$ où :

- E est un ensemble fini d'éléments appelés **sommets** (ou nœuds) ;
- et où Γ est :
 - soit un ensemble fini de couples (i, j) avec $i, j \in E$ appelés **arcs** ou arêtes orientées (les arcs (i, j) et (j, i) sont distincts) auquel cas on parle de **graphe orienté** ;
 - soit un ensemble fini de paires $\{i, j\}$ appelées **arêtes** auquel cas on parle de **graphe non orienté**.

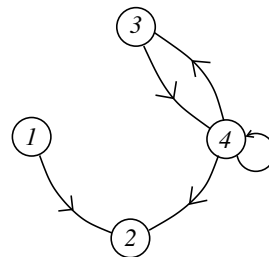
Remarques :

- dans un graphe orienté, un sommet i peut avoir un arc sur lui-même, i.e. un arc (i, i) ce qu'on appelle une **boucle** ; un graphe orienté sans boucle est dit *simple* ;
- un graphe orienté est plus général qu'un graphe non orienté puisque le premier peut “simuler” le deuxième en ayant deux arcs (i, j) et (j, i) pour chaque arête $\{i, j\}$ mais dans ce cas on comprend (surtout d'un point de vue informatique) que la version non orienté est plus légère.

Exemple :

Un graphe orienté (avec une boucle) :

$$E = \{1, 2, 3, 4\}$$
$$\Gamma = \{(1, 2), (3, 4), (4, 2), (4, 3), (4, 4)\}$$



Pour un exemple de **graphe non orienté** voir le problème posé sur un graphe dans l'introduction (1.5) avec le mini-exercice 2.

Exemples de modélisation par des graphes :

- Réseau routier, les sommets sont les intersections des routes, les arêtes représentent les routes. Avec les sens interdit en ville, la modélisation par un graphe orienté s'impose !
- Cheminement dans un réseau informatique ; réseaux/circuits électriques/électroniques ;
- Web modélisé par un graphe. Les sommets sont les pages Web et les arêtes sont les liens hypertexte entre ces différentes pages.
- Graphe des dépendances d'une application informatique ; réseaux sociaux.

Définition 11 Graphe valué (ou pondéré)

C'est un graphe (E, Γ) auquel on associe à chaque arc ou arête de Γ une valeur réelle (ou entière) $c : \Gamma \rightarrow \mathbb{R}$ appelée **capacité**. On le note (E, Γ, c) . La capacité d'un arc (i, j) est notée c_{ij} et apparaît graphiquement le plus souvent sous cette forme indiquant le sens $i \rightarrow j$:



Exemples : La capacité c_{ij} représente par exemple la longueur du tronçon de route (i, j) , le nombre max. de voitures par unité de temps entre deux villes i et j , la bande passante maximale entre les serveurs i et j ...

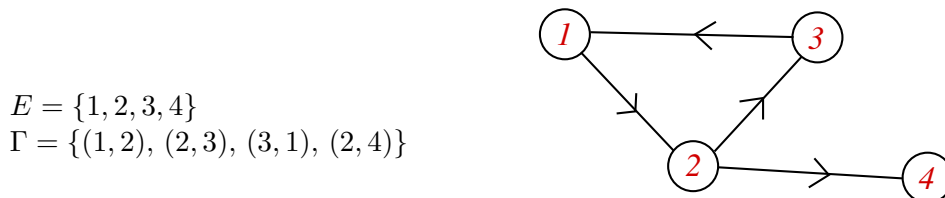
Remarques :

- Souvent une capacité est positive.
- Dans certain cas chaque arc ou arête peut être dotée de plusieurs valeurs. Par exemple si on modélise une route par un arc, on peut avoir besoin de sa distance, d'une limitation de la vitesse sur cette route, de l'état effectif de cette route (la limitation peut être 80 ou 90 km/h alors qu'il est difficile d'atteindre effectivement cette vitesse).

4.2 Représentation d'un graphe, matrices associées à un graphe

4.2.1 Listes d'adjacence

Pour un **graphe orienté** (E, Γ) , on peut définir pour chaque sommet i la liste $S(i)$ de ses successeurs, i.e., les sommets j tels que l'arc (i, j) existe dans le graphe. De même on peut aussi définir la liste $P(i)$ de ses prédécesseurs, i.e. les sommets j tels que l'arc (j, i) existe dans le graphe. Un sommet sans prédécesseur est appelé une **source** et un sommet sans successeur est appelé un **puits**. Exemple :



sommet	successeur S	prédécesseur P
1	2	3
2	3, 4	1
3	1	2
4	—	2

Pour ce graphe, il n'y a aucune source et un seul puits (4).

Pour un **graphe non orienté** on définit en chaque point la liste $V(i)$ de ses voisins, c'est à dire des sommets j tels que l'arête $\{i, j\}$ existe dans le graphe.

Remarque : la même information peut être codée par une matrice comme nous allons le voir ci-après.

4.2.2 Matrice d'adjacence sommet-sommet

Définition 12 Matrice d'adjacence sommet-sommet

Pour un graphe à n sommets¹, c'est une matrice booléenne de taille $n \times n$ telle que :

$$a_{ij} = \begin{cases} 1 & \text{si l'arc } (i, j) \text{ ou l'arête } \{i, j\} \text{ existe dans le graphe auquel (cas on a aussi } a_{ji} = 1) \\ 0 & \text{sinon} \end{cases}$$

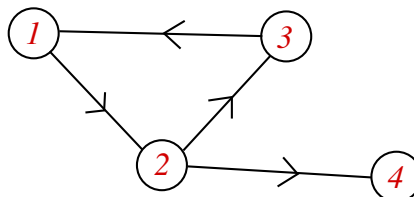
1. On considère ici que chaque sommet est repéré par un entier $i \in \llbracket 1, n \rrbracket$.

Pour un graphe valué par $\{c_{ij}\}$, on remplace le 1 par la capacité de l'arc ou de l'arête. Dans le cas d'un graphe non-orienté, on obtient des matrices symétriques.

Exemple :

$$E = \{1, 2, 3, 4\}$$

$$\Gamma = \{(1, 2), (2, 3), (3, 1), (2, 4)\}$$



La matrice d'adjacence est $A =$

	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	1	0	0	0
4	0	0	0	0

Remarques :

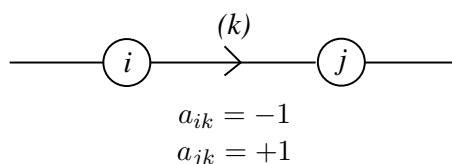
- pour un graphe orienté, la ligne i permet de repérer les successeurs du sommet i et la colonne i ses prédécesseurs ;
- pour un graphe non orienté la ligne i ou la colonne i (aspect symétrique) nous donne les voisins ;
- la matrice d'adjacence est utile en pratique lorsqu'un graphe est suffisamment dense (c'est à dire lorsqu'un sommet est relié à beaucoup d'autres sommets) sinon on utilise plutôt les listes d'adjacence ou un stockage de type "matrice creuse"² de la matrice d'adjacence (ce qui peut revenir un peu au même selon le stockage creux utilisé cf TP.).

4.2.3 Matrice d'incidence sommet-arc

Définition 13 *Matrice d'incidence sommet-arc*

Soit un graphe orienté **sans boucle** c-à-d sans arc (i, i) , avec n sommets (numérotés de 1 à n) et m arcs (idem numéroté de 1 à m). On définit A la **matrice d'incidence** de taille $n \times m$:

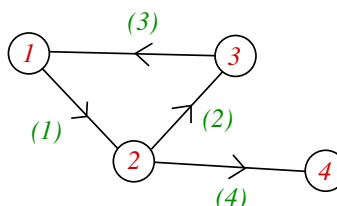
$$a_{ik} = \begin{cases} -1 & \text{si le sommet } i \text{ est l'extrémité } \mathbf{initiale} \text{ de l'arc } k \\ +1 & \text{si le sommet } i \text{ est l'extrémité } \mathbf{terminale} \text{ de l'arc } k \\ 0 & \text{sinon} \end{cases}$$



Exemple :

$$E = \{1, 2, 3, 4\}$$

$$\Gamma = \{\underbrace{(1, 2)}_1, \underbrace{(2, 3)}_2, \underbrace{(3, 1)}_3, \underbrace{(2, 4)}_4\}$$

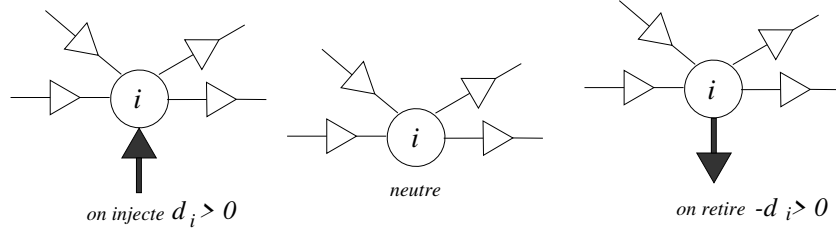


2. Pour une matrice qui comporte une faible part de coefficients non nuls, les stockages classiques (appelés "pleins") utilisent beaucoup de mémoire pour rien, un stockage creux ne stocke en général que des éléments non nuls.

La matrice d'incidence est $A =$

	(1,2)	(2,3)	(3,1)	(2,4)
1	-1	0	+1	0
2	+1	-1	0	-1
3	0	+1	-1	0
4	0	0	0	+1

Cette matrice est utile pour exprimer une loi de conservation de type *Kirchhoff*. Supposons qu'on modélise un flot (pensez par exemple à une intensité de courant ou un flux d'eau, etc.) dans un réseau modélisé par un graphe orienté (n sommets et m arcs). Le flot dans l'arc (i, j) est noté $f_{i,j}$. Un sommet i , en plus de ses successeurs ou prédécesseurs, peut injecter dans le réseau un flot d_i provenant de l'extérieur ($d_i > 0$) ou apporter un flot du réseau vers l'extérieur (on a alors $d_i < 0$) ou être neutre ($d_i = 0$) :



La conservation de ce flot en un sommet/nœud i s'écrit :

$$\underbrace{d_i}_{\substack{d_i > 0 \text{ flot entrant} \\ d_i < 0 \text{ flot sortant}}} + \underbrace{\sum_{k \in P(i)} f_{k,i}}_{\text{flot entrant}} = \underbrace{\sum_{j \in S(i)} f_{i,j}}_{\text{flot sortant}} \iff - \sum_{j \in S(i)} f_{i,j} + \sum_{k \in P(i)} f_{k,i} + d_i = 0$$

ce qui, grâce à la matrice d'incidence, peut s'écrire de manière compacte :

$$Af + d = 0$$

En effet considérons la ligne i , on peut l'écrire (sachant aussi que A ne contient que les valeurs -1, 0 et 1) :

$$\begin{aligned} (Af + d)_i &= (Af)_i + d_i \\ &= \sum_{\ell=1}^m a_{i,\ell} f_\ell + d_i \\ &= \sum_{\ell: a_{i,\ell} = -1} -f_\ell + \sum_{\ell: a_{i,\ell} = 1} f_\ell + d_i \end{aligned}$$

On sait que $a_{i,\ell} = -1$ pour les arcs ℓ (numérotation 1d) correspondant à des couples (i, j) avec $j \in S(i)$ et que $a_{i,\ell} = 1$ pour tous les arcs ℓ correspondant à des couples (k, i) avec $k \in P(i)$. D'où :

$$(Af + d)_i = - \sum_{j \in S(i)} f_{i,j} + \sum_{k \in P(i)} f_{k,i} + d_i$$

ce qui est exactement la loi de conservation du flot au sommet i .

4.3 Codage informatique d'un graphe

Si on considère des graphes **peu denses** (cas le plus fréquent), c'est à dire tels que *la plupart des sommets ne sont reliés (par un arc ou une arête) qu'à une petite partie des autres sommets*, on utilise **des listes d'adjacence** (graphe orienté) ou **des listes de voisins** (graphe non orienté). Pour chaque sommet i , on dispose de :

- la liste $S[i]$ de ses successeurs (ou la liste $V[i]$ de ses voisins) ;
- souvent pour un graphe orienté on va aussi disposer de la liste $P[i]$ de ses prédécesseurs ;
- si de plus les arcs sont valués, il y a une autre liste associée $Val[i]$ ayant la même structure que $S[i]$: si j est le k ème élément de $S[i]$ ($j = S[i][k]$) la valuation de l'arête (i, j) est accessible par $Val[i][k]$.

Remarques :

- la liste des prédécesseurs est utile pour certains algorithmes et peut être construite à partir de celle des successeurs en cas de besoin ;
- en cas de plusieurs valuations, on peut avoir plusieurs listes comme Val ;
- enfin certaines bibliothèques de codes travaillant sur les graphes vont demander des entrées du type matrice creuse (ce qui peut être juste un enrobage sur les listes précédentes, ce point sera vu en TP).

À partir de ces considérations, il y a deux familles de codage possible :

1. si $\#E = n$, l'idée naturelle est de numérotter les noeuds/sommets du graphe de 1 à n (ou de 0 à $n - 1$), et d'utiliser un tableau $T : T[i]$ permettant d'accéder aux diverses listes (disons $T[i].S$, $T[i].P$ et $T[i].Val$) associées au sommet i .

Avantages :

- légèreté (faible occupation mémoire) ;
- accès à un sommet donné très rapide.

Inconvénients :

- ajouts et suppressions de sommets et d'arêtes difficiles ;
- répondre à une question du genre : “*Est ce que le noeud j est un successeur du noeud i ?*” demande de balayer la liste des successeurs de i opération pas si rapide s'il y a suffisamment de successeurs ; normalement pour un graphe peu dense, il y a peu de successeurs cependant parfois un sommet peut être relié à une grosse partie des autres sommets et dans ce cas un table de hachage peut être préférable (voir ci-après).

2. **Autre possibilité :** utiliser une **table de hachage** (appelée dictionnaire en python). À chaque sommet est associé un identificateur unique³. Si i est un tel identificateur alors de même $T[i]$ permettra d'accéder aux diverses listes qui peuvent elles-même être des dictionnaires.

Avantages :

- ajouts et suppressions de sommets et d'arêtes faciles et rapides ;
- si la liste des successeurs est un dictionnaire, répondre à la question “*Est ce que le noeud j est un successeur du noeud i ?*” est une opération rapide (est ce que j est une clé du dictionnaire $T[i].succ$?).

Inconvénients :

- lourd en mémoire !
- l'accès aux caractéristiques d'un sommet est un peu moins rapide (il faut décoder la clé et un peu plus pour avoir la case mémoire correspondante).

Remarque : dans certains cas on a besoin de ces deux types de codage, i.e. de passer de l'un à l'autre. Par exemple lors de la construction d'un graphe, il est assez pratique d'utiliser la famille 2 (table de hachage), puis pour faire certains calculs (en général en utilisant une fonction d'une bibliothèque) de convertir en listes d'adjacences et/ou matrice d'adjacence creuse car ils seront beaucoup plus rapides puis si on doit modifier le graphe (enlever/rajouter certains sommets ou arcs/arêtes) de revenir en table de hachage. Ce point sera peut-être abordé en GRO-2.

3. Souvent une chaîne de caractères ou tout objet utilisable comme clé par le langage, python accepte beaucoup de choses comme clés...

4.4 Chemins et chaînes, cycles et circuits

Dans un graphe (E, Γ) **orienté**, on dit qu'il existe un **chemin** (de longueur p) du sommet i au sommet j s'il existe une séquence de sommets $\langle i_0, i_1, \dots, i_p \rangle$ avec $i_0 := i$ et $i_p := j$ et telle que :

$$\forall k \in \llbracket 0, p-1 \rrbracket, \quad (i_k, i_{k+1}) \in \Gamma, \text{ i.e. est un arc du graphe.}$$

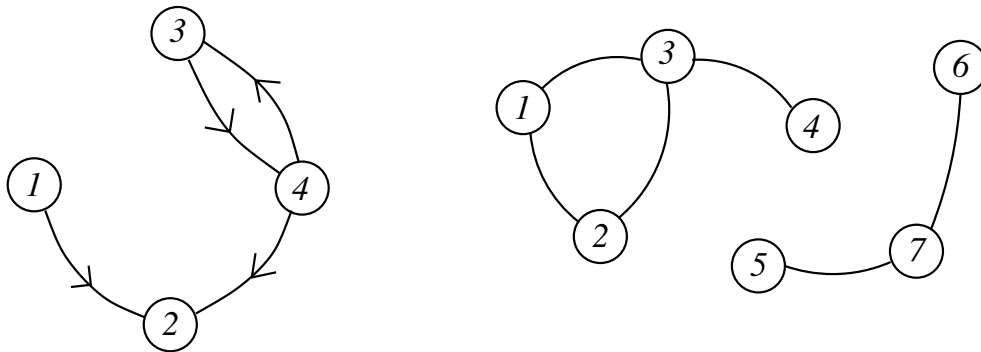
Pour un graphe **non orienté** même définition (ici $\{i_k, i_{k+1}\}$ est une arête du graphe) mais dans ce cas on parle plutôt d'une **chaîne** (de longueur p).

Un **chemin** ou une **chaîne** est dit *élémentaire* si ses sommets sont tous distincts.

On s'intéresse souvent aux chemins ou aux chaînes élémentaires partant d'un sommet s du graphe. Cela permet par exemple de connaître les sommets t accessibles à partir de s , de calculer leur longueur⁴ on s'intéresse souvent au chemin le plus court ce que permet l'algorithme de *Dijkstra* que vous avez peut-être déjà rencontré.

Dans un graphe **orienté** un **circuit** est un **chemin** dont le premier et le dernier sommet sont identiques. Idem, pour un graphe **non orienté** une **chaîne** dont le premier et le dernier sommet sont identiques est un **cycle** (les deux termes cycle et circuit semblent cependant interchangeables puisqu'on parle de graphe orienté acyclique, i.e. sans circuit...).

Exemples :



Sur le graphe orienté (à gauche) :

- $\langle 3, 4, 2 \rangle$ est un chemin élémentaire (de longueur 2) allant du sommet 3 au sommet 2 ;
- $\langle 4, 3, 4, 2 \rangle$ est un chemin non élémentaire allant de 4 à 2 ;
- $\langle 4, 3, 4 \rangle$ est un circuit.

Sur le graphe non orienté (à droite) :

- $\langle 4, 3, 2, 1, 3 \rangle$ est une chaîne non élémentaire allant de 4 à 3 ;
- $\langle 1, 3, 4 \rangle$ et $\langle 1, 2, 3, 4 \rangle$ sont deux chaînes élémentaires allant de 1 à 4 ;
- $\langle 1, 2, 3, 1 \rangle$ est un cycle.

On peut observer que ce graphe n'est pas connexe. Pour un graphe **non orienté** être **connexe** signifie qu'il existe une chaîne reliant tout sommet à tous les autres sommets. Ici on voit que le graphe n'est pas connexe (par exemple il n'existe pas de chaîne reliant 4 et 5). Ce graphe comporte deux composantes connexes (on a envie de dire qu'il s'agit de deux graphes distincts !). Les différentes composantes connexes d'un graphe non orienté peuvent s'obtenir via un **parcours de graphe** (voir ci-après).

4. Soit au sens du nombre d'arcs ou d'arêtes ou, dans le cas d'un graphe valué où les capacités sont les longueurs des arcs/arêtes, au sens de la plus petite distance entre s et t .

4.5 Parcours de graphes

Un certain nombre d'algorithmes utilisent de manière sous-jacente un parcours de graphe. Par exemple en partant d'un sommet s , on cherche à découvrir tous les sommets accessibles (à partir de s). On peut aussi vouloir parcourir **tous** les sommets d'un graphe (certains sommets ne sont pas forcément accessibles à partir d'un unique sommet). Nous allons voir ci-après les deux principaux parcours utilisés.

4.5.1 Le parcours en profondeur

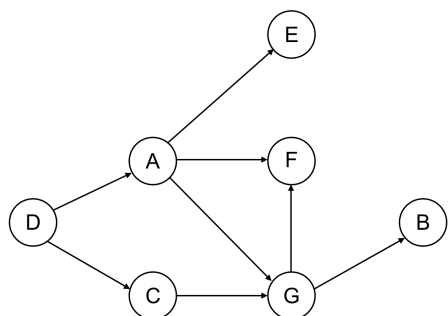
Il est appelé aussi DFS (pour *depth first search*).

À partir d'un sommet donné s

On utilise une **pile** P qu'on initialise avec s et un tableau de booléen **marque** de taille n (le nombre de sommets du graphe) qu'on initialise à faux sauf pour s . On procède de la façon suivante :

```
tant que  $P$  est non vide :  
    soit  $i$  le sommet situé en haut de pile  
    si il existe un sommet  $j$  non marqué successeur de  $i$  alors :  
        on marque  $j$   
        on empile  $j$   
    sinon on supprime  $i$  de la pile.
```

Testons cet algorithme pas à pas sur le graphe suivant à partir du sommet A :



listes des successeurs :

```
 $S[A] = [E, F, G]$   
 $S[B] = []$   
 $S[C] = [G]$   
 $S[D] = [A, C]$   
 $S[E] = []$   
 $S[F] = []$   
 $S[G] = [B, F]$ 
```

À l'initialisation, on marque A et on l'empile, puis :

1. le premier successeur de A non marqué est E qu'on marque et empile ;
2. E est en haut de pile mais n'a pas de successeur, on l'enlève de la pile ;
3. A est de nouveau en haut de pile mais maintenant le premier successeur non marqué est F qu'on marque et empile ;
4. F est en haut de pile mais n'a pas de successeur, on l'enlève de la pile ;
5. A est de nouveau en haut de pile et le premier successeur non marqué est maintenant G qu'on marque et empile ;
6. G est en haut de pile, son premier (et seul) successeur non marqué est B qu'on marque et empile ;
7. B est alors en haut de pile mais n'a pas de successeur, on l'enlève de la pile ;
8. G est de nouveau en haut de pile mais n'a plus de successeur non marqué, on l'enlève de la pile ;
9. A se retrouve en haut de pile mais n'a plus de successeur non marqué, on l'enlève de la pile ;
10. fin (pile vide).

L'ordre de découverte des sommets est A, E, F, G, B , ce sont les sommets qui sont accessibles à partir de A . Même sur cet exemple jouet, on comprend bien le nom de ce parcours puisqu'on essaie d'aller le plus en profondeur possible à partir d'un sommet (si E avait des successeurs ils auraient été visités avant les sommets F et G).

Pour parcourir tous les sommets

Dans la version précédente, on ne découvre que les sommets accessibles à partir du sommet de départ que l'on s'est donné. Si on veut parcourir tous les sommets, il faut "enrober" l'algorithme précédent par une boucle externe chargée de vérifier si certains sommets n'ont pas été visités (i.e. marqués) et repartir avec un de ces sommets.

```
tant que qu'il existe un sommet  $s$  non marqué :  
    marquer  $s$   
    empiler  $s$   
    tant que  $P$  est non vide :  
        soit  $i$  le sommet situé en haut de pile  
        si il existe un sommet  $j$  non marqué successeur de  $i$  alors :  
            on marque  $j$   
            on empile  $j$   
        sinon on supprime  $i$  de la pile.
```

Sur l'exemple précédent, si la boucle la plus externe explore les sommets dans l'ordre lexicographique (la liste des sommets serait $[A, B, C, D, E, F, G]$ alors tout se passe exactement comme précédemment (A est le premier sommet être empilé) et quand on revient sur cette boucle le premier sommet non marqué est C qui est alors empilé mais son seul successeur (G) est déjà marqué, C est enlevé de la pile qui est de nouveau vide ; on revient sur la boucle externe et on découvre alors le sommet D . L'ordre de découverte des sommets est donc :

A, E, F, G, B, C, D

Si on ajoute au fur et à mesure chaque sommet dépilé dans une liste en complétant la dernière ligne de l'algorithme (on supprime i de la pile et on l'ajoute dans cette liste) , on obtient :

E, F, B, G, A, C, D

Si on regarde l'ordre inverse :

D, C, A, G, B, E, F

il s'agit d'un **tri topologique** des sommets du graphe où l'ordre des sommets est tel qu'un sommet se trouve toujours avant ses successeurs : soit s_k le k ème sommet d'un tel tri alors tous les successeurs de s_k se trouvent à une place $k' > k$ dans ce tri. Il s'agit d'une des propriétés de ce parcours en profondeur valable si le graphe ne comporte pas de **circuits**.

Une autre utilisation de ce parcours est de calculer les composantes connexes d'un graphe non orienté. Pour cela il suffit de modifier légèrement l'algorithme précédent de cette façon :

```
tant que qu'il existe un sommet  $s$  non marqué :  
    marquer  $s$   
    empiler  $s$   
    initialiser une nouvelle liste  $L$  avec  $s$   
    tant que  $P$  est non vide :  
        soit  $i$  le sommet situé en haut de pile  
        si il existe un sommet  $j$  non marqué successeur de  $i$  alors :  
            on marque  $j$   
            on empile  $j$   
            on ajoute  $j$  dans  $L$   
        sinon on supprime  $i$  de la pile.
```

Chaque liste ainsi obtenue correspondra à une composante connexe du graphe (une seule liste si le graphe est connexe).

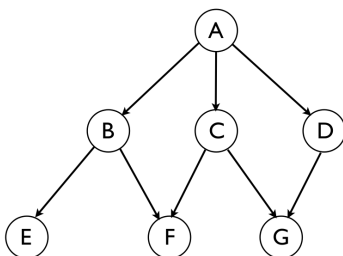
4.5.2 Parcours en largeur

À partir d'un sommet donné s

Appelé aussi BFS (pour *breath first search*) ce parcours opère lui en largeur ; il utilise **une file** (liste FIFO) pour l'exploration des sommets et de même un tableau de booléens initialisé à faux. À l'initialisation on marque s et on le met dans la file puis :

```
tant que que la file est non vide :  
    supprimer le sommet en tête de la file (soit  $i$  ce sommet)  
    pour  $j \in S[i]$  : (boucle selon l'ordre des successeurs de  $i$  dans la liste  $S[i]$ )  
        si  $j$  est non marqué :  
            on marque  $j$   
            on met  $j$  dans la file
```

Testons cet algorithme pas à pas sur le graphe suivant à partir du sommet A (on suppose que l'ordre des listes de successeurs est donné par l'ordre alphabétique) :



1. la file est donc initialisée avec le sommet A qui est marqué et on rentre dans la boucle **tant que** ;
2. A est enlevé de la file, la boucle **pour** marque et met successivement dans la file les 3 successeurs de A , soit B , C et D ;
3. on retourne sur la boucle **tant que** , le sommet en tête (B) est supprimé, la boucle **pour** marque et met dans la file les sommets E et F ;
4. on retourne sur la boucle **tant que** , le sommet en tête (C) est supprimé, la boucle **pour** marque et met dans la file le sommet G (le successeur F de C est déjà marqué) ;
5. on retourne sur la boucle **tant que** , le sommet en tête (D) est supprimé, mais la boucle **pour** ne fait rien (le successeur de D qui est G ayant déjà été marqué).
6. ensuite les 3 sommets encore présents dans la file (E , F et G) n'ayant pas de successeurs sont supprimés l'un après l'autre et l'algorithme s'arrête la file étant vide.

Pour parcourir tous les sommets

Dans l'exemple précédent tous les sommets étaient accessibles à partir de A , mais, de même que pour le parcours DFS, si on veut être sûr de parcourir tous les sommets du graphe on encapsule l'algorithme précédent par une boucle externe :

```
tant que qu'il existe un sommet  $s$  non marqué :  
    marquer  $s$  et le mettre dans la file  
    tant que que la file est non vide :  
        supprimer le sommet en tête de la file (soit  $i$  ce sommet)  
        pour  $j \in S[i]$  : (boucle selon l'ordre des successeurs de  $i$  dans la liste  $S[i]$ )  
            si  $j$  est non marqué :  
                on marque  $j$   
                on met  $j$  dans la file
```

Remarque : de même que le parcours le profondeur, le parcours en largeur peut être utilisé (avec la même petite modification) sur un graphe non orienté pour calculer les composantes connexes du graphe.

Calcul des distances du chemin le plus court entre un sommet et les autres

Le parcours en largeur, à partir d'un sommet s donné permet aussi de calculer pour un graphe (orienté ou non), la plus petite distance entre s et tous les autres sommets accessibles à partir de s en terme de **nombre d'arcs ou d'arêtes** ; pour cela on utilise un tableau D à n éléments, avec la petite modification suivante :

```
D[s] = 0 ; marquer s ; mettre s dans la file
tant que que la file est non vide :
    supprimer le sommet en tête de la file (soit  $i$  ce sommet)
    pour  $j \in S[i]$  : (boucle selon l'ordre des successeurs (ou voisins) de  $i$ )
        si  $j$  est non marqué :
            on marque  $j$ 
            on met  $j$  dans la file
             $D[j] = D[i] + 1$ 
```

Le tableau D peut être initialisé avec -1. En sortie $D[i]$ donne la longueur (en nombre d'arcs ou d'arêtes) du chemin le plus court entre les sommets s et i (avec $D[i]=-1$ si i n'est pas accessible à partir de s).

Remarques :

- cette propriété (de plus court chemin) est utilisée dans l'algorithme *d'Edmond-Karp* basé sur principe de *Ford-Fulkerson* allié à ce parcours en largeur pour calculer un flot maximal dans un graphe (cf GRO-2) ;
- l'algorithme de *Dijkstra* est une généralisation de ce parcours lorsque les distances sont données par une capacité (positive) ; il permet aussi de donner explicitement les chemins les plus courts (l'algorithme précédent ne donne que la distance la plus courte entre s et i et doit être complété si on veut aussi un chemin associé).

Annexe A

Correction des mini-exercices

A.1 mini-exercice 1

On pose $X = (X_{1f}, X_{1c}, X_{2f}, X_{2c}, X_{3f}, X_{3c})$, la fonction à minimiser est donc :

$$f(X) = C^\top X, \text{ avec } C = (1000, 1000, 2000, 2000, 1200, 1200)$$

Pour se ramener à des inégalités avec les variables à gauche et un signe $\leq$, on multiplie les inégalités correspondantes ((1.2) et (1.3)) par -1 . On a donc ;

$$\begin{aligned} -100X_{1f} - 200X_{2f} - 150X_{3f} &\leq -1500 \\ -100X_{1c} - 200X_{2c} - 150X_{3c} &\leq -3500 \end{aligned}$$

Les 3 autres inégalités sont dans le “bon” sens. Ces 5 inégalités se mettent alors sous la forme :

$$AX \leq b \text{ avec } A = \begin{pmatrix} -100 & 0 & -200 & 0 & -150 & 0 \\ 0 & -100 & 0 & -200 & 0 & -150 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}, b = \begin{pmatrix} -1500 \\ -3500 \\ 10 \\ 22 \\ 10 \end{pmatrix}$$

A.2 mini-exercice 2

Ce graphe comporte 10 sommets $S = \llbracket 0, 9 \rrbracket$, et les 18 arêtes

$$\Gamma = \{\{0, 1\}, \{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 5\}, \{5, 0\}, \{0, 6\}, \{1, 6\}, \{2, 6\}, \\ \{3, 6\}, \{4, 6\}, \{5, 6\}, \{5, 7\}, \{7, 8\}, \{7, 9\}, \{1, 8\}, \{8, 9\}, \{3, 9\}\}$$

Le degré maximal est obtenu pour le sommet 6, $\Delta G = 6$. Enfin, il est dit que la coloration du dessin est optimale soit $\chi(G) = 4$.

A.3 mini-exercice 3

On pose $x'_2 = x_2 - 1$ (pour se ramener à une variable positive) et $x_3 = y - z$ avec $y \geq 0$ et $z \geq 0$. Enfin il faut rajouter une variable d'écart e pour transformer l'inégalité en égalité. On obtient :

1. Pour la fonction objectif :

$$F(x_1, x_2, x_3) = 2x_1 + 5(x'_2 + 1) + 9(y - z) + 0e = 2x_1 + 5x'_2 + 9y - 9z + 0e - 5$$

2. Pour les contraintes :

$$\begin{aligned} 3x_1 + 7(x'_2 + 1) + 3(y - z) + e &= 12 & \iff & 3x_1 + 7x'_2 + 3y - 3z + e = 5 \\ 4x_1 + 5(x'_2 + 1) + 6(y - z) &= 15 & \iff & 4x_1 + 5x'_2 + 6y - 6z + 0e = 10 \end{aligned}$$

En notant $X := (x_1, x'_2, y, z, e)$, $C := (2, 5, 9, -9, 0)$, $b := (5, 10)$, le PL initial est équivalent au PL en forme standard suivant :

$$\begin{cases} \max_X \tilde{F}(X) = C^\top X \\ X \geq 0, AX = b \end{cases} \quad \text{avec } A = \begin{bmatrix} 3 & 7 & 3 & -3 & 1 \\ 4 & 5 & 6 & -6 & 0 \end{bmatrix}$$

Rmq : supposons qu'il y ait une solution optimale $(X^*, \tilde{F}(X^*))$, alors il est facile de revenir aux variables initiales $(x_1^* = X_1^*, x_2^* = X_2^* - 1, x_3^* = X_3^* - X_4^*)$ et de plus $F(x_1^*, x_2^*, x_3^*) = \tilde{F}(X^*) - 5$.

A.4 mini-exercice 4

Soit donc $\mathcal{H} = \{x \in \mathbb{R}^n : \sum_{i=1}^n a_i x_i = b\}$, x et y deux points quelconque de $\mathcal{H}$ et $\lambda \in [0, 1]$, montrons que $z := (1 - \lambda)x + \lambda y \in \mathcal{H}$. z a pour coordonnées $z_i = (1 - \lambda)x_i + \lambda y_i$. z vérifie-t-il l'équation ? On a :

$$\sum_{i=1}^n a_i z_i = \sum_{i=1}^n a_i ((1 - \lambda)x_i + \lambda y_i) = \sum_{i=1}^n a_i (1 - \lambda)x_i + \sum_{i=1}^n a_i \lambda y_i = (1 - \lambda) \underbrace{\sum_{i=1}^n a_i x_i}_{=b} + \lambda \underbrace{\sum_{i=1}^n a_i y_i}_{=b} = b$$

Soit donc $\mathcal{D} = \{x \in \mathbb{R}^n : \sum_{i=1}^n a_i x_i \leq b\}$, x et y deux points quelconque de $\mathcal{D}$ et $\lambda \in [0, 1]$, montrons que $z := (1 - \lambda)x + \lambda y \in \mathcal{D}$. z a pour coordonnées $z_i = (1 - \lambda)x_i + \lambda y_i$. z vérifie-t-il l'inéquation ? Le même calcul que ci-dessus conduit à :

$$\sum_{i=1}^n a_i z_i = \underbrace{(1 - \lambda) \sum_{i=1}^n a_i x_i}_{\substack{\geq 0 \\ \leq b}} + \underbrace{\lambda \sum_{i=1}^n a_i y_i}_{\substack{\geq 0 \\ \leq b}} \leq (1 - \lambda)b + \lambda b = b$$

A.5 mini-exercice 5

D'après le dessin, on doit être :

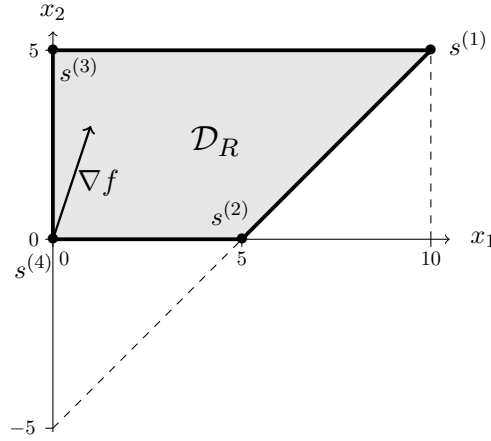
1. "au-dessus" de la droite passant par $(0, 0)$ et $(4, 0)$, c'est à dire $x_2 \geq 0$ ou encore $-x_2 \leq 0$;
2. "à gauche" (ou "au-dessus") de la droite passant par $(4, 0)$ et $(6, 3)$; cette droite a pour pente $3/2$ son équation est $x_2 = \frac{3}{2}(x_1 - 4) \iff x_1 = -\frac{2}{3}x_2 + 4$. Comme le polygône se trouve "à gauche", l'équation du demi-plan est donc $x_1 \leq \frac{2}{3}x_2 + 4$ soit encore $x_1 - \frac{2}{3}x_2 \leq 4$ (Remarque : on retrouve bien l'orientation obtenue avec la normale à la droite limite $= (1, -\frac{2}{3})$ puisque le demi-plan doit se situer dans le sens contraire (du fait du ≤ 4)).
3. "en dessous" (ou "à gauche") de la droite passant par $(3, 6)$ et $(6, 3)$ dont l'équation est $x_1 + x_2 = 9$. Ce qui nous donne $x_1 + x_2 \leq 9$.
4. "en dessous" (ou "à droite") de la droite passant par $(0, 0)$ et $(3, 6)$ dont l'équation est $x_2 = 2x_1$ (pente de 2). Ce qui nous donne $x_2 \leq 2x_1$, soit $-2x_1 + x_2 \leq 0$.

Ces 4 inéquations se mettent sous la forme :

$$\begin{pmatrix} 0 & -1 \\ 1 & -\frac{2}{3} \\ 1 & 1 \\ -2 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \leq \begin{pmatrix} 0 \\ 4 \\ 9 \\ 0 \end{pmatrix}$$

A.6 mini-exercice 6

1. On obtient le polygône suivant qui comporte 4 sommets $s^{(1)}$, $s^{(2)}$, $s^{(3)}$ et $s^{(4)}$:



2. La formulation standard. Il faut rajouter deux variables d'écart pour transformer les deux inéquations en équations :

$$\begin{cases} x_1 & -x_2 & +e_1 & & = & 5 \\ & x_2 & & +e_2 & = & 5 \end{cases} \iff \underbrace{\begin{pmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}}_A X = \underbrace{\begin{pmatrix} 5 \\ 5 \end{pmatrix}}_b$$

avec $X = (x_1, x_2, e_1, e_2)$. Enfin $C = (1, 3, 0, 0)$.

3. Ici $m = 2$ (nombre de contraintes d'égalité) et $n = 4$ (nombre de variables) a priori il y a les $C_4^2 = 6$ choix possibles suivants pour une base : $\{1, 2\}$, $\{1, 3\}$, $\{1, 4\}$, $\{2, 3\}$, $\{2, 4\}$ et $\{3, 4\}$.

- $\{1, 2\}$ est bien une base puisque les vecteurs

$$A^1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, A^2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

sont bien linéairement indépendants), appelons-là B_1 . Les variables de base sont x_1 et x_2 , les variables hors-base e_1 et e_2 . La solution de base associée est obtenue en imposant les variables hors-base à être nulle, soit $e_1^{(1)} = 0$ et $e_2^{(1)} = 0$, ce qui nous donne (pour calculer les composantes en base) le système suivant à résoudre :

$$\begin{cases} x_1^{(1)} & -x_2^{(1)} & = & 5 \\ & x_2^{(1)} & = & 5 \end{cases} \iff \begin{cases} x_1^{(1)} & = & 10 \\ x_2^{(1)} & = & 5 \end{cases}$$

Les composantes étant toutes positives ou nulles, on obtient bien une solution de base réalisable : $X^{(1)} = (10, 5, 0, 0)$. Sa projection (orthogonale sur les 2 premiers axes) correspond au sommet $s^{(1)}$ ($\mathcal{P}(X^{(1)}) = s^{(1)}$).

- $\{1, 3\}$ n'est pas une base ($A^1 = A^3$).
- $\{1, 4\}$ est bien une base (les vecteurs A^1 et A^4 forment une matrice identité), appelons-là B_2 . Les variables de base sont x_1 et e_2 . Les variables hors-base x_2 et e_1 , la solution de base associée à B_2 est obtenue en leur donnant la valeur 0 ($x_2^{(2)} = 0$, $e_1^{(2)} = 0$), on obtient directement :

$$\begin{cases} x_1^{(2)} & = & 5 \\ e_2^{(2)} & = & 5 \end{cases}$$

Les composantes étant toutes positives ou nulles, on obtient bien une solution de base réalisable : $X^{(2)} = (5, 0, 0, 5)$. De même $\mathcal{P}(X^{(2)}) = s^{(2)}$.

- $\{2, 3\}$ est bien une base (les vecteurs A^2 et A^3 sont bien linéairement indépendants), appelons-là B_3 . Les variables de base sont x_2 et e_1 . Les variables hors-base x_1 et e_2 , la solution de base associée à B_3 est obtenue en leur donnant la valeur 0 ($x_1^{(3)} = 0, e_2^{(3)} = 0$). Pour trouver la valeur des variables en base, on résout le système :

$$\begin{cases} -x_2^{(3)} + e_1^{(3)} = 5 \\ x_2^{(3)} = 5 \end{cases} \iff \begin{cases} e_1^{(3)} = 10 \\ x_2^{(3)} = 5 \end{cases}$$

Les composantes étant toutes positives ou nulles, on obtient bien une solution de base réalisable : $X^{(3)} = (0, 5, 10, 0)$ et $\mathcal{P}(X^{(3)}) = s^{(3)}$.

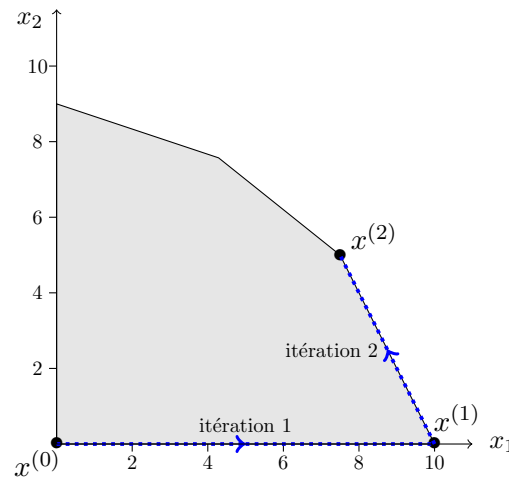
La suite de la correction est abrégée (même raisonnements).

- $\{3, 4\}$ est bien une base (les vecteurs A^3 et A^4 forment une matrice identité), appelons-là B_4 . Les variables de base sont e_1 et e_2 . Les variables hors-base x_1 et x_2 , la solution de base associée à B_4 est $X^{(4)} = (0, 0, 5, 5)$, elle est bien réalisable et $\mathcal{P}(X^{(4)}) = s^{(4)}$.
- Enfin $\{2, 4\}$ est aussi une base appelons-là B_5 . Les variables de base sont x_2 et e_2 . Les variables hors-base x_1 et e_1 , la solution de base associée à B_5 est $X^{(5)} = (0, -5, 0, 10)$, **elle n'est pas réalisable** et ne fait donc pas partie du domaine.

A.7 mini-exercice 7

On note $x^{(k)}$ la projection de $X^{(k)}$ sur les 2 premières composantes. On a donc

$$x^{(0)} = (0, 0); x^{(1)} = (10, 0); x^{(2)} = \left(\frac{15}{2}, 5\right) = x^*$$



A.8 mini-exercice 8

$$(S_0), (), (S_1), (), (S_2), ()$$

La file de priorité contient initialement le sommet S_0 qui est retiré de la file pour la création de ses enfants mais seul l'enfant gauche (S_1) est créé et mis dans la file. S_1 est ensuite retiré de la file pour la création de ses enfants et seul l'enfant gauche S_2 est créé et mis dans la file puis retiré pour créer ses enfants mais aucune création n'a lieu, fin de la recherche.